

João Pedro Coelho Encarnação

Modelo Físico e Dinâmico de Simulação de Árvore Brônquica para Treino de Broncoscopia



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

João Pedro Coelho Encarnação

Modelo Físico e Dinâmico de Simulação de Árvore Brônquica para Treino de Broncoscopia

Relatório de Projeto Final de Licenciatura
Licenciatura em Engenharia de Eletrotecnia e Computadores

Trabalho efetuado sob orientação do:
Professor Jorge Semião



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

Dedicatória

Dedico este trabalho aos meus pais, Anabela e António Encarnação, e à minha irmã, Ana Sofia, por me apoiarem nos momentos mais difíceis e por nunca terem duvidado de mim.

Sem eles, nunca teria chegado até aqui, nem seria quem sou hoje.

Agradecimentos

Deixo aqui o meu maior e mais sincero agradecimento ao meu orientador, Professor Jorge Semião, por me ter dado a oportunidade de trabalhar num projeto tão desafiante e interessante, por ter demonstrado sempre disponibilidade para ajudar e por todo o apoio prestado ao longo do desenvolvimento deste trabalho.

Agradeço também à equipa médica envolvida no projeto, pela disponibilidade demonstrada, pelas reuniões realizadas e pelo contributo fundamental na validação anatómica e funcional do modelo desenvolvido. As observações e sugestões recebidas permitiram aproximar o trabalho das necessidades reais associadas ao treino de broncoscopia.

Gostaria ainda de agradecer a todos os professores, colegas e amigos da Licenciatura em Engenharia de Eletrotécnica e Computadores, por todo o apoio, pela entreaajuda e pela presença ao longo deste percurso académico.

Foi um privilégio e uma honra conviver e aprender com todos vós.

Resumo

O presente relatório descreve o desenvolvimento de um modelo articulado de uma árvore brônquica para apoio à simulação e ao treino de videobroncofibroscopia, um trabalho que parte da necessidade de criar uma plataforma física de treino capaz de representar, de forma controlada e realista, os movimentos internos observáveis durante o procedimento, combinando técnicas como a impressão 3D, a atuação mecânica, o controlo eletrónico, o software embarcado e a interface homem-máquina.

A solução foi organizada em duas partes complementares. O Modelo Físico, constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servomotores e pelo microcontrolador ESP32-S3, executa fisicamente os movimentos, enquanto a HMI, alojada num Raspberry Pi 4B, é responsável por permitir ao utilizador controlar o sistema de forma simples e intuitiva, sem necessidade de memorizar comandos internos ou valores.

No Modelo Físico foram implementados vários perfis de movimento, nomeadamente a respiração, o batimento cardíaco e a tosse, recorrendo a estruturas de trajetórias, a máquinas de estados, a comunicação série e a equações de cinemática inversa para converter as posições desejadas nos ângulos dos manipuladores. A interface gráfica, por sua vez, permite selecionar modos de funcionamento e estados de operação, como iniciar ou interromper o ensaio, além de disponibilizar outras funcionalidades, entre as quais estabelecer e acompanhar a comunicação série com o ESP32 e preparar os parâmetros de movimento, colocando à disposição dos operadores um conjunto alargado de ferramentas.

Ao longo de todo o desenvolvimento foram validados, testados e modificados os vários blocos fundamentais de ambos os sistemas, o Modelo Físico e a HMI, identificando-se no final as principais limitações do protótipo atual e as sugestões de melhoria a considerar no futuro.

Índice Geral

	Página
Lista de Abreviaturas e Termos	ix
Lista de Figuras	xii
Lista de Tabelas	xiv
1 Introdução	1
1.1 Enquadramento	1
1.2 Motivação	2
1.3 Objetivos	2
1.4 Metodologia	3
1.5 Organização do relatório	3
2 Estado da Arte	4
2.1 Broncoscopia e treino por simulação	4
2.2 Fidelidade dos simuladores	5
2.3 Simuladores de broncoscopia	6
2.4 Impressão 3D aplicada à simulação médica	7
2.4.1 Técnicas de impressão 3D	7
2.4.2 Aplicação em simuladores de árvores brônquicas	8
2.5 Movimentos observáveis durante a broncoscopia	9
2.5.1 Movimento respiratório	10
2.5.2 Movimento cardíaco	11
2.5.3 Tosse e perturbações rápidas	11
2.5.4 Interação com o broncoscópio	12
2.5.5 Patologias e deformações locais	12
2.6 Síntese crítica	13
3 Desenvolvimento do Hardware	15
3.1 Requisitos do sistema	15
3.2 Impressão 3D da árvore brônquica	16

3.2.1	Da tomografia ao modelo STL	17
3.2.2	Programas utilizados no tratamento do modelo	17
3.2.3	Seleção de materiais	19
3.2.4	Disposição do modelo para impressão	20
3.2.5	Evolução do processo de impressão 3D	24
3.3	Seleção do sistema de atuação	26
3.4	Eletrônica de controlo e alimentação	28
3.4.1	Testes iniciais de eletrónica	29
3.4.2	Gestão de energia do Raspberry Pi	30
3.4.3	Arquitetura e dimensionamento da fonte de alimentação . . .	30
3.4.4	Da protoboard à PCB	31
3.5	Integração mecânica	31
4	Desenvolvimentos do Software e Configurações	34
4.1	Arquitetura geral do software e ambiente de desenvolvimento	34
4.2	Arquitetura do firmware ESP32-S3	35
4.3	Movimentos representados no protótipo	40
4.4	Geração de trajetórias (Python/Raspberry Pi)	41
4.4.1	Desenvolvimento e iterações	42
4.4.2	Módulo final	43
4.5	Estruturas de armazenamento de dados	50
4.6	Máquinas de estados e execução dos movimentos	52
4.6.1	Desenvolvimento e iterações	52
4.6.2	Módulo final	56
4.7	Preparação e sincronização da tosse	60
4.8	Cinemática inversa do manipulador delta	62
4.8.1	Desenvolvimento e iterações	64
4.8.2	Módulo final	66
4.9	Comunicação série	71
4.9.1	Comunicação série no firmware (ESP32)	71
4.9.1.1	Desenvolvimento e iterações	72
4.9.1.2	Módulo final	72
4.9.2	Comunicação série no Python (Raspberry Pi)	80
4.10	Interface HMI	81
4.10.1	Escolha da tecnologia e evolução até à versão atual	81
4.10.2	Arquitetura da aplicação final	83
4.10.3	Funcionalidades principais	86
4.10.4	Configurações de movimentos	86
5	Testes e Análise de Resultados	89

5.1	Teste de Laboratório	89
5.2	Testes clínicos	95
6	Conclusão	98
6.1	Síntese do trabalho	98
6.2	Limitações	99
6.3	Trabalho futuro	100
	Bibliografia	101
	Apêndice	107
	Anexos	108

Lista de Abreviaturas e Termos

CA Corrente Alternada; tipo de alimentação elétrica ligada à entrada da fonte de alimentação do sistema.

CAD *Computer-Aided Design*; desenho assistido por computador, usado no tratamento do modelo 3D da árvore brônquica.

CHUA Centro Hospitalar Universitário do Algarve; instituição parceira que forneceu os dados clínicos e anatômicos usados no projeto.

DEV *Development*; modo de utilização da HMI que dá acesso a ferramentas de debugging e de envio manual de comandos.

DTR *Data Terminal Ready*; sinal de controlo da porta série que pode provocar o reset do ESP32-S3.

EEPROM *Electrically Erasable Programmable Read-Only Memory*; memória não volátil usada para guardar configurações persistentes do firmware.

ESP32-S3 Microcontrolador usado para executar o firmware, interpretar comandos e gerar os sinais de controlo.

FDM *Fused Deposition Modeling*; tecnologia de impressão 3D por extrusão de filamento termoplástico, usada na maioria das peças do projeto.

FSM *Finite State Machine*; máquina de estados finitos usada para organizar a lógica de movimento, leitura série e interpretação de comandos.

GPIO *General Purpose Input/Output*; pinos digitais usados para ligação a periféricos.

HAT *Hardware Attached on Top*; placa de expansão para Raspberry Pi, acoplada aos pinos GPIO para adicionar ferramentas, periféricos ou capacidade de processamento.

HMI *Human Machine Interface*; interface homem-máquina usada para permitir ao utilizador controlar o modelo físico através do Raspberry Pi.

IK *Inverse Kinematics*; cinemática inversa, usada para converter coordenadas cartesianas em ângulos dos servos.

JSON *JavaScript Object Notation*; formato usado para guardar as configurações dos movimentos definidas na interface.

LiPo Bateria de polímero de lítio.

MG90S Servo motor de engrenagens metálicas com um grau de liberdade de aproximadamente 180°.

PCB *Printed Circuit Board*; placa de circuito impresso.

PEBA Filamento de impressão 3D de polieter-bloco-amida, elastómero termoplástico flexível e leve.

PETG *Polyethylene Terephthalate Glycol*; filamento de impressão 3D rígido, compatível com o processo FDM.

PLA Filamento de impressão 3D de ácido polilático, usado sobretudo pela facilidade de impressão e rigidez.

PVA *Polyvinyl Alcohol*; filamento de impressão 3D solúvel em água, considerado como material de suporte.

PWM *Pulse Width Modulation*; técnica usada para controlar os servos através da largura de pulso.

QR *Quick Response*; código usado para aceder rapidamente à interface móvel a partir de um telemóvel ou tablet.

RGB *Red, Green, Blue*; sistema de cor usado no LED de indicação de estado.

RTS *Request To Send*; sinal de controlo da porta série que pode provocar o reset do ESP32-S3.

SLA *Stereolithography*; tecnologia de impressão 3D por cura de resina fotopolimérica com luz ultravioleta.

SLS *Selective Laser Sintering*; tecnologia de impressão 3D por sinterização seletiva de material em pó.

STL Formato de ficheiro de malha 3D usado para representar a geometria de superfície da árvore brônquica antes da impressão.

TAC Tomografia Axial Computorizada; tecnologia de imagem médica usada para obter o modelo tridimensional da árvore brônquica.

TPC Filamento de impressão 3D de copoliéster termoplástico, material flexível usado em peças elásticas e funcionais.

TPE Filamento de impressão 3D de elastómero termoplástico, família de materiais com comportamento elástico semelhante a borracha.

TPU Filamento de impressão 3D de poliuretano termoplástico, usado em peças flexíveis e elásticas.

UART *Universal Asynchronous Receiver/Transmitter*; interface de comunicação série usada entre o Raspberry Pi e o ESP32-S3.

UPS *Uninterruptible Power Supply*; sistema de alimentação que mantém o equipamento ligado durante falhas ou interrupções temporárias da alimentação principal.

USB-C *Universal Serial Bus Type-C*; conector universal usado para alimentação e transferência de dados.

Lista de Figuras

2.1	Modelos de treino em broncoscopia [13].	5
2.2	Modelos brônquicos impressos em multi-material [26].	9
2.3	Modelo computacional do movimento respiratório [29].	10
3.1	Tratamento da malha da árvore brônquica no Fusion 360.	19
3.2	Escala de dureza Shore aplicada a filamentos flexíveis, com exemplos de objetos do dia a dia em pontos equivalentes da escala (fonte: Recreus). . .	20
3.3	Modelo impresso com suportes em PLA.	25
3.4	Modelo final da árvore brônquica impresso em material flexível.	26
3.5	Teste inicial de atuador local com suporte impresso.	27
3.6	Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.	29
3.7	Sistema mecânico final com os dois manipuladores delta.	33
4.1	Esquema global de comunicação entre o firmware do ESP32-S3 e a HMI no Raspberry Pi 4B.	34
4.2	Estrutura global das máquinas de estados e blocos de controlo do firmware. . .	37
4.3	Desenvolvimento da arquitetura principal do ESP32.	39
4.4	Trajectoria calculada para o movimento de batimento cardíaco, repre- sentação 2D dos pontos de passagem.	45
4.5	Trajectoria calculada para o movimento de batimento cardíaco, repre- sentação 3D dos pontos de passagem.	45
4.6	Trajectoria calculada para o movimento respiratório, representação 2D no plano ZX.	47
4.7	Trajectoria calculada para o movimento respiratório, visualização 3D.	47
4.8	Trajectoria calculada para a vibração associada à tosse, representação 2D dos pontos de passagem.	48
4.9	Trajectoria calculada para a vibração associada à tosse, representação 3D dos pontos de passagem.	49
4.10	Estrutura <code>Manipulador_Config</code>	51
4.11	Estruturas <code>MotionStorage</code> e <code>MotionInstance</code>	52
4.12	Andamento variável obtido apenas por densidade de pontos.	53

4.13	Comparação entre a interpolação linear e a interpolação polinomial testadas.	55
4.14	Sequência final de processamento, repetida a cada ciclo e para cada manipulador.	55
4.15	Esquema da <code>FSM_Motion_Update</code>	58
4.17	Esquema da <code>FSM_preparacao_tosse</code>	62
4.16	Fluxograma da função <code>AtualizarTosseAleatoria()</code>	63
4.18	Convenção de ângulo referenciada ao horizonte exterior do manipulador, usada na <i>Delta-Kinematics-Library</i> [65].	65
4.19	Terminologia usada para os componentes do manipulador delta: base, braço, antebraço e plataforma móvel.	66
4.20	Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.	67
4.21	Distribuição a 120° dos três servos em torno do centro da base do manipulador delta.	68
4.22	Projeção do braço passivo L_2 no plano lateral do servo, resultante do deslocamento lateral da plataforma móvel.	69
4.23	Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.	70
4.24	Esquema da <code>FSM_Serial_Reader</code>	74
4.25	<code>FSM_Serial_Command</code> (parte 1 de 3): comandos simples e submenu L.	77
4.26	<code>FSM_Serial_Command</code> (parte 2 de 3): submenu M e entrada no comando longo W.	78
4.27	<code>FSM_Serial_Command</code> (parte 3 de 3): seleção de campo, escrita do valor e comandos de EEPROM.	79
4.28	Ecrã principal da HMI no Raspberry Pi.	82
4.29	Esquema global dos ficheiros e clientes que interagem com a interface HMI.	83
4.30	Esquema interno do <code>RB_PI4B_Main_PI.py</code>	85
4.31	Esquema interno do <code>calculo_movimentos.py</code>	88
5.1	Comparação entre deslocamento pretendido e deslocamento obtido no ensaio linear do manipulador delta.	91
5.2	Tamanho dos saltos medidos entre posições consecutivas no ensaio linear do manipulador delta.	92
5.3	Comparação entre a frequência de batimento cardíaco configurada e a frequência medida no vídeo do ensaio.	94
5.4	Comparação entre a frequência respiratória configurada e a frequência medida no vídeo do ensaio.	95
5.5	Ensaio clínico no CHUA.	96

Lista de Tabelas

4.1	Movimentos considerados para o projeto.	40
4.2	Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.	49
4.3	Pontos calculados para os movimentos usados nos testes da interface.	50
4.4	Comandos simples disponíveis na comunicação série.	75
4.5	Subcomandos do menu de leitura, teste e depuração.	75
4.6	Subcomandos de seleção do modo fisiológico.	76
4.7	Etiquetas dos comandos longos e variáveis internas correspondentes.	76
5.1	Resultados do ensaio de deslocamento linear do manipulador delta nos eixos X , Y e Z	90
5.2	Resultados do ensaio temporal do movimento de batimento cardíaco.	93
5.3	Resultados do ensaio temporal do movimento de respiração.	94
A.1	Componentes principais previstos para o sistema.	107

Capítulo 1

Introdução

Este documento descreve o trabalho desenvolvido no âmbito de um projeto final de curso, dedicado à conceção de um modelo físico e dinâmico de árvore brônquica para treino de broncoscopia, apresentando de seguida o contexto que motivou esta escolha, os objetivos traçados e a forma como o presente relatório se encontra organizado.

1.1 Enquadramento

A broncoscopia flexível é um procedimento essencial em Pneumologia, Anestesiologia, Medicina Intensiva e Cirurgia Torácica, cuja execução exige coordenação psicomotora, reconhecimento anatómico, capacidade de navegação broncoscópica e tomada de decisão num ambiente clínico variável. Por esse motivo, a simulação tem sido proposta como uma etapa relevante no treino de novos médicos na área, permitindo suavizar a curva de aprendizagem, possibilitar treinos repetidos e aumentar a segurança antes da prática em doentes reais [1–5].

O projeto apresentado neste relatório teve origem numa proposta de trabalho elaborada pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [6], que identificou a necessidade de um modelo de simulação de árvore brônquica, impresso em 3D e de baixo custo, para treino de broncoscopia flexível. A partir dessa proposta, o objetivo deste trabalho passou a ser o desenvolvimento de um modelo articulado de uma árvore brônquica, capaz de reproduzir, de forma legítima, os movimentos observáveis durante as broncoscopias, partindo da utilidade já reconhecida dos modelos físicos impressos em 3D e acrescentando-lhes uma componente dinâmica, de modo a aproximar a experiência de treino das condições reais de observação broncoscópica [7, 8].

O projeto foi dividido em duas secções complementares:

- O Modelo Físico, que inclui todo o sistema de controlo e atuação motora, dividido em vários manipuladores delta, bem como o próprio modelo 3D da árvore brônquica;
- A HMI, ou interface homem-máquina, alojada num Raspberry Pi 4B com um mo-

nitor touchscreen, cuja função é permitir que o utilizador controle o modelo físico de forma simples, intuitiva e natural, através de botões e menus dedicados, sem necessidade de conhecer os comandos da comunicação série entre os dois blocos.

1.2 Motivação

Existem vários simuladores comerciais de broncoscopia, contudo estes facilmente podem apresentar custos monetários extremamente elevados, o que limita a sua disponibilidade e acessibilidade em contextos de ensino e treino, razão pela qual a impressão 3D surge como uma forte alternativa, ao permitir criar modelos anatómicos acessíveis, personalizáveis e adaptáveis a diferentes níveis de dificuldade [7, 8]. Contudo, muitos destes modelos físicos permanecem estáticos, falhando na representação de fenómenos como a deslocação respiratória das vias aéreas, a variação de calibre dos brônquios, as pequenas oscilações induzidas pelo coração e as perturbações rápidas associadas à tosse, resultantes da resposta do corpo humano à inserção de um corpo externo nos brônquios [9–12].

A motivação central deste trabalho consistiu, portanto, na capacidade de combinar a acessibilidade e a versatilidade dos modelos físicos impressos em 3D com a possibilidade de implementação de movimentos anatómicos legítimos, comprovando assim a viabilidade de um simulador dinâmico e mecanicamente atuado, uma abordagem que, tanto quanto foi possível apurar junto da literatura disponível, ainda não se encontra documentada.

1.3 Objetivos

Este projeto pretende investigar, desenvolver, construir e documentar um protótipo articulado de uma árvore brônquica para treino de videobroncofibroscopia.

Os seus objetivos específicos são:

- Estudar requisitos anatómicos, mecânicos e pedagógicos para os simuladores de broncoscopia;
- Desenvolver um plano e uma estrutura física compatível com impressão 3D e atuação mecânica;
- Integrar sensores ou atuadores e todo um sistema de eletrónica de controlo flexível, para os vários cenários de treino;
- Implementar firmware e configurações pré-feitas de controlo para os diferentes perfis de movimento;
- Desenvolver uma interface, HMI, para um controlo simples e intuitivo do modelo;

- Definir uma estratégia de testes para avaliação da usabilidade, da repetibilidade e da robustez dos sistemas, bem como verificação do sincronismo dos movimentos em termos de tempo e de espaço percorrido pelos manipuladores robóticos;
- Documentar as decisões de projeto, limitações e possíveis melhorias futuras.

1.4 Metodologia

Este projeto seguiu uma metodologia iterativa de desenvolvimento, começando por uma revisão bibliográfica alargada e pela definição dos requisitos anatómicos, mecânicos e pedagógicos do simulador. A primeira metade do tempo dedicado ao projeto centrou-se no modelo físico da árvore brônquica, a peça impressa em 3D construída e refinada ao longo de várias iterações, validadas em conjunto com a equipa médica responsável pelo projeto, um investimento de tempo que se justifica pela quantidade de tentativas, materiais e geometrias necessárias até se alcançar uma solução anatomicamente fiel e mecanicamente viável. Só a partir daí o restante hardware e o software foram desenvolvidos, ocupando a segunda metade do projeto, em paralelo e por camadas sucessivas, com protótipos intermédios a validar decisões de projeto antes de avançar para a fase seguinte, permitindo corrigir limitações identificadas em cada etapa em vez de as arrastar até ao final do processo. Esta abordagem culminou numa fase de testes e de validação experimental do protótipo final, descrita em detalhe no capítulo 5.

1.5 Organização do relatório

O relatório está organizado em seis capítulos. O primeiro apresenta o enquadramento, a motivação e os objetivos do projeto, enquanto o segundo reúne a revisão do estado da arte, abordando o treino por simulação, a impressão 3D e os movimentos observáveis durante a broncoscopia. O terceiro capítulo descreve o desenvolvimento do hardware, seguindo-se no quarto capítulo a apresentação da arquitetura de software, dos movimentos representados no protótipo, da cinemática inversa, das máquinas de estados, da comunicação série e de todo o sistema por detrás da HMI. O quinto capítulo apresenta os testes realizados ao modelo físico e à integração dos movimentos, terminando o relatório no sexto capítulo, onde se sintetizam as conclusões e as melhorias futuras a considerar.

Capítulo 2

Estado da Arte

Antes de se justificar o desenvolvimento de um modelo físico e dinâmico de árvore brônquica, importa perceber onde se situa este trabalho relativamente ao que já existe em simulação médica para broncoscopia, pelo que este capítulo percorre o estado atual do conhecimento nesta área, desde o papel da simulação na aprendizagem da broncoscopia e o conceito de fidelidade que permite comparar diferentes simuladores entre si, até ao contributo da impressão 3D na criação de modelos anatómicos e aos principais movimentos fisiológicos observáveis durante o procedimento, que servem de referência direta ao trabalho desenvolvido nos capítulos seguintes.

2.1 Broncoscopia e treino por simulação

A broncoscopia flexível é um procedimento médico endoscópico que permite observar diretamente a árvore traqueobrônquica, recolher amostras, auxiliar diagnósticos e apoiar intervenções terapêuticas. A aprendizagem deste procedimento exige coordenação entre visão broncoscópica, manipulação fina do broncoscópio e interpretação anatómica. A simulação pormenorizada é, por isso, uma ferramenta relevante para repetir tarefas, treinar orientação espacial e reduzir a dependência inicial de treino em doentes reais [1–4].

As revisões e estudos de treino em broncoscopia apontam para ganhos na aquisição de competências quando os simuladores são integrados de forma estruturada no ensino. O valor pedagógico de um simulador depende não só do realismo visual, mas também da relação entre o exercício proposto, os objetivos de aprendizagem e a forma como o desempenho é avaliado [2, 3, 5].

Trabalhos mais recentes procuram comparar diretamente diferentes modalidades de treino entre si. Um estudo dinamarquês comparou quatro modelos distintos de treino em broncoscopia, incluindo modelos físicos e virtuais, concluindo que os modelos físicos de baixo custo se mantêm competitivos face a alternativas tecnologicamente mais sofisticadas e dispendiosas [13], enquanto outro estudo mostrou que sessões curtas de treino guiadas por um simulador produzem resultados de aprendizagem semelhantes aos obtidos sob

orientação direta de um tutor humano, reforçando o valor pedagógico da simulação mesmo sem supervisão constante [14].

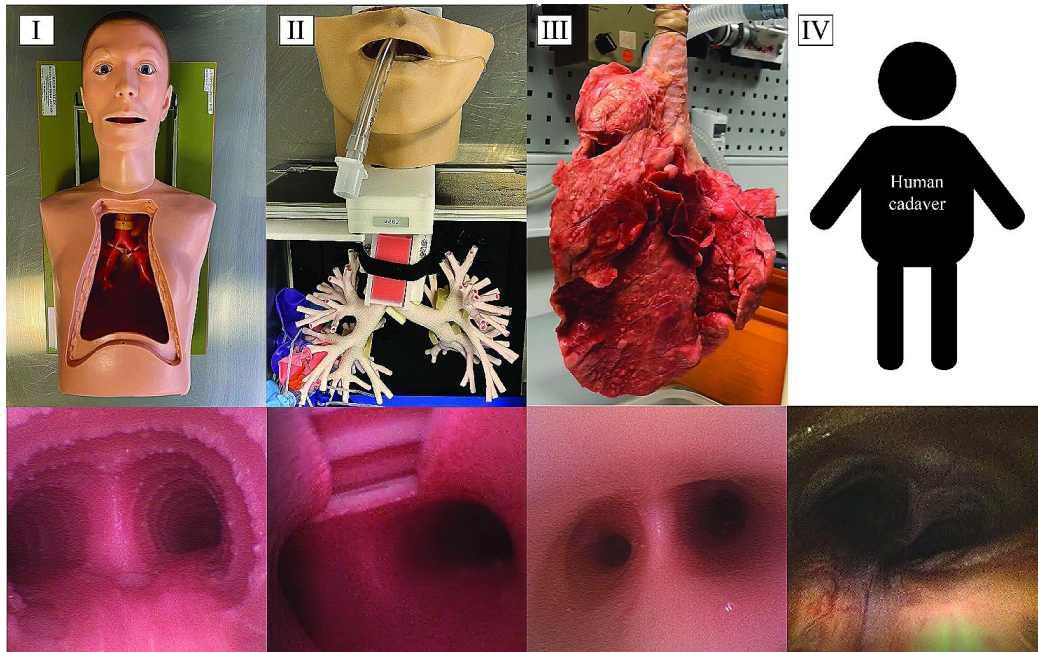


Figura 2.1: Modelos de treino em broncoscopia [13].

2.2 Fidelidade dos simuladores

A fidelidade de um simulador pode ser entendida em várias dimensões:

- Fidelidade anatómica → refere-se à correspondência entre a geometria do modelo e a anatomia real, como à forma, proporção e organização espacial das vias aéreas;
- Fidelidade mecânica → refere-se à resistência ao avanço, flexibilidade e resposta ao contacto;
- Fidelidade funcional → refere-se à capacidade de reproduzir movimentos, variações de calibre e eventos transitórios como a tosse.

Esta distinção aproxima-se da lógica proposta por Miller para a avaliação de competências clínicas, que separa aquilo que um profissional sabe, sabe fazer, demonstra saber fazer e efetivamente faz na prática, situando os simuladores físicos precisamente na transição entre o demonstrar e o fazer [15]. Importa, no entanto, notar que a relação entre fidelidade e aprendizagem não é necessariamente linear, havendo trabalhos recentes que questionam se um maior realismo se traduz sempre em mais competência adquirida, sugerindo que o envolvimento ativo do formando pode pesar tanto quanto o rigor da réplica anatómica [16].

Na broncoscopia, um modelo estático pode ser útil para situações de treinos iniciais de navegação e reconhecimento anatômico. No entanto, durante um procedimento real, o operador encontra-se num ambiente vivo e dinâmico, em constante movimento. A respiração, os batimentos cardíacos, a tosse, a sedação e a própria interação do broncoscópio com as paredes internas exercem comportamentos e reações que modificam continuamente o ambiente e a sensação de controlo. Esta diferença cria oportunidade para os simuladores físicos de baixo custo que incorporem movimentos e fisiologia de forma controlada [1, 4, 8].

2.3 Simuladores de broncoscopia

Os atuais simuladores de broncoscopia recorrem a vários tipos de sistemas, nomeadamente treinos em ambientes puramente virtuais, em manequins comerciais, em modelos de bancada e, mais recentemente, em modelos impressos em 3D.

Os sistemas virtuais, como o simulador ORSIM, permitem gerar cenários de dificuldade variável e obter métricas automáticas de desempenho, tendo demonstrado boa validade e fiabilidade na avaliação de competências em broncoscopia flexível, distinguindo de forma consistente entre principiantes e peritos [17]. Contudo, estes sistemas dependem de hardware háptico dedicado para simular a resistência ao avanço e o contacto com as paredes das vias aéreas, um aspeto que continua a ser tecnicamente difícil de reproduzir com fidelidade, uma vez que a deteção de força na interação entre instrumento e tecido em endoscopia minimamente invasiva (termo mais abrangente, que inclui a broncoscopia como um dos seus procedimentos) permanece uma área ativa de investigação, sem uma solução consensual e amplamente adotada [18]. Modelos físicos baseados em tecido biológico real, como um biosimulador reutilizável construído a partir de pulmões suínos ventilados, procuram contornar esta limitação ao oferecer uma resposta tátil genuína, tendo sido avaliados favoravelmente por broncoscopistas experientes e principiantes em comparação direta com um simulador virtual comercial [19]. Estes modelos biológicos, no entanto, não são reutilizáveis indefinidamente, exigem preservação especializada e apresentam variabilidade anatômica de espécime para espécime, o que limita a sua padronização enquanto ferramenta de ensino e de avaliação objetiva.

Esta lacuna, entre a normalização própria dos sistemas virtuais e o realismo tátil dos modelos biológicos, abre espaço para os modelos impressos em 3D, que procuram oferecer geometria anatômica consistente e reproduzível, associada a materiais capazes de aproximar a resposta mecânica dos tecidos reais.

2.4 Impressão 3D aplicada à simulação médica

A impressão 3D é uma tecnologia de produção de objetos tridimensionais, realizada através da adição de material camada a camada, podendo o método de adição ou aglomeração dos materiais variar bastante consoante a tecnologia utilizada, entre as quais se destacam o FDM, a SLA e a SLS, cada uma assente num princípio de fabrico distinto e, por isso, mais adequada a diferentes tipos de aplicação. Com o passar dos anos e o amadurecimento da tecnologia e dos materiais disponíveis, este método de produção veio proporcionar uma oportunidade inovadora na criação de estruturas e geometrias complexas, a um baixo custo e numa fração do tempo face aos outros métodos de produção, afetando todas as áreas científicas e industriais, incluindo a medicina. Esta permite a produção de protótipos anatómicos com custo relativamente baixo, a correção e o ajuste de dimensões em pouco tempo e o teste de materiais inovadores que, com outros métodos, seria impensável, tanto pelo custo de produção como pela complexidade dos processos.

2.4.1 Técnicas de impressão 3D

O FDM (Fused Deposition Modeling, ou modelação por deposição de material fundido) é a tecnologia mais acessível e mais difundida, consistindo na extrusão de um filamento termoplástico aquecido através de um bico, que deposita o material camada a camada sobre uma base de impressão. É compatível com uma grande variedade de materiais, como o PLA, o PETG, o ABS ou filamentos flexíveis como o TPU, permitindo a produção de peças estruturalmente resistentes a um custo reduzido, ainda que com uma resolução e um acabamento de superfície mais limitados, sendo geralmente visíveis as linhas de cada camada. Para geometrias com zonas em consola ou sem apoio inferior, o FDM necessita ainda de estruturas de suporte, que têm de ser removidas após a impressão.

A SLA (Stereolithography) recorre a uma cuba de resina fotopolimérica líquida, curada seletivamente camada a camada por uma fonte de luz ultravioleta, um laser ou um projetor. Esta tecnologia permite obter peças com elevada resolução e um acabamento de superfície muito mais fino do que o FDM, sendo por isso indicada para modelos com geometrias detalhadas. Em contrapartida, exige equipamento de pós-processamento dedicado, nomeadamente para lavagem das peças em álcool isopropílico e cura adicional sob luz ultravioleta, além de recorrer a materiais mais dispendiosos e, em geral, mais frágeis do que os filamentos termoplásticos.

Por fim, a SLS (Selective Laser Sintering) utiliza um laser para sinterizar seletivamente um material em pó, tipicamente uma poliamida, camada a camada dentro de um leito de pó. Uma das suas principais vantagens é dispensar estruturas de suporte dedicadas, uma vez que o próprio pó não sinterizado sustenta as zonas em consola durante a impressão, permitindo fabricar geometrias mais complexas. As peças resultantes apresentam ainda propriedades mecânicas mais uniformes do que as obtidas por FDM. Contudo,

o equipamento e os materiais são substancialmente mais caros, o que torna esta tecnologia mais comum em contexto industrial do que em pequenos projetos ou protótipos.

No contexto deste projeto, esta comparação foi particularmente relevante na escolha entre FDM e SLA para a impressão da árvore brônquica, decisão discutida em detalhe na secção 3.2.4.

A utilização clínica e educativa da impressão 3D em modelos anatómicos exige, contudo, um controlo de qualidade rigoroso ao longo de todo o processo, desde a segmentação da imagem médica até à impressão final, sendo possível identificar e quantificar erros próprios de cada etapa, como o erro de segmentação, o erro de edição digital e o erro de impressão propriamente dito, cuja soma determina a fidelidade geométrica final do modelo obtido [20]. Esta preocupação com o rigor dimensional não se limita a modelos de treino, estendendo-se também a aplicações clínicas diretas, como o desenho e o fabrico de stents personalizados para o tratamento de estenoses das vias aéreas, benignas ou malignas, a partir da anatomia específica de cada doente [21–23], ou ainda à produção de modelos de traqueia para treino de cricotiroidotomia de emergência e de modelos de vias aéreas superiores para treino de entubação endotraqueal, ambos criados com o objetivo de tornar este tipo de treino mais acessível e menos dependente de equipamento comercial dispendioso [24, 25].

2.4.2 Aplicação em simuladores de árvores brônquicas

No caso dos simuladores de árvores brônquicas, os modelos impressos em 3D ganharam uma grande importância por permitirem transformar dados anatómicos, obtidos por exemplo a partir de exames de tomografia computadorizada (TAC), em geometrias físicas. Em broncoscopia, esta abordagem pode representar a árvore traqueobrônquica com boa correspondência espacial e permitir a criação de simuladores específicos para treino de navegação [7, 8], ainda que a sua principal limitação, quando usados de forma isolada, seja a ausência de movimento e de resposta fisiológica. Modelos multi-material, combinando simultaneamente componentes rígidos e flexíveis numa só impressão, têm vindo a ser explorados precisamente para aproximar a experiência tátil da broncoscopia real sem comprometer a robustez estrutural do conjunto [26], enquanto outras abordagens recorrem à impressão a cores, atribuindo uma cor distinta a cada segmento ou ramo brônquico, de modo a tornar mais perceptível a organização da árvore traqueobrônquica e a facilitar simultaneamente o treino de navegação e a aprendizagem da nomenclatura anatómica [27].

Contudo, a ideia de impressão destes modelos apresenta desafios e detalhes muito importantes, tais como a escolha do material, pois facilmente influencia o grau de dificuldade de impressão, a flexibilidade, a resistência, a sensação ao toque e a disponibilidade de cor. Materiais rígidos podem servir para validar forma e montagem, enquanto ma-



Figura 2.2: Modelos brônquicos impressos em multi-material [26].

teriais flexíveis podem aproximar-se melhor à resposta mecânica, ainda que aumentem consideravelmente a dificuldade de impressão e calibração. Assim, a impressão 3D deve ser entendida também como uma plataforma de iteração, não apenas como uma técnica de fabrico final.

2.5 Movimentos observáveis durante a broncoscopia

A árvore traqueobrônquica não se comporta como uma estrutura fixa durante uma broncoscopia. Mesmo quando o broncoscópio se mantém praticamente parado, a imagem observada pode apresentar deslocações lentas, alterações de orientação, variações do lúmen e perturbações rápidas. Para o operador, estes movimentos não aparecem isolados:

- A respiração cria uma componente periódica dominante;
- O batimento cardíaco acrescenta pequenas oscilações locais;
- A tosse introduz eventos abruptos;
- O contacto do broncoscópio com a parede pode modificar temporariamente a posição ou a forma do segmento observado.

Do ponto de vista de um simulador físico, estes fenómenos não precisam de ser reproduzidos com toda a complexidade fisiológica para terem valor pedagógico, mesmo que acabem sempre por apresentar um maior realismo. O mais importante é que o utilizador deixe de observar um modelo completamente estático e passe a treinar num ambiente com

movimento por vezes imprevisível e suficientemente coerente com o que é visto durante o procedimento real. Por isso, os movimentos descritos de seguida foram analisados quanto à sua relevância visual, dificuldade de implementação e utilidade, seguindo a categorização e os intervalos de frequência e amplitude propostos num documento de trabalho elaborado especificamente para este projeto pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [28], que serviu de base clínica direta às secções seguintes.

2.5.1 Movimento respiratório

O movimento respiratório é a componente mais frequente e previsível, ocorrendo durante o processo cíclico da inspiração e expiração, descrito como a deslocação dos pulmões com predominância craniocaudal, acompanhado por pequenas componentes laterais e por alterações locais da geometria das vias aéreas. A frequência respiratória normal situa-se aproximadamente entre 12 e 20 ciclos por minuto, embora possa variar com ansiedade, sedação, esforço respiratório, patologia e contexto clínico.

Análises mostram que a fase respiratória pode alterar a posição de estruturas pulmonares e influenciar modelos de fluxo nas vias aéreas [11, 12], existindo ainda, além da deslocação global, variações do calibre brônquico ao longo do ciclo respiratório, um fenómeno já descrito por trabalhos clássicos que demonstraram os mecanismos das alterações de calibre dos brônquios durante a respiração normal e das suas variações rítmicas [9, 10]. Mais recentemente, técnicas de modelação computacional têm procurado prever e simular este movimento de forma específica ao paciente, recorrendo a modelos de elementos finitos capazes de estimar o deslocamento dos brônquios ao longo do ciclo respiratório a partir de imagem médica, alcançando erros de previsão inferiores a 3% quando validados com dados reais [29]. Para o protótipo desenvolvido, esta componente foi interpretada como um movimento lento e periódico, adequado para ser representado por trajetórias sinusoidais, suaves e repetíveis.

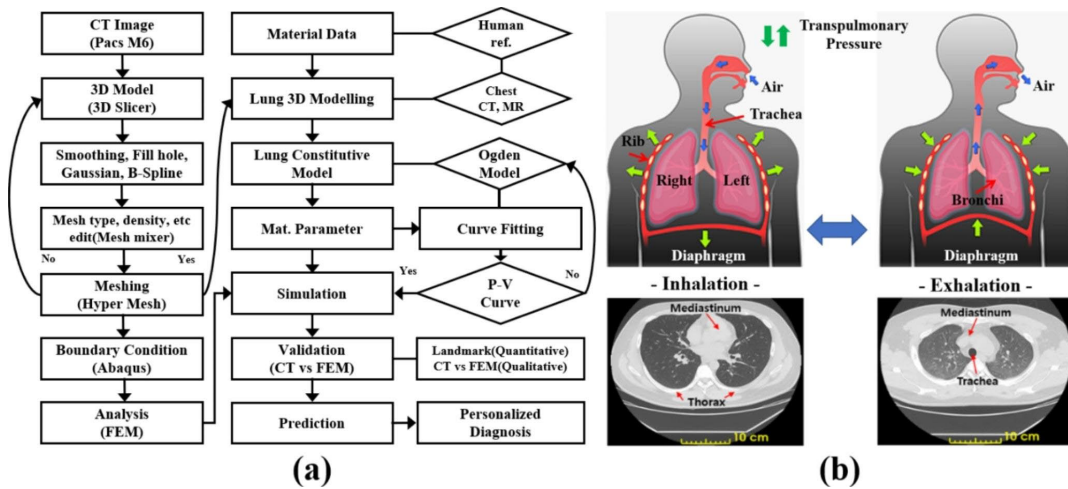


Figura 2.3: Modelo computacional do movimento respiratório [29].

2.5.2 Movimento cardíaco

O movimento cardíaco introduz oscilações mais rápidas e de menor amplitude, com intervalos regulares entre ocorrências, cuja influência tende a ser mais perceptível em regiões próximas do coração, em particular junto ao brônquio principal esquerdo e a estruturas adjacentes. Ao contrário da respiração, que domina a posição global da árvore brônquica, o batimento cardíaco surge antes como uma vibração ou perturbação sobreposta.

Estudos com imagem 4D confirmam esta distinção, tendo quantificado separadamente as contribuições respiratória e cardíaca no movimento de estruturas pulmonares e observado uma influência cardíaca proporcionalmente maior em lesões centrais e superiores, mais próximas do coração e dos grandes vasos [30]. Esta influência não se limita à posição da árvore brônquica, estendendo-se também ao próprio fluxo de ar no seu interior. Imagem dinâmica obtida por radiação de sincrotrão revelou que o batimento cardíaco gera oscilações de pressão e de fluxo de ar mensuráveis nos brônquios principais, num fenómeno localizado junto à carina até então difícil de demonstrar experimentalmente [31]. Em casos anatómicos particulares, esta proximidade pode mesmo traduzir-se em compressão mecânica direta do brônquio principal esquerdo por estruturas cardiovasculares adjacentes, como a aorta descendente ou uma artéria pulmonar dilatada, um efeito documentado em população pediátrica com anomalias intracardíacas [32].

Num modelo de treino, esta componente é útil para quebrar a sensação de movimento puramente sinusoidal e para acrescentar uma perturbação de menor amplitude, pelo que, no contexto deste projeto, o batimento cardíaco não foi tratado como uma reconstrução anatómica exata da interação coração-pulmão, mas antes como um perfil adicional de movimento, mais rápido e discreto, combinado com a respiração.

2.5.3 Tosse e perturbações rápidas

A tosse é um evento curto, abrupto e menos previsível, provocando, durante uma broncoscopia, a deslocação súbita das vias aéreas, com tendência para a ocorrência de réplicas adjacentes à deslocação principal, alteração temporária do campo visual, variação do fluxo de ar e perda momentânea de orientação. Para quem treina, este tipo de evento é importante porque obriga a recuperar a posição visual e a manter controlo do broncoscópico após uma perturbação.

A tosse é, em si mesma, um reflexo vital complexo, composto por uma fase inspiratória, uma fase compressiva com encerramento da glote e uma fase expiratória explosiva, coordenada por recetores das vias aéreas e por vias nervosas aferentes e eferentes bem descritas na literatura fisiológica [33]. A modelação computacional recente deste fenómeno tem-se debruçado sobre a dinâmica de gotículas e da película mucosa nas vias aéreas superiores durante um evento de tosse, com o objetivo de relacionar as suas características com diferentes patologias pulmonares [34], o que confirma o interesse científico atual em

compreender este movimento para além da sua simples deteção visual durante um procedimento.

No simulador, a tosse deve por isso ser representada como um evento transitório, com duração reduzida e amplitude superior à do batimento cardíaco, podendo a sua ocorrência ser programada de forma aleatória dentro de limites definidos, o que permite criar cenários de treino mais realistas sem obrigar à reprodução completa da fisiologia da tosse.

2.5.4 Interação com o broncoscópio

A passagem do broncoscópio também influencia a geometria observada, dado que o contacto com as paredes pode causar deformações locais, resistência ao avanço, pequenas oscilações e alterações da orientação da imagem. Esta componente é especialmente importante para o realismo tátil, porque aproxima o simulador da sensação física de navegação no interior das vias aéreas.

Apesar da sua relevância, esta interação mostra-se a mais difícil de implementar, exigindo materiais flexíveis adequados, geometrias calibradas com espessura e resistência adequadas e, idealmente, sensores ou mecanismos capazes de responder ao contacto. Esta dificuldade é também reconhecida na investigação em robótica endoscópica (área mais ampla, da qual a robótica aplicada à broncoscopia faz parte), onde a deteção de força na interação entre instrumento e tecido continua a ser identificada como um dos principais desafios em cirurgia minimamente invasiva [18], tendo já sido exploradas soluções como broncoscópios robóticos de balão, especificamente concebidos para procedimentos nas vias aéreas e capazes de aplicar forças controladas sobre as suas paredes sem lesionar o tecido, ainda que este tipo de solução acrescente complexidade mecânica significativa ao sistema [35]. Esta complexidade torna particularmente difícil a implementação deste tipo de fenómeno num simulador.

2.5.5 Patologias e deformações locais

Estenoses, obstruções, variações anatómicas e alterações patológicas modificam a navegação e o aspeto interno das vias aéreas, sendo que a impressão 3D permite criar geometrias específicas para representar alguns destes cenários de forma estática, o que já pode ser útil para treino de reconhecimento anatómico e planeamento de procedimentos. Um exemplo clínico bem documentado deste tipo de fenómeno é a traqueobroncomalácia, uma condição em que o amolecimento da cartilagem traqueal e brônquica provoca colapso dinâmico excessivo das vias aéreas, observado por videobroncoscopia em cerca de 69% dos asmáticos com doença grave, contra apenas 1,6% em indivíduos saudáveis [36]. Também a broncoconstrição ativa tem sido alvo de modelação matemática multiescala, procurando explicar como a complacência não linear da parede brônquica modula a resposta dinâmica

das vias aéreas a estímulos broncoconstritores [37].

No entanto, a simulação ativa de estenose variável, colapso local ou deformação dinâmica do lúmen exige um conjunto complexo de atuadores distribuídos, materiais compatíveis e uma estratégia de controlo mais complexa.

2.6 Síntese crítica

Esta análise comprova que existem necessidades que nem sempre são satisfeitas em simultâneo. Por um lado, os simuladores devem ser suficientemente acessíveis para poderem ser usados em contexto académico e profissional, permitindo treinos repetidos. Por outro, devem evitar uma representação demasiado estática das vias aéreas, porque a broncoscopia real ocorre num ambiente anatómico em constante movimento.

Os modelos impressos em 3D respondem bem à primeira necessidade, pois permitem criar geometrias anatómicas de baixo custo e iterar rapidamente o projeto, como demonstram diversos exemplos comerciais e académicos de árvores brônquicas estáticas, alguns já disponíveis há várias décadas [7, 26, 27, 38–40]. Contudo, quando usados isoladamente, tendem a limitar-se à fidelidade geométrica.

Algumas publicações têm procurado atenuar exatamente esta limitação, conferindo algum grau de dinamismo a modelos impressos em 3D e a outras réplicas físicas de vias aéreas, embora nenhuma delas recorra a atuação mecânica da própria estrutura anatómica. Um primeiro exemplo consiste na insuflação controlada de ar ou soro através de uma porta lateral, fazendo com que o próprio modelo impresso se expanda e contraia de forma passiva, uma solução já aplicada a um protótipo de via aérea impresso em 3D capaz de emular movimento fisiológico e patológico [8]. Outro trabalho, mais próximo da atuação robótica, recorre a seringas acionadas por uma guia linear motorizada para empurrar ar através de condutas rígidas que representam a árvore brônquica e a traqueia, replicando fisicamente tanto a respiração como a tosse. Contudo, mesmo nesta solução, os brônquios e a traqueia permanecem estruturas rígidas e estáticas, sendo apenas o fluxo de ar no seu interior que é mecanicamente controlado, sem qualquer tentativa de reproduzir o batimento cardíaco [41].

Fora do domínio da broncoscopia, outros trabalhos recorrem a atuação mecânica real para simular a mecânica ventilatória, mas fazem-no ao nível do diafragma ou da parede torácica, não da própria árvore brônquica. É o caso de um simulador respiratório que combina músculos artificiais pneumáticos com um diafragma sintético para insuflar pulmões reais ou artificiais [42], de um simulador eletromecânico de pulmão que combina componentes poliméricos e tecido pulmonar orgânico para reproduzir uma respiração realista [43] e de modelos deformáveis usados em radioterapia para replicar mecanicamente a expansão e a contração de um pulmão, sem qualquer representação da árvore brônquica no seu interior [44]. À mesma categoria pertencem ainda simuladores pulmonares controlados

por motor de passo ou por pistão servo-controlado, desenvolvidos sobretudo para testar e calibrar ventiladores mecânicos [45, 46], bem como patentes que atuam mecanicamente a parede torácica de manequins de treino, por bolsas de ar ou por sistemas biela-manivela, para simular respiração ou manobras de reanimação [47, 48]. Outros dispositivos comerciais patenteados representam a árvore brônquica de forma mais completa, incluindo canais para brônquio esquerdo, direito e traqueia, mas destinam-se a treinar procedimentos de drenagem postural com sensores de posição, não a reproduzir movimento fisiológico [49]. Uma última categoria de dispositivos recorre efetivamente a atuadores mecânicos reais sobre estruturas anatómicas de treino, mas aplicados à via aérea superior, como a mandíbula, a língua ou a articulação temporomandibular em simuladores de gestão de via aérea e de entubação orotraqueal, nunca chegando a atuar a árvore brônquica em profundidade e as suas bifurcações [50].

Do lado puramente computacional, têm também sido propostos métodos capazes de simular visualmente a deformação respiratória das vias aéreas durante a navegação broncoscópica, por renderização de imagem em tempo real a partir de dados de tomografia computadorizada, as imagens médicas que descrevem em cortes transversais a anatomia interna do paciente, mas sem qualquer correspondência física. O resultado é sempre uma simulação gráfica, não um modelo tangível que o formando possa manipular com um broncoscópio real [51]. Já no campo da robótica cirúrgica aplicada à broncoscopia, os poucos manipuladores desenvolvidos destinam-se a operar sobre vias aéreas reais de doentes ou de cadáveres, como ferramentas de tratamento e não como modelos de treino, permanecendo a anatomia sempre estática durante a sua atuação [52]. Mesmo alargando esta análise a protótipos ibero-americanos, como um simulador de broncoscopia com realimentação háptica desenvolvido no México, cuja árvore brônquica nunca chega a ser impressa fisicamente e permanece apenas como um modelo gráfico renderizado [53], a conclusão mantém-se a mesma.

Em síntese, não se encontrou em toda a literatura revista nenhum modelo de árvore brônquica impresso em 3D em que a própria estrutura anatómica seja mecanicamente atuada, por manipuladores robóticos ou mecanismo equivalente, de forma a reproduzir de forma simultânea os principais fenómenos fisiológicos observáveis durante a broncoscopia. É precisamente esta lacuna que o presente projeto procura preencher.

A componente dinâmica, mesmo que simplificada, acrescenta valor ao treino, pois obriga o utilizador a lidar com deslocações por vezes imprevisíveis e com a perda parcial do enquadramento, devido a perturbações temporárias.

Assim, o objetivo deste projeto encontra-se em combinar uma árvore brônquica física, obtida por impressão 3D, com uma arquitetura de atuação capaz de reproduzir de forma controlada os fenómenos anatómicos dinâmicos identificados ao longo deste capítulo, respondendo a uma lacuna que a literatura revista ainda não preenche.

Capítulo 3

Desenvolvimento do Hardware

Este capítulo documenta a fase que ocupou a primeira metade do tempo dedicado a este projeto e que serviu de alicerce a todo o trabalho descrito nos capítulos seguintes: o desenvolvimento do modelo físico da árvore brônquica impresso em 3D e de toda a estrutura mecânica e eletrônica que viria a sustentar a sua atuação. O capítulo percorre esse desenvolvimento do hardware, desde as primeiras impressões de teste até à montagem final do sistema dos dois manipuladores delta, eletrônica de controlo e suportes para guardar os componentes de forma segura, um processo que não foi linear, uma vez que cada iteração revelou limitações que levaram a novas abordagens do design, dos materiais e da arquitetura mecânica. O que se segue documenta esse caminho de escolhas intermédias, bem como os problemas que as motivaram.

3.1 Requisitos do sistema

Como já referido, o modelo deve permitir o treino de videobroncofibroscopia com um modelo físico de baixo custo, robusto, substituível e fácil de montar, tendo os requisitos principais sido agrupados em quatro eixos:

- → Realismo anatómico;
- → Facilidade de utilização;
- → Movimento fisiológico controlado;

No plano anatómico, o modelo deve representar a árvore brônquica com dimensões aproximadas, ângulos de ramificação e características compatíveis com dados obtidos por

tomografia computadorizada, enquanto no plano mecânico deve permitir a introdução de movimentos de respiração, batimento cardíaco e tosse sem comprometer a estabilidade do conjunto. No plano pedagógico, deve ainda permitir a repetição de exercícios e a adaptação do grau de dificuldade e, no plano técnico, deve permitir configurar a amplitude e frequência dos movimentos e as posições do movimento.

Do ponto de vista funcional, o sistema foi separado em dois blocos:

- 1º → Modelo Físico, constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servo-motores e seus respetivos drivers e pelo microcontrolador ESP32-S3, executando fisicamente os movimentos e mantendo um comportamento previsível mesmo quando recebe comandos externos;
- 2º → HMI, suportada pelo Raspberry Pi 4B e pelo monitor touchscreen, apresentando ao utilizador, através de botões e menus intuitivos, comandos de alto nível, como iniciar, parar, escolher modo fisiológico ou ajustar parâmetros, ficando a cargo do software a tradução desses comandos, o cálculo e a geração de coordenadas para os movimentos pretendidos e a execução física dos mesmos, o que torna o sistema mais simples e intuitivo para o utilizador;

Um dos requisitos obrigatórios do desenvolvimento do modelo físico é a capacidade de reproduzir movimentos fisiológicos de forma controlada, com frequências e amplitudes compatíveis com os valores observados em humanos, sendo esta fidelidade de movimento o elemento que distingue este protótipo dos simuladores puramente estáticos já existentes no mercado. Esta exigência tornou-se também o maior desafio de todo o projeto, uma vez que o modelo físico e a HMI têm de funcionar de forma paralela e sincronizada, sem comprometer a precisão temporal dos movimentos sobrepostos nem o sincronismo entre os comandos enviados pela interface e a resposta física do modelo.

3.2 Impressão 3D da árvore brônquica

Esta secção documenta o processo mais importante de todo o projeto, o processo de impressão 3D da árvore brônquica, desde o modelo STL fornecido pela equipa médica até à peça final, fisicamente impressa e testada. O desenvolvimento deste modelo impresso condicionou diretamente as restantes fases do projeto, uma vez que a atuação mecânica, a eletrónica, o firmware e a HMI foram concebidos a partir das dimensões, da geometria e

das limitações mecânicas desta estrutura física. Este processo consumiu grande parte do tempo dedicado ao desenvolvimento do modelo físico, exigindo sucessivas iterações de tratamento de malha, escolha de materiais, orientação de impressão e ajuste dos parâmetros de fabrico, até se alcançar uma solução simultaneamente fiel à anatomia original e mecanicamente viável para a atuação robótica prevista. As técnicas de impressão 3D consideradas para este projeto, FDM, SLA e SLS, foram já comparadas na secção 2.4.1, sendo o FDM a tecnologia disponível na universidade e por isso a base de todo o processo aqui descrito.

3.2.1 Da tomografia ao modelo STL

O ponto de partida do projeto foi o modelo tridimensional da árvore brônquica, obtido a partir de dados de uma tomografia computadorizada (TAC), a tecnologia de imagem médica utilizada pela equipa clínica para obter o scan 3D, que foi realizada e entregue pelo grupo de médicos encarregues do projeto, em formato de malha 3D (STL). Este tipo de ficheiro define apenas a geometria da superfície do objeto, sob a forma de uma malha de triângulos, sem qualquer volume ou espessura associada, pelo que o modelo obtido correspondia apenas à superfície interna da árvore brônquica, uma espécie de casca oca, sendo por isso necessário atribuir-lhe espessura manualmente antes da impressão 3D.

3.2.2 Programas utilizados no tratamento do modelo

Esta operação foi feita inicialmente no programa gratuito **Tinkercad** (da Autodesk), com o objetivo de realizar um teste rápido em PLA e confirmar a viabilidade geométrica da impressão.

Depois de uma confirmação visual, passou-se para um programa mais indicado para este trabalho, o **Fusion 360**, um programa CAD da Autodesk destinado ao desenho 3D de sólidos, superfícies e malhas, que dispõe de uma ferramenta interna chamada “Forms”. Ao contrário do ambiente normal do Fusion 360, onde se trabalha maioritariamente com objetos sólidos e geometria bem definida, o Forms é exclusivamente indicado para trabalhar diretamente com scans 3D e as suas irregularidades geométricas, o que o tornava particularmente adequado a este caso. Neste ambiente, não se edita diretamente o ficheiro de malha: em vez disso, é criada uma aproximação à superfície através de uma camada virtual, sendo apenas a essa camada que depois é possível atribuir um valor de espessura, criando assim um verdadeiro objeto 3D da árvore brônquica. Esta abordagem

permitia editar em tempo real a geometria da camada e corrigir irregularidades existentes na malha, bem como descartar secções consideradas desnecessárias, como as extremidades mais pequenas do scan 3D, sem qualquer utilidade para o modelo devido à incompatibilidade entre a largura das vias nessas extremidades e a largura da sonda broncoscópica. Contudo, observou-se rapidamente que este método era demorado e ineficiente: o Forms trabalha com formas primitivas do tipo T-Spline, como o cilindro sem base nem topo, compostas por vértices, arestas e faces deslocáveis, rodáveis ou escaláveis através da ferramenta Edit Form. Não era possível aplicar esta aproximação ao modelo completo de uma só vez, sendo necessário trabalhar ramo a ramo, o que, seguido de sucessivas costuras e correções, tornava o método muito demorado.

Por esta razão, optou-se por um outro programa, o **MeshMixer**, também da Autodesk, direcionado exclusivamente para trabalhar e modificar diretamente ficheiros de malha 3D. Ao contrário do Forms, que edita apenas uma camada virtual de aproximação, o MeshMixer permite selecionar diretamente os triângulos da malha através de ferramentas de pincel e laço, corrigindo na fonte as imperfeições e cortando as secções desnecessárias. A ferramenta Inspector fecha automaticamente as malhas abertas e repara falhas de continuidade na superfície, enquanto o Offset permite atribuir espessura diretamente à malha, sem depender de uma camada intermédia como no Forms. Uma outra vantagem do MeshMixer está na forma como lida com sobreposições: quando, com o aumento de espessura (scale-up), duas malhas ficam sobrepostas, a ferramenta Boolean Union une-as automaticamente numa só peça, enquanto o Fusion 360 não consegue processar essas sobreposições e obriga a manter uma distância mínima entre elas, exigindo várias iterações e ajustes manuais.

Mesmo depois da transição para o MeshMixer, o **Fusion 360** continuou a ser bastante utilizado ao longo da modelação da árvore brônquica, uma vez que continuava a ser necessário desenhar as peças de encaixe que permitiam ligar o modelo à estrutura de suporte dos manipuladores. O fluxo de trabalho consistia em tratar o modelo no MeshMixer, atribuindo-lhe espessura e cortando os limites dos tubos necessários, para depois o exportar em formato STL e importá-lo no Fusion 360, onde não era feita qualquer conversão ou tratamento adicional da malha. Nesse ambiente, desenhavam-se apenas as peças de encaixe como objetos sólidos independentes, posicionados de forma a manterem-se dentro dos limites internos da malha, sem nunca ultrapassar para o lado oco do modelo. O ficheiro completo, incluindo a malha importada e as peças de encaixe recém-desenhadas, era depois exportado em conjunto como um único STL, de forma a evitar que o programa mantivesse os dois tipos de objeto como entidades distintas, o que poderia criar problemas mais tarde na importação para o slicer. Esta união era possível devido à própria natureza do formato STL, que, ao contrário de formatos como o STEP ou o OBJ, não

transporta qualquer informação sobre a estrutura interna ou a separação entre objetos de um ficheiro, sendo composto apenas por coordenadas de vértices e pela sua conectividade em triângulos. Um ficheiro STL binário, em particular, nem sequer tem capacidade para representar mais do que um único conjunto de triângulos, pelo que exportar a malha e as peças sólidas em conjunto resultava sempre numa única superfície contínua, sem qualquer separação reconhecível entre os dois tipos de objeto originais.

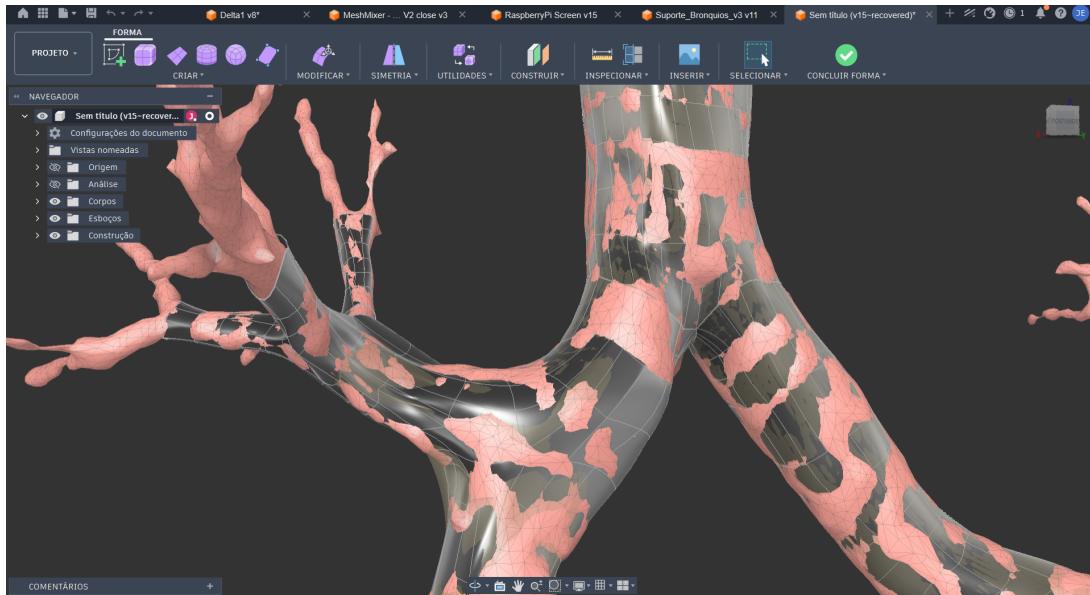


Figura 3.1: Tratamento da malha da árvore brônquica no Fusion 360.

3.2.3 Seleção de materiais

A seleção de materiais foi considerada uma decisão crítica ao longo de todo o projeto. Como o objetivo do modelo era aproximar-se o máximo possível da realidade anatómica, era necessário utilizar materiais suficientemente moles, tendo sido identificados por isso vários filamentos flexíveis como PEBA, TPU, TPE e TPC, bem como diferentes durezas comerciais, por exemplo 95A, 85A e 70A.

Nos filamentos maleáveis, quanto menor a dureza do material, mais difícil se torna a sua impressão. Os filamentos mais duros alimentam-se com facilidade através do extrusor, enquanto os mais moles tendem a comprimir-se e a deformar-se nas engrenagens de alimentação em vez de serem corretamente empurrados, o que aumenta significativamente a tendência para entupimentos e para defeitos de impressão, como o stringing.

O TPU exige ainda um controlo mais apertado de vários parâmetros de impressão, nomeadamente a temperatura, que deve manter-se numa gama estreita, uma vez que

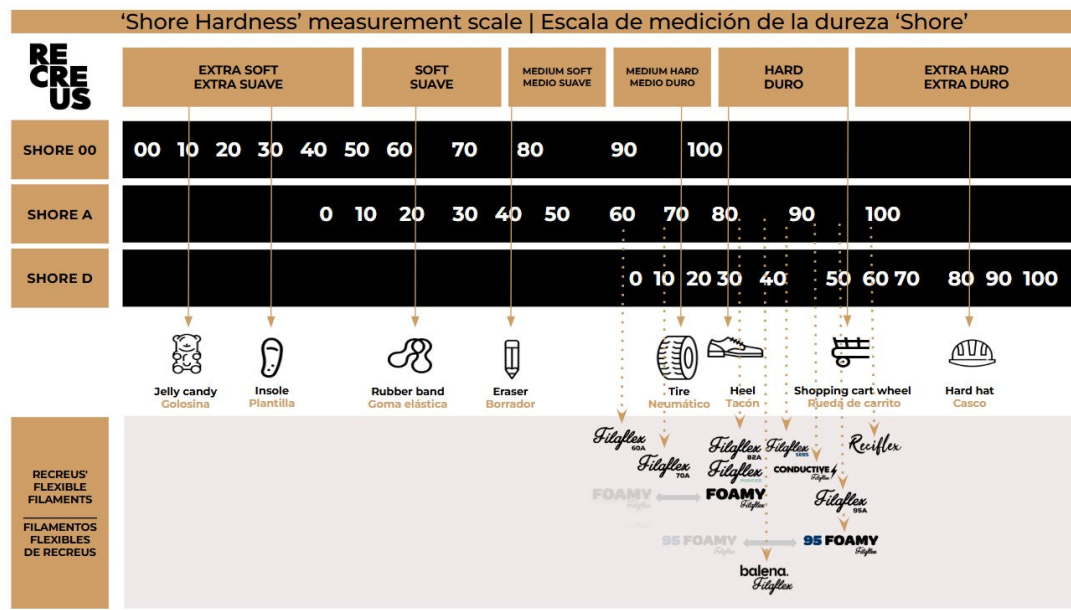


Figura 3.2: Escala de dureza Shore aplicada a filamentos flexíveis, com exemplos de objetos do dia a dia em pontos equivalentes da escala (fonte: Recreus).

poucos graus a mais podem ser suficientes para provocar stringing ou distorções mais graves nas secções inclinadas por excesso de fluidez do material, a retração, que deve ser reduzida ao mínimo, já que retrações demasiado longas tendem a provocar entupimentos em vez de os evitar (devido à criação de tensão entre a zona do extrusor e a secção de convecção, que por sua vez faz com que o material adira às paredes do bico e crie resistência) e a velocidade de impressão, que deve ser mantida baixa, para permitir uma extrusão mais controlada.

A dureza mais comum no mercado é a 95A, existindo ainda assim empresas que produzem filamentos especializados com durezas tão baixas como 85A ou mesmo 70A, sendo este último especialmente problemático, por se deformar com facilidade durante a impressão e por exigir um extrusor direct-drive dedicado para não encravar. Por esta razão, os testes iniciais foram realizados apenas com TPU 95A, com o objetivo de validar a viabilidade de impressão do modelo em materiais flexíveis antes de se avançar para materiais mais difíceis de processar.

3.2.4 Disposição do modelo para impressão

Um outro desafio foi decidir como dispor o modelo para impressão em FDM, a única tecnologia disponível na universidade. Apesar de não ser uma opção viável devido à inacessibilidade, a impressora SLA eliminaria alguns dos problemas de suporte da impressão

em FDM, apresentando ainda mais vantagens, tais como:

- Vantagens do SLA: dispensaria suportes em várias zonas do modelo, simplificando a geometria de impressão e reduzindo o risco de stringing no interior de peças ocas e ramificadas, um dos problemas que mais viria a afetar a impressão em FDM, como descrito mais adiante nesta secção. Permite também obter um acabamento de superfície muito mais fino e sem linhas de camada visíveis, característica relevante para reproduzir com fidelidade a superfície interna lisa da árvore brônquica. Além disso, existe atualmente uma grande variedade de resinas flexíveis e de silicones fotocuráveis compatíveis com esta tecnologia, com durezas Shore A que vão de aproximadamente 80A [54] até valores bastante mais baixos, como 70A [55], 63A ou 55A [56], 50A [57] ou mesmo 40A em resinas de silicone puro [58], um intervalo mais alargado e mais fácil de alcançar de forma fiável do que o observado nos filamentos de FDM, em que durezas abaixo de 70A se tornam cada vez mais problemáticas de imprimir, precisamente porque o processo de cura de uma resina líquida não depende da extrusão mecânica de um filamento sólido através de engrenagens e de um bico;
- Desvantagens do SLA: equipamento com custo muito superior e dependente de equipamento adicional de pós-processamento, nomeadamente lavagem das peças em álcool isopropílico e cura adicional sob luz ultravioleta, o que tornava a sua aquisição incomportável para o projeto. Os volumes de impressão são também tipicamente mais reduzidos do que os disponíveis em impressoras FDM, o que poderia condicionar a impressão do modelo completo da árvore brônquica numa só peça.

Mantendo-se a impressão em FDM, foram equacionadas três opções para a disposição do modelo:

1. **Impressão em peça única com suportes exteriores.** O modelo seria impresso de uma só vez, com suportes colocados do lado de fora para sustentar as zonas sem apoio inferior, que ficariam essencialmente a ser impressas “no ar”:

- Vantagens: simplicidade de modificação da impressão e possibilidade de reaproveitar os próprios suportes como estrutura de fixação do modelo à base com os atuadores;
- Desvantagens: tempo de impressão elevado, uma vez que todo o modelo seria impresso de uma só vez, obrigando a reimprimir a peça inteira em caso de falha, com desperdício considerável de filamento, tempo e energia, além de os

suportes só poderem ser colocados no exterior do modelo, deixando sem sustentação zonas interiores igualmente sujeitas a grandes deformações, sobretudo em materiais mais moles.

2. Desdobramento e impressão individual das ramificações. Esta opção passava por separar literalmente cada ramificação da malha num objeto independente e desdobrar a secção cilíndrica de cada ramo num plano, à semelhança do desdobramento de papel para construir um sólido, atribuindo-lhe espessura já na forma planificada e devolvendo-lhe a forma tridimensional apenas depois de impresso:

- Vantagens: possibilidade de modificar facilmente cada ramo de forma individual e de integrar sensores ou atuadores diretamente no modelo, caso necessário;
- Desvantagens: incerteza quanto à viabilidade do método, uma vez que, embora fosse possível desdobrar um ficheiro STL num plano com programas como o Blender, não havia garantia de que o mesmo funcionasse com um scan tão complexo e detalhado, podendo gerar erros ou geometrias inesperadas. A montagem após a impressão era também um risco, dado que os filamentos maleáveis são imprevisíveis e podem esticar ao serem retirados da base de impressão, criando folgas na união das peças e exigindo uma estratégia própria de costura entre secções, com uma probabilidade reduzida de sucesso ao longo de tantas ramificações.

3. Moldagem por enchimento de um molde negativo. Em vez de imprimir diretamente o modelo, esta opção previa a impressão de um molde negativo, utilizado depois para verter resina ou silicone e obter o positivo da árvore brônquica. Foi uma ideia forte durante grande parte do projeto, funcionando quase sempre como plano secundário:

- Vantagens: possibilidade de usar um material mais rígido no molde e um material extremamente mole no modelo positivo, bem como reutilizar o mesmo molde para produzir vários modelos sem o descartar;
- Desvantagens: como a árvore brônquica é oca, era necessário produzir um molde negativo tanto para a superfície exterior como para o espaço interior do modelo, ficando o positivo reduzido a uma película fina, com a espessura pretendida, encaixada entre esses dois moldes negativos. Isto obrigava a unir com precisão as três secções de molde em volta dessa película, o negativo interior e os dois lados do negativo exterior. A orientação das ramificações em várias direções dificultava ainda a remoção dos moldes sem danificar o modelo. Por fim, a geometria complexa e extensa não dava garantias de que

fosse possível injetar resina ou silicone de forma uniforme em todos os cantos, com risco de inconsistências na espessura das paredes.

Mais tarde, motivadas pela futura aquisição de uma impressora Bambu Lab com dois bicos de impressão independentes (assunto detalhado mais adiante), foram ainda teorizadas duas variações aos métodos já descritos:

1. **Suportes solúveis em PVA**, uma variação da impressão em peça única com suportes exteriores, substituindo o material dos suportes pelo filamento solúvel PVA:
 - Vantagens: por ser solúvel em água, permitiria colocar suportes também no interior do modelo, em regiões críticas, aumentando a qualidade de impressão sem complicar o processo de remoção;
 - Desvantagens: o PVA tem um custo por quilo superior ao do PLA e mesmo ao do TPU, já de si caro nas durezas mais baixas. Por ser sempre perdido após a dissolução em água, o custo de utilização tornar-se-ia inoportuno. Esta variação acabou por nunca ser testada, por falta de tempo para adquirir o material.
2. **Molde negativo interno com imersão em resina**, uma variação do método dos moldes, recorrendo a uma resina especial de uso médico que solidifica em contacto com o ar, mantendo uma dureza específica de acordo com o tempo de exposição antes de a reação ser interrompida com um químico próprio. O processo consistiria em imprimir apenas o molde negativo interno da árvore brônquica, num material de fácil remoção como o PVA e mergulhá-lo repetidamente na resina, à semelhança do processo de fabrico de velas, sendo o número de imersões definido pela espessura pretendida para o modelo positivo. No final, bastaria dissolver o molde negativo em água:
 - Vantagens: processo de produção muito simples;
 - Desvantagens: o material do molde negativo tem um custo elevado, sendo sempre perdido no processo. Depois de impresso, o modelo, já sem qualquer rigidez estrutural própria, seria também difícil de fixar à estrutura de suporte dos manipuladores, exigindo peças de fixação dedicadas e de integração complexa.

Por estas razões, optou-se por manter o método de impressão em peça única com suportes exteriores.

3.2.5 Evolução do processo de impressão 3D

Inicialmente, os suportes eram impressos no mesmo material do modelo, devido às limitações da maquinaria disponível na altura, com apenas um bico de impressão. Este método funcionava razoavelmente bem com filamentos de dureza elevada, como o 95A, mas mostrava-se bastante problemático com materiais mais moles, como o Filaflex TPE 70A. A mesma característica que tornava estes materiais interessantes para o projeto tornava-os também problemáticos. Ao usar o mesmo material para o suporte e para o objeto, toda a impressão se comportava como um único e grande objeto mole, cuja vibração aumentava com a altura já impressa, entrando em ressonância com o próprio movimento da cabeça de impressão e criando grandes inconsistências. Por frações de milissegundo e de milímetro, a posição da cabeça de impressão deixava de corresponder à posição real da última camada impressa. (vou adicionar imagens sobre isto nesta secção)

Uma das soluções equacionadas para este problema passava por substituir o filamento por um material especial de dureza variável, com um agente expansivo interno que utiliza o próprio calor do bico como catalisador, permitindo teoricamente atingir durezas entre 80A e 50A consoante a temperatura, segundo a informação disponibilizada pela marca. O único inconveniente deste material seriam os resíduos gerados pelo agente expansivo, que poderiam comprometer a qualidade de impressão, ainda que de forma mitigável. Esta solução acabou por nunca ser necessária, pois, sensivelmente na mesma altura, a universidade adquiriu três novas impressoras 3D, entre elas uma Bambu Lab H2D, equipada com dois bicos independentes que permitem imprimir, em simultâneo, materiais com propriedades e compatibilidades distintas, uma aquisição que resolveu o problema.

Passou-se então a um método que mantinha o TPE 70A como material principal do modelo, mas com PLA como material de suporte, funcionando simultaneamente como suporte estrutural e como suporte de amortecimento de vibração para o modelo principal, como se observa na figura 3.3.

Este método trouxe uma melhoria enorme na qualidade de impressão, mas tornou mais visíveis outros problemas. Como já referido, os materiais flexíveis de dureza muito baixa são especialmente difíceis de imprimir com fiabilidade. Este caso não foi exceção. Detetaram-se, durante e após a impressão, inconsistências de aderência entre camadas e entre paredes do objeto, camadas inteiras com falta de material, tornando-as propensas a quebrar ou rasgar e, o pior de todos os problemas, uma quantidade excessiva de stringing no interior do modelo, precisamente a zona mais importante para este trabalho. Isto



Figura 3.3: Modelo impresso com suportes em PLA.

persistia mesmo após inúmeras calibrações dos parâmetros de retração, da velocidade de impressão, da temperatura de impressão, do Pressure Advance e dos níveis de humidade do filamento. O Pressure Advance é uma funcionalidade do firmware do slicer que ajusta antecipadamente a pressão de extrusão nas mudanças de velocidade, reduzindo o babar de material e o stringing nos cantos. O TPU é, além disso, frequentemente descrito como uma “esponja”, por ser um material higroscópico, ou seja, por absorver facilmente humidade do ar, sendo por isso necessário secá-lo constantemente e mantê-lo selado o máximo de tempo possível.

Foram também testadas técnicas e opções internas do slicer para mitigar o stringing, como alterar a localização da costura das paredes para uma zona menos visível, calibrar constantemente os valores de transição entre paredes, modificar as opções de geometria de retração e forçar as deslocações da cabeça de impressão a ocorrerem, sempre que possível, sobre as zonas de enchimento.

Ainda assim, nada resolvia de forma satisfatória o problema do stringing, nem mesmo técnicas de limpeza manual, como escovilhões para aquários, jatos de ar comprimido ou a queima das próprias linhas no interior do modelo. Algumas destas técnicas conseguiam reduzir a visibilidade do problema nas ramificações mais próximas da entrada, mas nas ramificações mais profundas era praticamente impossível.

A única forma de resolver, ou melhor, de mitigar este problema, foi escolher ou-

tro material como filamento flexível principal, o TPU MorPhlex 95A/75A, da empresa BiQU. Ao contrário dos outros filamentos de dureza variável considerados, apresenta uma transição de dureza fixa, começando com uma dureza de 95A antes de ser impresso e estabilizando permanentemente numa dureza de $75A \pm 3A$ depois de impresso e aquecido. Esta característica torna-o muito mais estável, fiável e seguro do que as alternativas anteriores, permitindo, com a calibração correta, atingir uma qualidade de impressão quase perfeita e praticamente sem resíduos. Um outro ponto forte deste filamento é a sua cor, exclusiva da BiQU, muito próxima da cor real do interior do pulmão, o que o tornava ainda mais adequado a este trabalho.



Figura 3.4: Modelo final da árvore brônquica impresso em material flexível.

Este material revelou, no entanto, uma deformação predominantemente plástica em vez de elástica quando sujeito a grandes esforços repetidos, como as contrações e expansões constantes que a atuação ativa das paredes exigiria, uma característica que viria a pesar mais tarde na avaliação da viabilidade de atuadores dedicados a essa deformação.

3.3 Seleção do sistema de atuação

A definição do sistema de atuação foi uma das partes que mais evoluiu durante o projeto, sobretudo nas fases iniciais. A descrição dos movimentos pretendidos ainda não se encontrava completamente consolidada e, por isso, foram exploradas soluções que mais tarde se revelaram pouco adequadas ao comportamento real que se pretendia reproduzir. Esta incerteza levou à análise de alternativas bastante diferentes entre si, desde motores

de vibração, pensados para criar zonas de pressão localizada que pudessem sugerir o batimento cardíaco ou pequenas compressões das vias aéreas, até micro motores de passo, considerados para apertar gradualmente as paredes do modelo e simular constrangimentos associados ao ciclo respiratório.



Figura 3.5: Teste inicial de atuador local com suporte impresso.

Com a evolução do modelo da árvore brônquica e após a primeira reunião de acompanhamento com a equipa médica, tornou-se mais clara a natureza dos movimentos a implementar. O objetivo principal deixou de ser a criação de deformações locais independentes e passou a ser a reprodução controlada de deslocações globais e perturbações visíveis, associadas à respiração, ao batimento cardíaco e à tosse. Esta clarificação obrigou a rever a arquitetura de atuação inicialmente imaginada, baseada na distribuição de vários atuadores ao longo da árvore brônquica. Embora essa solução pudesse parecer direta para movimentos localizados, acabaria por tornar o sistema pesado do ponto de vista mecânico, aumentando o número de motores, exigindo uma estrutura de suporte muito próxima do modelo e introduzindo dificuldades adicionais de montagem, calibração, sincronização e manutenção.

A solução adotada passou, por isso, para uma utilização de manipuladores robóticos capazes de posicionar uma plataforma móvel dentro de um volume de trabalho definido por coordenadas cartesianas. Nesta abordagem, o movimento deixa de depender de vários atuadores distribuídos pela árvore brônquica e passa a ser gerado a partir da trajetória imposta à plataforma, permitindo atuar sobre a árvore brônquica sem instalar motores diretamente no modelo. A complexidade é, assim, transferida para a descrição matemática do movimento, em particular para o cálculo da cinemática inversa, permitindo alterar perfis de respiração, batimento cardíaco ou tosse sobretudo por software, sem obrigar a modificar fisicamente a disposição dos motores.

Dentro desta lógica, foram estudadas diferentes configurações de manipuladores delta, uma vez que este tipo de robô paralelo utiliza vários braços ligados a uma plataforma móvel comum. Ao contrário de um manipulador serial, em que cada junta depende diretamente da anterior, num manipulador delta os braços trabalham em conjunto para

posicionar a plataforma no espaço. Esta geometria permite obter movimentos rápidos, repetíveis e compactos, mantendo os atuadores fixos na base e reduzindo a massa em movimento.

Foram considerados dois tipos principais de solução. A primeira recorria a seis atuadores, dois por braço, permitindo controlar a translação da plataforma nos eixos X , Y e Z e, adicionalmente, a sua orientação através de três graus de rotação. Esta configuração oferecia maior liberdade de movimento, mas introduzia uma complexidade mecânica e matemática superior à necessária para os fenómenos definidos para o protótipo. A segunda opção, posteriormente escolhida, utiliza apenas três atuadores, um por braço, controlando a posição da plataforma nos três eixos cartesianos. Apesar de não permitir comandar diretamente a rotação da plataforma, esta solução revelou-se suficiente para representar os deslocamentos pretendidos, simplificando a montagem, reduzindo o número de componentes e tornando mais direta a implementação da cinemática inversa.

Com esta decisão, a seleção dos atuadores passou a privilegiar servo-motores compactos, acessíveis e fáceis de controlar, tendo sido escolhidos os MG90S para a construção dos manipuladores. Esta escolha foi também reforçada pela comparação direta com motores de passo, avaliados em paralelo durante os testes iniciais de eletrónica. Apesar de os motores de passo garantirem maior capacidade de carga, a sua natureza de controlo em malha aberta obriga a uma rotina de homing sempre que o sistema é ligado, para que o motor recupere a noção da sua posição real, enquanto um servo-motor recebe apenas um comando de posição que pretende atingir (em formato de um sinal PWM) enviado num único sentido, cabendo ao seu controlador interno ajustar automaticamente, através de uma malha de controlo PID. Esta diferença simplificava significativamente o arranque e o funcionamento do sistema, sendo esta escolha ainda coerente com o objetivo de manter o sistema simples e repetível, concentrando a coordenação dos movimentos no firmware e no cálculo das posições da plataforma.

Esta escolha mecânica foi acompanhada, ainda numa fase inicial do projeto, pelo início do desenvolvimento do código de cinemática inversa associado a este tipo de manipulador, descrito em detalhe na secção 4.8.

3.4 Eletrónica de controlo e alimentação

O desenvolvimento da eletrónica teve início a meio do desenvolvimento do hardware, depois de confirmada a viabilidade de impressão do modelo e da aquisição dos componentes

eletrônicos. A arquitetura de alimentação e eletrônica adotada, ilustrada na figura 3.6, parte de uma fonte central de 5 V, distribuída pelo módulo UPS/HAT ao Raspberry Pi 4B, ao monitor e, através de dois drivers dedicados, aos seis servo-motores dos dois manipuladores delta, enquanto o ESP32-S3 comunica com o Raspberry Pi por porta série e envia os sinais PWM que comandam cada driver.

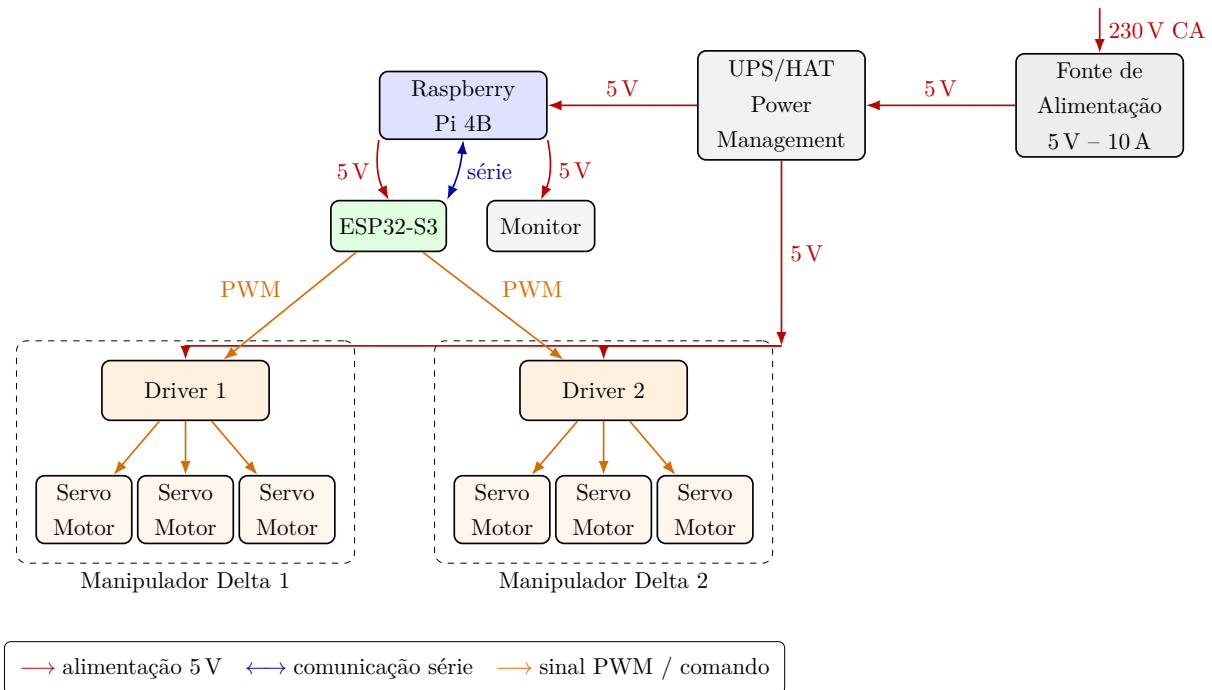


Figura 3.6: Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.

3.4.1 Testes iniciais de eletrônica

Os primeiros testes de eletrônica focaram-se exclusivamente na validação individual de cada componente, antes de qualquer tentativa de integração. Começou por se confirmar uma alimentação estável e a comunicação com o ESP32, verificando se este arrancava corretamente e se este permitia programar, sendo nesta fase alimentado apenas pela porta série do computador, uma solução pensada como temporária mas que viria a manter-se definitiva, durante grande parte do desenvolvimento. (vou adicionar aqui os esquemas elétricos de cada teste individual)

Em seguida, testaram-se isoladamente os servo-motores MG90S, confirmando que estes podiam ser ligados e controlados. Estes testes foram realizados com recurso a uma fonte externa regulável, com um limite de corrente definido em 2 A. Os MG90S apresentam um intervalo de funcionamento especificado pelo fabricante entre 4,8 V e 6 V, sendo que

o tempo de resposta do servo varia em função da tensão escolhida dentro deste intervalo. Por esse motivo, os testes foram realizados alimentando constantemente os servos a 6 V, de modo a obter uma velocidade de resposta favorável para os movimentos pretendidos.

3.4.2 Gestão de energia do Raspberry Pi

Durante os testes foi ainda determinada a transição de três manipuladores para apenas dois manipuladores delta, reduzindo a complexidade face aos resultados pretendidos e diminuindo ainda mais a carga total sobre a fonte de alimentação. A esta simplificação juntou-se a necessidade de proteger o Raspberry Pi de perdas de alimentação inesperadas, um cuidado já conhecido de experiência anterior, uma vez que a perda de energia durante a escrita no cartão SD onde reside o sistema operativo pode corromper esse cartão ou, em casos mais graves, danificá-lo de forma permanente. Por esse motivo, foi selecionada a UPS/HAT Suptronics X1209, que dispõe de duas entradas de alimentação distintas, uma de 5 V e 5 A e outra entre os 6 V e os 18 V até 3 A, permitindo uma maior flexibilidade na escolha e dimensionamento do sistema elétrico e eletrónico. Ao contrário de grande parte das alternativas pesquisadas, que ou se limitavam a placas de gestão de energia bastante dispendiosas, ou eram placas incapazes de assegurar uma transição fiável entre a alimentação de rede e a alimentação por bateria, por serem pensadas apenas para funcionar a partir da bateria, esta HAT combina simultaneamente as funções de UPS e de gestão de energia, o que justificou a sua escolha. Foi associada a uma bateria LiPo de 2100 mA h e 3,7 V, para permitir suportar o Raspberry Pi por tempo suficiente para encerrar corretamente.

3.4.3 Arquitetura e dimensionamento da fonte de alimentação

O dimensionamento da fonte teve por base o consumo dos seis servos MG90S então considerados. Este servo apresenta um consumo nominal entre 150 e 250 mA, mas pode atingir cerca de 700 mA em condição de bloqueio (stall), pelo que, ainda que fosse pouco provável que todos os seis servos entrassem em stall em simultâneo, se optou por dimensionar sempre para o pior caso possível, de forma a garantir o funcionamento do sistema mesmo nas condições mais exigentes, resultando este valor de bloqueio numa corrente total próxima dos 4,2 A para os seis servos, contra apenas 1,7 A em regime nominal. Somando ainda o consumo da UPS/HAT Suptronics X1209, responsável por alimentar o Raspberry Pi, estimado em 5 A, a corrente total estimada para o pior caso possível rondava os 9,2 A, tendo sido por isso escolhida uma fonte de alimentação de 5 V e 10 A, com margem suficiente

acima do valor estimado.

Com a fonte selecionada, foram realizados testes com os dois manipuladores a funcionar em simultâneo, tendo-se observado uma pequena variação de tensão, entre 100 e 200 mV, associada aos picos de corrente na comutação dos servos. Ainda que este valor não pareça elevado, procurou-se minimizá-lo ao máximo, uma vez que o Raspberry Pi é particularmente sensível a variações na sua alimentação. Normalmente, a placa UPS filtra este tipo de perturbação antes que esta afete o Raspberry Pi, mas procurou-se mesmo assim reduzi-la, para evitar os casos raros em que a própria UPS perdesse essa capacidade de filtragem, ainda que por apenas alguns segundos. A pesquisa realizada confirmou ser prática comum a colocação de condensadores de desacoplamento junto à alimentação de cada servo, para atenuar precisamente este tipo de queda de tensão transitória, com a documentação e os fóruns consultados a recomendarem valores tipicamente entre os 230 e os 700 μF . Optou-se por condensadores de 470 μF junto de cada servo. (vou adicionar imagens sobre isto)

3.4.4 Da protoboard à PCB

Depois de concluídos os testes do circuito numa protoboard, o mesmo foi transferido para uma PCB pré-furada, onde foram implementados os drivers referidos anteriormente, um por cada manipulador delta, cada um responsável por controlar os três servos correspondentes. Foi também montada e soldada uma PCB própria para alojar o microcontrolador ESP32-S3. Cada driver recebe os comandos enviados pelo ESP32 e a alimentação diretamente da fonte principal, distribuída através de um barramento comum a cada um dos três servos, sempre por intermédio do respetivo condensador de desacoplamento. O ESP32, por sua vez, pode ser alimentado pelo Raspberry Pi através de USB-C, mantendo simultaneamente a ligação série necessária à comunicação.

3.5 Integração mecânica

Esta secção aborda a integração mecânica final do sistema, que passou pela união entre o modelo físico da árvore brônquica e os dois manipuladores delta, através de encaixes e estruturas de suporte dedicadas, bem como pela construção de uma caixa própria para a HMI. Em ambos os casos, procurou-se obter um produto final mais acabado, com cavidades próprias para embutir toda a eletrónica associada, tanto no modelo físico como

na HMI.

Antes de qualquer integração com os manipuladores, o modelo físico da árvore brônquica dispunha apenas de um suporte simples, sem qualquer função de teste, cujo propósito se limitava a permitir a sua validação individual e, junto da equipa médica, a avaliação da representação anatómica do modelo 3D.

A integração com os manipuladores delta seguiu depois um percurso próprio ao longo do projeto. Numa primeira fase, cada manipulador foi montado sobre uma base independente, sem qualquer ligação entre si, o que permitiu conduzir testes individuais de encaixe para validar a estrutura, aproveitando e adaptando parte do trabalho já disponível no repositório de código aberto *How to Build Your Robot* [59].

Nas iterações seguintes, os manipuladores passaram a dispor de encaixes próprios, desenhados e montados separadamente da base principal do suporte, de modo a permitir substituir apenas um dos lados caso fosse necessária alguma modificação futura, sem afetar o restante conjunto. Esta unificação numa base comum reduziu significativamente o tempo de montagem e desmontagem do sistema completo, além de manter fixa a posição relativa entre os dois manipuladores, um requisito importante para a repetibilidade dos movimentos representados no modelo. Nessa mesma altura foi ainda incluída uma cavidade destinada a toda a eletrónica (fonte de alimentação, drivers e microcontrolador), uma solução que se revelou pouco versátil, por exigir uma estrutura demasiado larga para os acomodar a todos.

Na versão final, o suporte passou a incluir apenas o espaço necessário para os encaixes dos manipuladores, um suporte vertical para o modelo impresso e uma cavidade destinada exclusivamente ao microcontrolador e aos drivers, tendo a fonte de alimentação e as respetivas ligações sido transferidas para a caixa da HMI. Esta estrutura foi sendo constantemente iterada, de modo a compactar cada vez mais o conjunto completo do modelo físico e da HMI. Todo este processo foi acompanhado de testes de encaixe e calibrações sucessivas, de modo a alcançar uma estrutura mais limpa e acabada.

Os componentes estruturais próprios de cada manipulador delta, como a base circular, os braços superiores, os elos de ligação e a plataforma móvel central, foram inicialmente impressos em PLA para uma primeira verificação de encaixes e tolerâncias, tendo sido mais tarde substituídos por PETG preto. A montagem final do conjunto, representada na figura 3.7, concentrou os dois manipuladores, o modelo brônquico e a base de suporte numa estrutura única, reduzindo folgas e tornando a posição relativa entre os atuadores mais repetível.



Figura 3.7: Sistema mecânico final com os dois manipuladores delta.

Para acomodar o Raspberry Pi 4B e a fonte de alimentação de forma compacta e robusta, foi projetada e impressa uma caixa em PETG. Esta estrutura agrupa o Raspberry Pi 4B, o monitor de 7 polegadas, a fonte de alimentação de 5 V e 10 A, a placa UPS com a respetiva bateria LiPo, um adaptador para os pinos GPIO do Raspberry Pi, o adaptador USB-C e toda a cablagem existente para sustentar a HMI.

Capítulo 4

Desenvolvimentos do Software e Configurações

O software foi desenvolvido em paralelo com o hardware, mas em camadas progressivas. Começou por testes elementares de comunicação série para confirmar que o ESP32 e o Raspberry Pi conseguiam trocar dados de forma fiável. A partir daí foram adicionados, um a um, os blocos de geração de trajetórias, de máquinas de estados, de cinemática inversa e de interface gráfica. Este capítulo descreve essa construção por partes, da arquitetura global até aos detalhes de calibração e sincronização, seguindo a ordem em que cada bloco foi desenvolvido e validado.

4.1 Arquitetura geral do software e ambiente de desenvolvimento

O software do projeto foi dividido em dois sistemas distintos que comunicam entre si através de um único canal de comunicação série bidirecional. O primeiro é o firmware do ESP32-S3, responsável pela execução e gestão física dos movimentos. O segundo é a HMI, alojada num Raspberry Pi 4B, responsável pela interação com o utilizador. A figura 4.1 ilustra esta relação de alto nível entre os dois sistemas.



Figura 4.1: Esquema global de comunicação entre o firmware do ESP32-S3 e a HMI no Raspberry Pi 4B.

O firmware do ESP32 foi inteiramente desenvolvido no ambiente PlatformIO, integrado no Visual Studio Code. Esta escolha surgiu numa altura de transição natural do projeto: durante muito tempo, a programação de microcontroladores tinha sido feita

através do Arduino IDE, sempre em conjunto com placas Arduino, sendo que a adoção do ESP32 neste projeto se tornou a oportunidade ideal para migrar para um ambiente mais adequado. Ao contrário do Arduino IDE, que antigamente exigia a instalação de definições externas para reconhecer placas fora do ecossistema Arduino, o PlatformIO oferece suporte nativo à quase totalidade das placas atualmente disponíveis no mercado. A esta vantagem juntam-se ainda funcionalidades como sugestões inteligentes de código em tempo real, semelhantes a um sistema de auto-completar avançado, a integração direta com o Git através do próprio Visual Studio Code, a possibilidade de trabalhar simultaneamente com várias linguagens no mesmo ambiente e a verificação de erros em tempo real durante a escrita do código. Ainda que a curva de aprendizagem inicial seja mais exigente do que a do Arduino IDE, a prática rapidamente revela um ambiente de desenvolvimento consideravelmente mais poderoso e completo.

A HMI, por sua vez, foi maioritariamente desenvolvida em Python 3, por ser uma linguagem com acesso a diversas bibliotecas dedicadas ao desenvolvimento de interfaces e aplicações, facilitando consideravelmente o processo de desenvolvimento.

O desenvolvimento da interface foi inicialmente repartido em pequenas funcionalidades independentes, começando apenas pelas mais importantes: a comunicação série do lado do Python, o cálculo de pontos das trajetórias e a estrutura principal da própria interface. Todo este desenvolvimento inicial foi executado e iterado no IDE Thonny, escolhido pela sua interface simples e pela curva de aprendizagem reduzida, adequada a esta fase inicial de testes. Mais tarde, todo o desenvolvimento em Python foi migrado para um ambiente dentro do próprio Visual Studio Code, o que permitiu reunir os códigos de ambos os sistemas e dispositivos, bem como todas as configurações e a documentação associada, num único repositório, facilitando o transporte, a execução e a partilha entre dispositivos, bem como o acompanhamento da evolução do projeto através do Git.

4.2 Arquitetura do firmware ESP32-S3

O firmware do ESP32-S3 foi concebido como um sistema independente de controlo em tempo real, responsável por receber e interpretar os comandos da HMI, gerir o estado interno dos movimentos, executá-los de forma coordenada e precisa e, por fim, converter as trajetórias pretendidas em sinais próprios para os atuadores dos manipuladores delta, os servo-motores MG90S. Por este motivo, a maior parte dos blocos mais importantes do código foram construídos recorrendo a máquinas de estados (FSM), uma abordagem que confere ao firmware uma natureza flexível e independente, sem comprometer a precisão nem a eficiência da execução.

Esta arquitetura não partiu do zero, sendo utilizado como ponto de partida um exemplo de referência para manipuladores delta [60], disponível publicamente e adaptado à geometria e aos servos usados neste projeto. Depois de confirmado o funcionamento

básico da cinemática e do acionamento individual dos servos, esse código de referência foi sendo progressivamente modificado e organizado para complementar as necessidades do projeto à medida que eram introduzidos os outros blocos de funções, de forma separada do resto do funcionamento do código, o que permitiu reescrever cada bloco, estruturas de dados, geração de movimento e máquinas de estados de forma independente.

O código está assim organizado em dois tipos de blocos: blocos de máquinas de estados e blocos de funções. Entre as FSM contam-se a `FSM_Serial_Reader`, a `FSM_Serial_Command`, a `FSM_Tosse_trigger`, a `FSM_Monitor_trigger`, a `FSM_Preparação_Tosse` e, sobretudo, a `FSM_Motion_update`, responsável pela execução dos movimentos. Entre os blocos de funções destacam-se a `Intermed_position()`, a `Fusao_plus_IK_D1()` e a `Fusao_plus_IK_D2()`, além das estruturas de dados que suportam todo o código, como `Coordinate`, `MotionStorage` e `MotionInstance`.

Muitos destes blocos foram construídos de forma flexível face ao tipo de variáveis com que trabalham, permitindo reutilizar uma única função para vários tipos de movimento em situações distintas, sem replicar a mesma lógica várias vezes. É o caso da `FSM_Motion_update` e da `Intermed_position`, chamadas de forma independente para a respiração, o batimento cardíaco e a tosse, alterando apenas os parâmetros e as trajetórias associadas a cada caso.

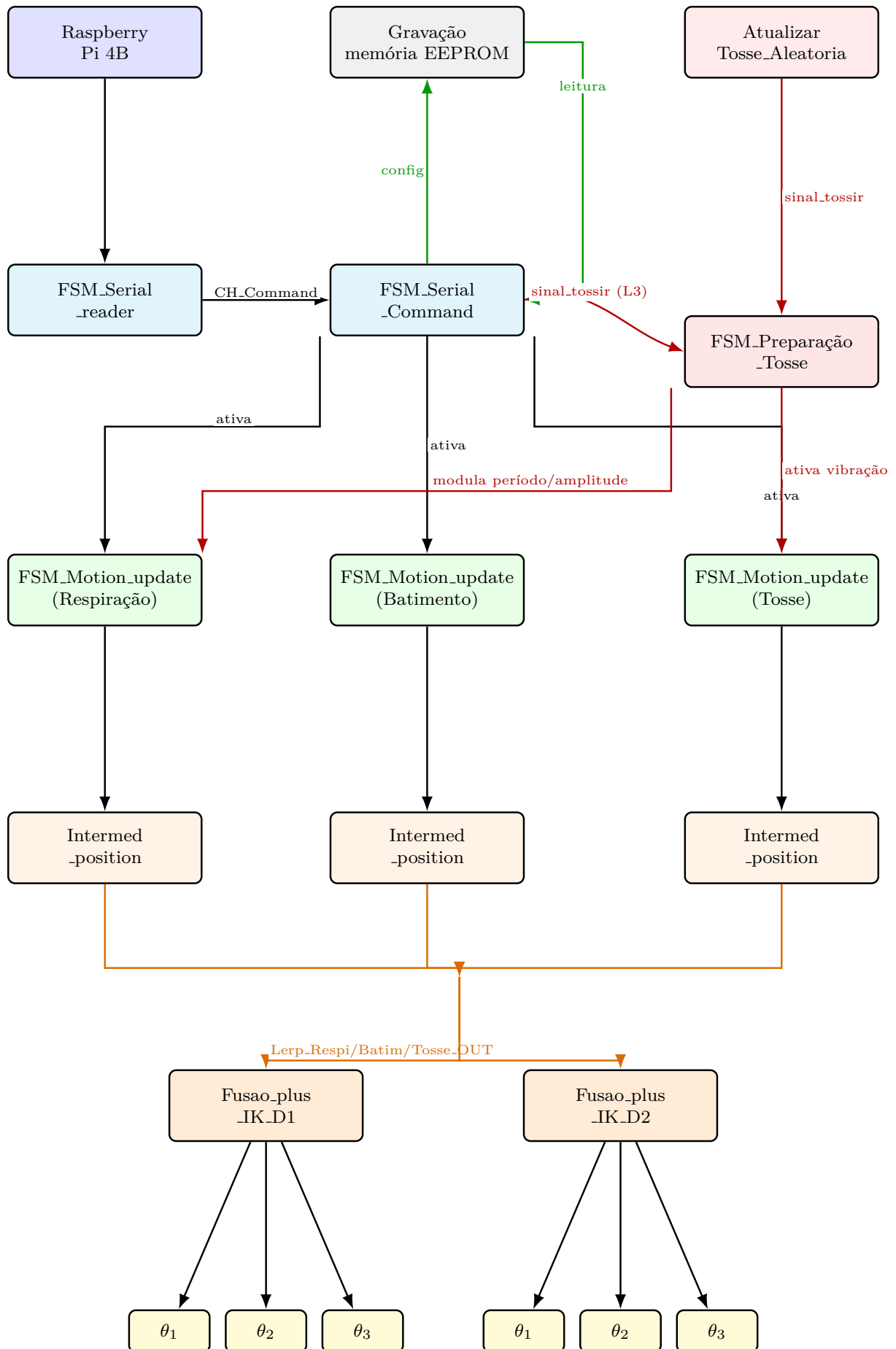


Figura 4.2: Estrutura global das máquinas de estados e blocos de controlo do firmware.

O percurso dos dados começa na `FSM_Serial_Reader`, que funciona como camada de receção global, recolhendo os caracteres vindos da HMI sem perturbar a execução dos movimentos. Depois de recebidos, os comandos são redirecionados, um carácter de cada vez ou numa única sequência de texto, para a `FSM_Serial_Command`, onde são distinguidos comandos simples, como o arranque, a paragem, a seleção de modo, a gravação e leitura de dados na memória EEPROM ou até a verificação dos dados nas estruturas internas, dos comandos-chave parametrizados, usados apenas para alterar e atualizar informações de configuração e definições. É através destes últimos que são atualizadas as flags de funcionamento e as estruturas de movimento, refletindo de imediato qualquer alteração na configuração ativa do sistema.

Em paralelo, a `FSM_Motion_update` de cada fenómeno fisiológico ativo (respiração, batimento cardíaco e vibração da tosse) percorre a respetiva trajetória discreta, usando apenas índices, direção, pausas e tempos internos. O sinal que despoleta a tosse (`sinal_tossir`) pode ser gerado automaticamente pela função `Atualizar_Tosse_Aleatoria`, de forma aleatória, ou manualmente através de um comando série processado pela `FSM_Serial_Command`, cabendo depois à `FSM_Preparação_Tosse` converter esse sinal na execução física da tosse, ajustando temporariamente a respiração e ativando a componente de vibração.

Nesta fase, as posições de cada movimento ainda são apenas índices, sem qualquer significado direto para os manipuladores. Por isso os dados dos índices são redirecionados para a função `Intermed_position`, onde são convertidos em valores de real significado (coordenadas cartesianas). Em sequência a esta conversão são associados os valores do índice atual e o índice futuro e calculado uma coordenada intermédia (interpolação) entre o ponto atual e o próximo, em função do tempo decorrido e do tempo até ao ponto seguinte, resultando numa saída de posições limpa, suave e precisa ao longo de toda a simulação.

No final do sistema, os blocos de fusão de coordenadas e de cinemática inversa, `Fusao_plus_IK_D1` e `Fusao_plus_IK_D2`, reúnem toda a informação proveniente dos movimentos ativos e convertem a posição resultante (ainda em coordenadas cartesianas), nos ângulos a alcançar em tempo real por cada um dos atuadores dos dois manipuladores delta.

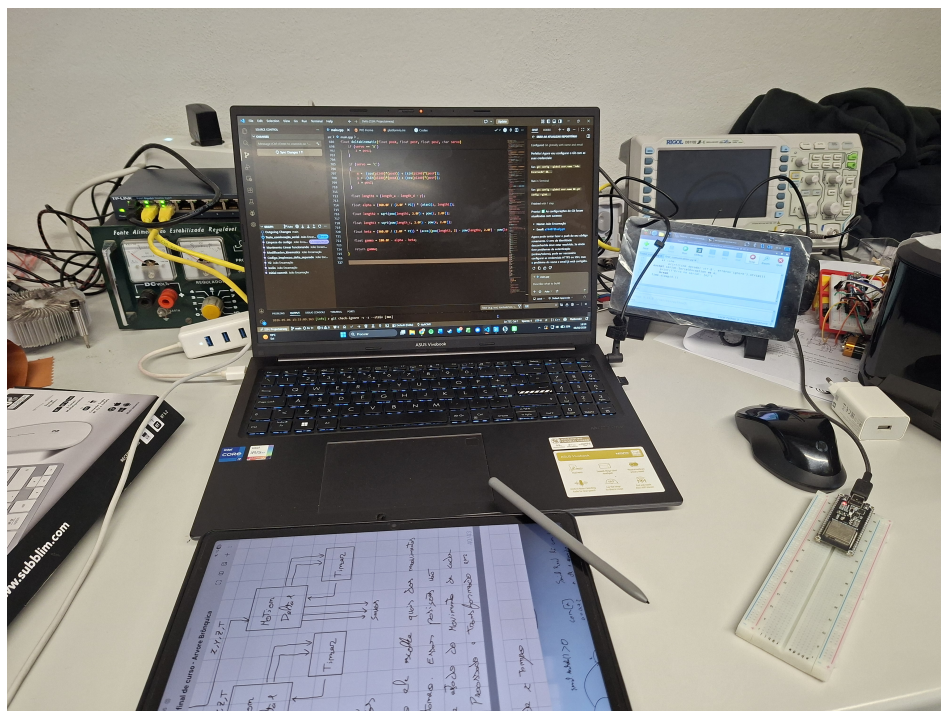


Figura 4.3: Desenvolvimento da arquitetura principal do ESP32.

4.3 Movimentos representados no protótipo

O estado da arte identificou vários movimentos observáveis durante a broncoscopia. No entanto, para o projeto, foram selecionados apenas os movimentos com melhor relação entre relevância clínica, viabilidade mecânica e simplicidade de controlo: respiração, batimento cardíaco e tosse. Os valores de frequência e amplitude usados nas secções seguintes, resumidos na tabela 4.1, foram retirados diretamente do documento de trabalho fornecido pela equipa de Pneumologia do CHUA [28].

Tabela 4.1: Movimentos considerados para o projeto.

Movimento	Frequência	Amplitude pre- vista	Observações
Respiração	12–20 ciclos/min	5–15 mm	Predominantemente craniocaudal, com uma componente lateral associada
Batimento cardíaco	60–120 bpm	1–3 mm	Compressão direcional localizada, com contração breve e pausa variável
Tosse	Evento curto	5–25 mm	Abrupto e irregular, altera temporariamente a respiração

A respiração foi descrita com base num deslocamento predominantemente craniocaudal (vertical), como um movimento cíclico, de frequência aproximada de 12 a 20 ciclos por minuto e amplitude extensa, de cerca de 5 a 15 mm. A este deslocamento principal associa-se ainda uma componente lateral de menor magnitude, confirmada em modelos computacionais do movimento brônquico ao longo do ciclo respiratório, nos quais o deslocamento lateral surge sistematicamente depois da predominância craniocaudal [29]. Do ponto de vista da modelação, esta componente lateral pode ser entendida como resultante do facto de a árvore brônquica não acompanhar em linha reta o movimento vertical imposto pelo diafragma, e sim deslizar lateralmente de forma perceptível à medida que a base pulmonar desce e sobe, o que resulta numa trajetória com uma curvatura suave em vez de um simples deslocamento vertical linear.

Já o batimento cardíaco foi associado a um deslocamento predominante no brônquio principal esquerdo, devido à proximidade anatômica do coração, prevendo-se um movimento de baixa amplitude, cerca de 1 a 3 mm, com frequência aproximada de 60 a 120 batimentos por minuto. Esta proximidade leva a que o coração e os vasos sanguíneos junto dele entrem em contacto direto com o brônquio, comprimindo-o periodicamente a cada batimento, um efeito também documentado em observações por broncoscopia [61] e confirmado junto da equipa de Pneumologia do CHUA. A compressão resultante afeta

tipicamente o brônquio como um todo e não apenas um ponto localizado, ocorrendo preferencialmente na direção lateral, com uma contribuição secundária no sentido ântero-posterior, em vez de se distribuir uniformemente em torno da sua circunferência [28]. Por essa razão, o percurso do movimento cardíaco não é um simples movimento retilíneo, traçando antes um pequeno percurso fechado a cada ciclo. No protótipo, este movimento foi por isso representado como uma oscilação fechada, alongada preferencialmente na direção lateral, sobreposta ao movimento respiratório.

O ciclo cardíaco em si também não é temporalmente simétrico, sendo composto por duas fases principais. A fase sistólica (o momento de contração do coração) tem uma duração absoluta, relativamente curta, mantendo-se na ordem das poucas centenas de milissegundos. Pelo contrário, é a fase diastólica (o intervalo de repouso entre contrações sucessivas), que absorve quase toda a variação do período à medida que a frequência cardíaca aumenta, encolhendo de forma muito mais acentuada do que a sístole [62]. Esta relação entre uma contração breve e praticamente constante e uma pausa mais longa e variável consoante a frequência cardíaca foi também confirmada junto da equipa de Pneumologia do CHUA. Contudo, esta variação periódica, tanto na respiração como no batimento cardíaco, não foi replicada no projeto, tendo-se optado por uma oscilação de velocidade constante ao longo de todo o período de treino, de modo a simplificar o cálculo e a calibração do movimento sem comprometer o seu contributo visual (sendo ainda possível alterar os valores de período, mas apenas fora da execução de treino).

A tosse foi caracterizada como um evento abrupto, irregular e de maior amplitude relativa e, em vez de ser apenas uma trajetória independente, deve alterar temporariamente o comportamento respiratório, aumentando a amplitude e introduzindo uma perturbação rápida.

4.4 Geração de trajetórias (Python/Raspberry Pi)

Com os requisitos clínicos de cada movimento já definidos na secção anterior, o passo seguinte consistiu em traduzir essa descrição qualitativa em trajetórias discretas, calculáveis e executáveis pelo modelo.

Essa tradução foi implementada inteiramente em ambiente de Python com auxílio da biblioteca matplotlib, sendo responsável por transformar os parâmetros configuráveis de cada movimento, primeiramente em curvas, de seguida em pontos e finalmente em gráficos que futuramente seriam reaproveitados e apresentados diretamente na HMI. Este cálculo foi inicialmente desenvolvido fora da componente de interface, para permitir validar, ainda numa fase inicial do desenvolvimento de hardware e software, a geração de curvas e de pontos em coordenadas cartesianas admissíveis para posterior inserção no firmware do ESP32. Uma decisão de arquitetura particularmente relevante nesta fase foi a escolha de descrever cada movimento recorrendo apenas a um plano 2D específico do

espaço 3D do manipulador (por exemplo, XY, XZ ou YZ), em vez de calcular antecipadamente, em Python, a fusão completa de todos os movimentos numa única trajetória tridimensional já pronta para o manipulador. Esta segunda abordagem foi de facto a primeira ideia considerada e manteve-se como hipótese de trabalho durante bastante tempo, consistindo em gerar previamente uma trajetória global fundida, resultando numa única matriz de posições a enviar ao manipulador.

Esta abordagem revelou-se, no entanto, pouco flexível para desenvolvimentos futuros, uma vez que exigiria que o período do batimento cardíaco fosse sempre um múltiplo inteiro do período da respiração, de modo a que a trajetória fundida se repetisse de forma consistente ao longo do tempo. Como os dois movimentos não têm uma relação de período exata entre si, cada secção da trajetória teria um tempo diferente, tornando impossível prever antecipadamente, ao longo do array de pontos, todos os instantes em que o batimento poderia ocorrer relativamente à respiração. No caso da tosse, cuja ocorrência é ainda menos previsível, esta abordagem seria mesmo impossível.

A possibilidade de descrever cada movimento recorrendo apenas a um plano 2D específico, dentro do espaço 3D mais amplo do manipulador, resolveu este problema. Esta escolha permitiu transferir a etapa de fusão dos movimentos para o próprio ESP32, executando-a quase em tempo real e possibilitando que modificações e eventos, como a tosse, ocorressem diretamente sobre essa fusão, sem necessidade de calcular antecipadamente planos ou simulações complexas para um único movimento universal do manipulador.

4.4.1 Desenvolvimento e iterações

Estabelecida esta decisão de arquitetura, passou-se para a execução, validação e iteração sucessiva do cálculo de cada curva, dando-se início com a implementação da geração de curvas e cálculo de pontos, sendo o batimento cardíaco o primeiro movimento de todos a ser descrito e testado em formato de curva no script. Foi utilizada a fórmula de uma elipse para descrever o deslocamento cardíaco, definida por dois parâmetros geométricos principais, o comprimento e a largura, que permitiam controlar diretamente o formato da curva através dos semieixos a e b :

$$x_0(u) = a \cos(u), \quad y_0(u) = b \sin(u).$$

Esta elipse-base necessitou ainda de duas características adicionais: a capacidade de se posicionar em qualquer ponto do gráfico (para permitir recentrar a curva relativamente à origem) e a capacidade de rodar em torno desse mesmo ponto central.

A rotação foi implementada com base na matriz de rotação convencional,

$$R(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix},$$

aplicada ao ponto $(x_0(u), y_0(u))$ da ellipse-base, resultando em

$$\begin{bmatrix} x_r(u) \\ y_r(u) \end{bmatrix} = R(\alpha) \begin{bmatrix} x_0(u) \\ y_0(u) \end{bmatrix} = \begin{bmatrix} a \cos(u) \cos(\alpha) - b \sin(u) \sin(\alpha) \\ a \cos(u) \sin(\alpha) + b \sin(u) \cos(\alpha) \end{bmatrix}.$$

Por fim, a posição da ellipse no gráfico é determinada pelas coordenadas (x_c, y_c) , correspondentes ao centro da ellipse. Este deslocamento é livre e completamente independente da rotação $R(\alpha)$: o ângulo α orienta apenas a forma da ellipse em torno do seu próprio centro, podendo esse centro ser colocado em qualquer ponto do gráfico. Só na função completa, apresentada adiante, estas coordenadas são somadas ao ponto já rodado da ellipse, posicionando a ellipse rodada no local pretendido do gráfico.

Na segunda iteração foi acrescentada a curva parabólica usada pela respiração e pela tosse que, por se apresentar no plano ZX , exigiu duas correções: garantir que a parábola abria sempre no sentido de Z^+ (de acordo com o sentido físico do movimento representado) e ajustar a escala dos eixos gráficos para que a proporção da curva fosse mantida corretamente, através do comando `set_aspect('equal', adjustable='box')` em vez de `axis('equal')`, este último incompatível com os limites de eixo definidos manualmente. A curva base, comum aos dois movimentos, é dada por:

$$X_0(u) = u, \quad Z_0(u) = \frac{H}{L^2} u^2, \quad 0 \leq u \leq L,$$

em que H é a altura máxima e L é a largura máxima. Na terceira versão foi introduzido um sistema de controlo do sentido do percurso na curva elíptica do batimento cardíaco (horário ou anti-horário). Nesta fase, sempre que os valores de a e b eram alterados, o reposicionamento da ellipse, de modo a manter o vértice de referência junto à origem do gráfico, tinha ainda de ser feito manualmente, ajustando x_c e y_c .

4.4.2 Módulo final

Com todas as bases implementadas, foi construído em torno delas o módulo `calculo_movimentos.py`, que permite introduzir parâmetros geométricos, o número de pontos e outras calibrações. Através de fórmulas específicas para cada movimento, este módulo devolve sequências de coordenadas no espaço tridimensional, numa ordem e orientação específicas, já preparadas para serem utilizadas e fundidas no firmware do ESP32.

De forma geral, cada movimento é definido por uma curva contínua $\mathbf{p}(u)$ e por um

conjunto discreto de N pontos:

$$\mathbf{p}_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} = \mathbf{p}(u_i), \quad i = 0, 1, \dots, N - 1. \quad (4.1)$$

No módulo `calculo_movimentos.py`, a posição da elipse do batimento cardíaco passou a ser descrita por um deslocamento orbital, em vez das coordenadas independentes (x_c, y_c) das versões anteriores, com o objetivo de facilitar o cálculo e a parametrização da curva. Nesta representação, o centro da elipse é definido por uma distância R à origem do gráfico e pelo mesmo ângulo α usado anteriormente, sendo a orientação da própria elipse dada por $\alpha + \delta$, em que δ é um pequeno ajuste fino da rotação interna da elipse (por padrão, $\delta = 0^\circ$). A curva final é dada por:

$$X(u) = R \cos(\alpha) + a \cos(u) \cos(\alpha + \delta) - b \sin(u) \sin(\alpha + \delta), \quad (4.2)$$

$$Y(u) = R \sin(\alpha) + a \cos(u) \sin(\alpha + \delta) + b \sin(u) \cos(\alpha + \delta), \quad (4.3)$$

$$Z(u) = 0, \quad (4.4)$$

$$\{0 \leq u < 2\pi\}$$

podendo o sentido de percurso ser horário ou anti-horário. Para manter o vértice de referência da elipse sempre ancorado junto à origem do gráfico, independentemente do ângulo α escolhido, o valor de R é calculado automaticamente a partir de a , b e δ :

$$R = \frac{ab}{\sqrt{b^2 \cos^2(\delta) + a^2 \sin^2(\delta)}}.$$

No conjunto de parâmetros usado nos testes da interface, foram considerados os diâmetros $a = 3,5 \text{ mm}$ e $b = 1,5 \text{ mm}$, $\delta = 0^\circ$, $\alpha = 20^\circ$, $N = 16$ pontos, sentido horário e um período $T = 0,3 \text{ s}$. Para estes parâmetros, o valor de R calculado automaticamente foi $R = 3,50 \text{ mm}$.

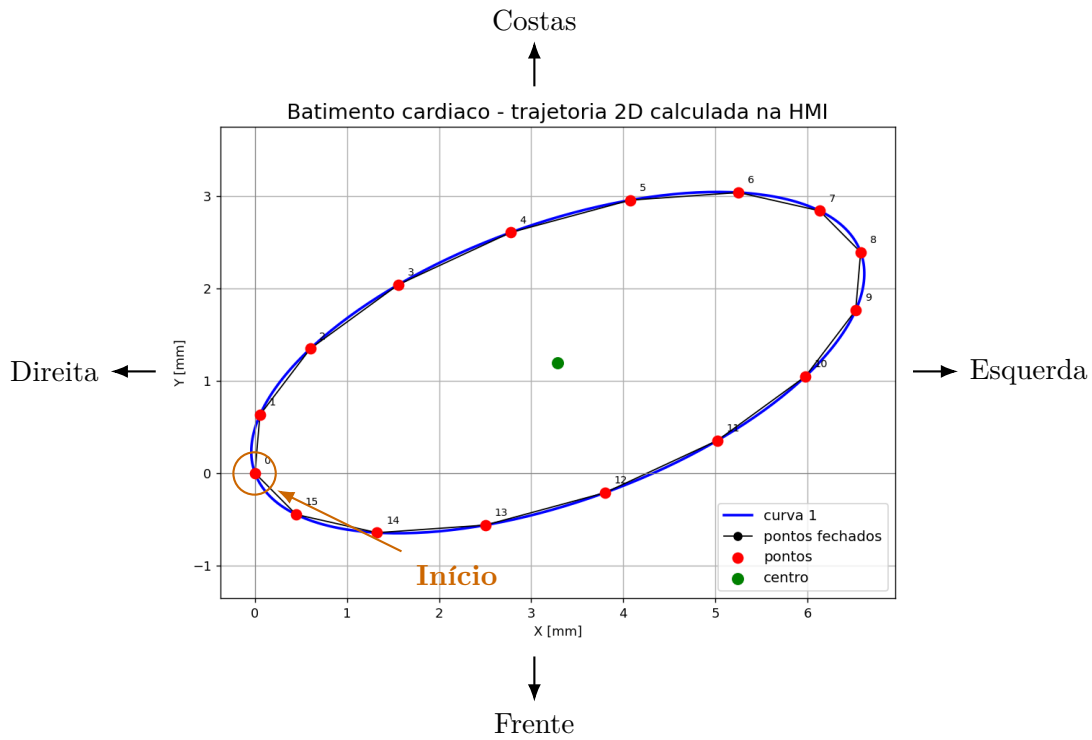


Figura 4.4: Trajetória calculada para o movimento de batimento cardíaco, representação 2D dos pontos de passagem.

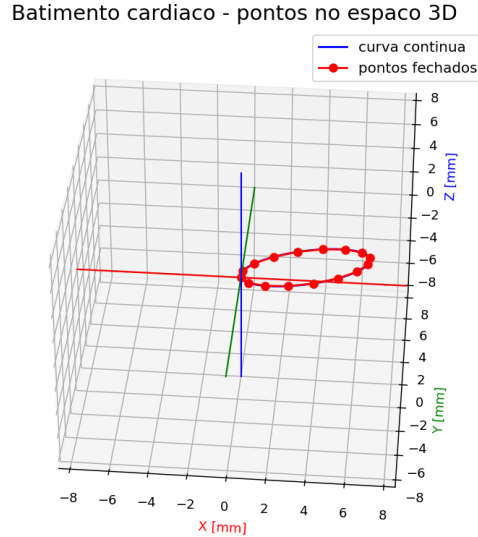


Figura 4.5: Trajetória calculada para o movimento de batimento cardíaco, representação 3D dos pontos de passagem.

Como explicado anteriormente, para a respiração foi utilizada a mesma parábola, no plano ZX , de modo a captar o acoplamento entre a componente craniocaudal dominante e a ligeira componente lateral. Ao longo da curva, a coordenada Z cresce de forma acentuada, refletindo o deslocamento vertical imposto pelo diafragma, com variação lenta no início e mais acentuada no final do trajeto, enquanto a coordenada X avança de forma

aproximadamente constante, traduzindo o deslizar lateral da árvore brônquica associado a esse movimento. A mesma curva é também usada como deslocamento rápido associado à tosse, alterando apenas o período de duração e o momento de ativação do evento. Dentro do cálculo da curva de respiração existe ainda uma componente de ajuste que permite configurar a inclinação do plano ZX em relação ao eixo X , possibilitando orientar o deslocamento do movimento respiratório para um verdadeiro deslocamento crânio-caudal, caso necessário (por padrão, esta componente encontra-se neutra/desativada):

$$X_{3D}(u) = u, \quad (4.5)$$

$$Y_{3D}(u) = \frac{H}{L^2} u^2 \sin(\theta), \quad (4.6)$$

$$Z_{3D}(u) = \frac{H}{L^2} u^2 \cos(\theta), \quad (4.7)$$

$$\{0 \leq u \leq L\}$$

o mesmo intervalo já usado na curva base. Por estar limitado a este intervalo, o parâmetro u nunca é negativo, pelo que nenhuma das três coordenadas assume valores negativos: todas partem de 0, em $u = 0$ e crescem até ao respetivo valor máximo, atingido em $u = L$, sendo esse máximo igual a L para X_{3D} e igual a H para o conjunto de Y_{3D} e Z_{3D} (repartido entre as duas consoante o ângulo θ). Nos valores usados nos testes, foram considerados $H = 20\text{ mm}$, $L = 12\text{ mm}$, $N = 10$ pontos, $\theta = 0^\circ$, um período $T = 4\text{ s}$, pausa inicial de $0,4\text{ s}$, pausa final de $0,1\text{ s}$ e ponto inicial igual a 5, este último definido para iniciar a trajetória a partir de uma posição intermédia.

Respiracao

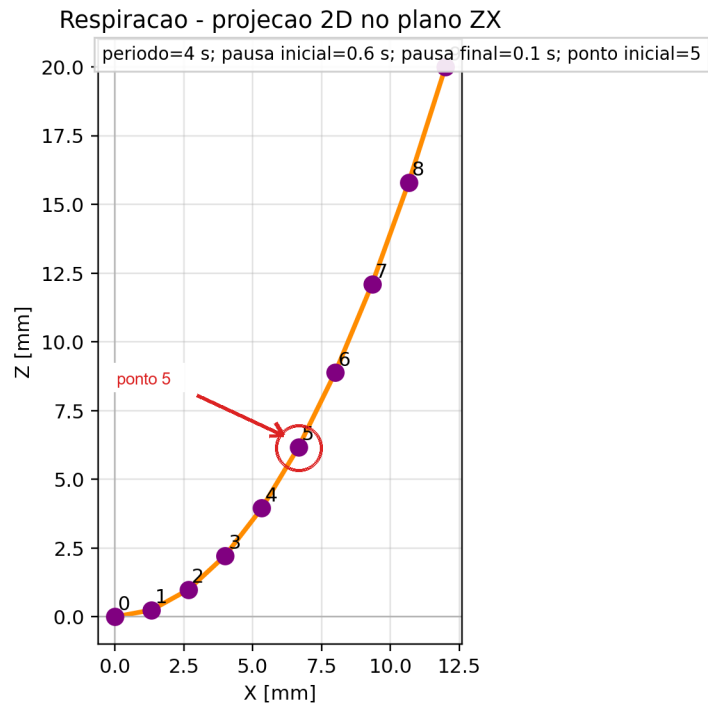


Figura 4.6: Trajetória calculada para o movimento respiratório, representação 2D no plano ZX.

Respiracao - vista 3D

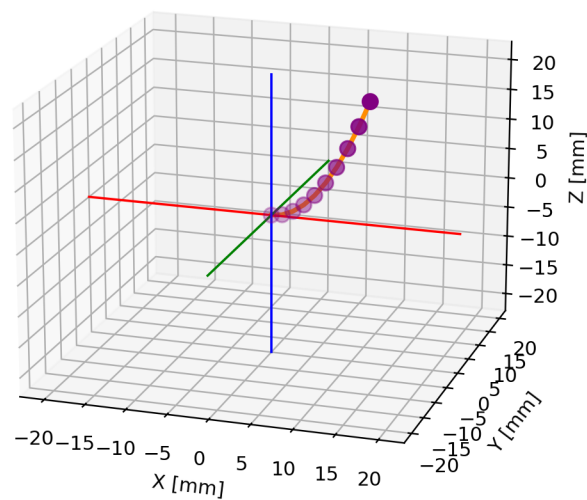


Figura 4.7: Trajetória calculada para o movimento respiratório, visualização 3D.

Além da alteração do movimento respiratório, a tosse inclui uma vibração curta em torno da posição de referência. Esta vibração foi representada, na sua forma base, por

uma circunferência no plano XY , sendo esta curva ainda modificada durante a execução no firmware do ESP32, de modo a produzir uma forma de vibração final com um comportamento mais irregular e aproximadamente aleatório:

$$X(u) = r \cos(u), \quad (4.8)$$

$$Y(u) = r \sin(u), \quad (4.9)$$

$$Z(u) = 0. \quad (4.10)$$

O raio r controla a amplitude da vibração, o número de pontos controla a resolução da trajetória e o período define a rapidez do evento, tendo sido usado, no caso apresentado, $r = 3 \text{ mm}$, $N = 10$ pontos, sentido horário e período de $0,2 \text{ s}$.

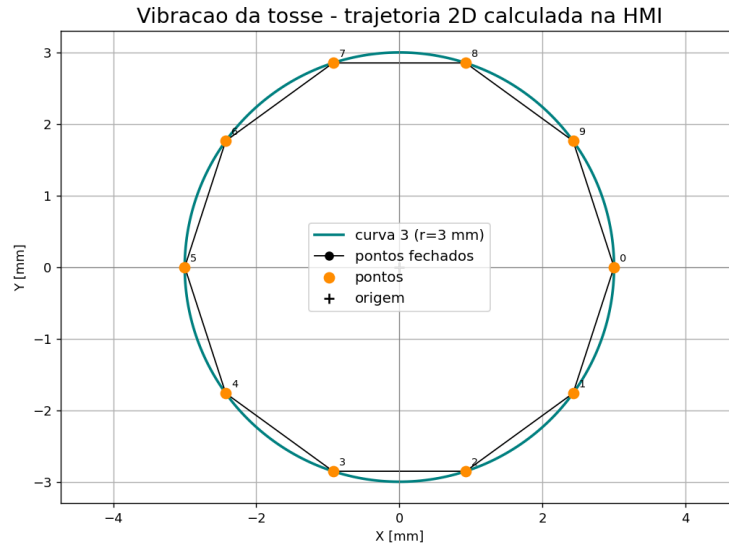


Figura 4.8: Trajetória calculada para a vibração associada à tosse, representação 2D dos pontos de passagem.

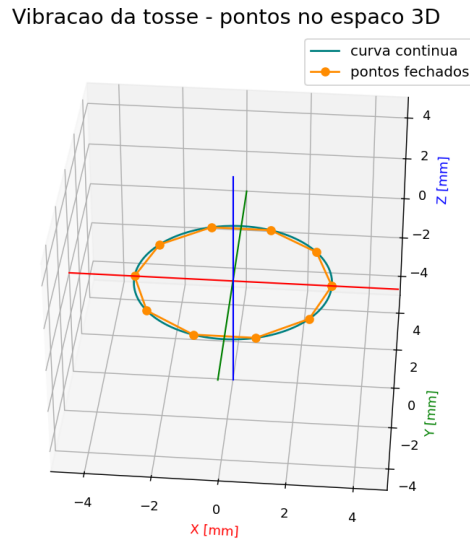


Figura 4.9: Trajetória calculada para a vibração associada à tosse, representação 3D dos pontos de passagem.

Tabela 4.2: Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.

Movimento	Parâmetros geométricos	Parâmetros temporais e de execução
Batimento cardíaco	a , b , R , α , número de pontos e sentido da elipse	Período, pausa inicial, pausa final e ponto inicial
Respiração	Altura H , largura L , ângulo θ e número de pontos da parábola	Período, pausa inicial, pausa final, ponto inicial e modo bidirecional
Vibração da tosse	Raio r , número de pontos e sentido da circunferência	Período, pausa inicial, pausa final e ponto inicial

Como o número de pontos é um parâmetro configurável em cada movimento, as trajetórias não precisam de ter obrigatoriamente a mesma discretização. A tabela 4.3 apresenta o conjunto de pontos usado no teste documentado, calculado diretamente pelo módulo Python da HMI, com 16 pontos para o batimento cardíaco e 10 pontos para os restantes dois movimentos (assinalados com um traço nas linhas em que não têm ponto correspondente).

Tabela 4.3: Pontos calculados para os movimentos usados nos testes da interface.

Ponto	Batimento cardíaco			Respiração			Vibração da tosse		
	X	Y	Z	X	Y	Z	X	Y	Z
0	0,00	0,00	0,00	0,00	0,00	0,00	3,00	0,00	0,00
1	0,05	0,63	0,00	1,33	0,00	0,25	2,43	-1,76	0,00
2	0,60	1,35	0,00	2,67	0,00	0,99	0,93	-2,85	0,00
3	1,56	2,04	0,00	4,00	0,00	2,22	-0,93	-2,85	0,00
4	2,78	2,61	0,00	5,33	0,00	3,95	-2,43	-1,76	0,00
5	4,07	2,96	0,00	6,67	0,00	6,17	-3,00	0,00	0,00
6	5,25	3,04	0,00	8,00	0,00	8,89	-2,43	1,76	0,00
7	6,13	2,84	0,00	9,33	0,00	12,10	-0,93	2,85	0,00
8	6,58	2,39	0,00	10,67	0,00	15,80	0,93	2,85	0,00
9	6,52	1,76	0,00	12,00	0,00	20,00	2,43	1,76	0,00
10	5,98	1,05	0,00	—	—	—	—	—	—
11	5,02	0,35	0,00	—	—	—	—	—	—
12	3,80	-0,21	0,00	—	—	—	—	—	—
13	2,50	-0,56	0,00	—	—	—	—	—	—
14	1,33	-0,65	0,00	—	—	—	—	—	—
15	0,45	-0,45	0,00	—	—	—	—	—	—

4.5 Estruturas de armazenamento de dados

O funcionamento do firmware depende de um conjunto reduzido de estruturas que organizam a informação usada durante a execução. Esta organização foi importante para evitar que os parâmetros geométricos, os pontos de trajetória e o estado temporário das máquinas de estados ficassem dispersos pelo código principal, o que dificultaria tanto a calibração como a alteração posterior dos movimentos.

As estruturas mais importantes para o controlo dos manipuladores e para a execução das trajetórias começam pela `Manipulador_Config`, resumida na figura 4.10 e concebida para concentrar toda a informação referente a cada manipulador Delta, tanto configurações como dados internos. Nela encontram-se definidos os comprimentos das várias ligações e dimensões do manipulador ($L1$, $L2$, $L3$), os deslocamentos associados à posição e à rotação global do manipulador, os limites mecânicos admissíveis, expressos em ângulos, os intervalos de PWM calibrados para cada servo, bem como variáveis internas de apoio ao próprio cálculo de IK. Assim, a cinemática inversa pode ser aplicada aos dois manipuladores a partir da mesma lógica matemática, mudando apenas os parâmetros específicos de montagem e calibração de cada conjunto.

O armazenamento da informação de cada movimento foi deliberadamente dividido

Manipulador_Config
L1, L2, L3
servoOffsetX, servoOffsetZ
angleMin, angleMax
rotationOffsetZ
servo1/2/3: MinPulse, MaxPulse
theta1/2/3: MinPulse, MaxPulse
servo1/2/3Pulse (saída IK)

Figura 4.10: Estrutura Manipulador_Config.

em duas estruturas distintas, uma fixa e outra dinâmica, em vez de reunir tudo numa única estrutura. Esta separação torna o seu uso mais versátil, já que o sistema, sempre que precisa de modificar ou atualizar variáveis auxiliares associadas à execução de um movimento, não necessita de carregar uma estrutura completa com toda a informação, incluindo os arrays de pontos da trajetória, que em certos casos poderia representar um volume considerável de dados. Desta forma, o funcionamento do firmware fica otimizado, evitando sobrecarregar o código e sem comprometer o próprio cálculo das posições em tempo real.

A componente fixa desta divisão é a estrutura **MotionStorage**, que descreve a trajetória em si, através dos arrays de pontos *X*, *Y* e *Z*, do número de pontos, do período total, dos tempos de pausa no início e no fim, do ponto inicial e da indicação de o movimento ser, ou não, bidirecional. Estes dados correspondem à parte estática do movimento: definem o que deve ser executado, mas não dizem em que ponto da execução o sistema se encontra num dado instante.

A componente dinâmica, por sua vez, pertence à estrutura **MotionInstance**, que representa uma instância em funcionamento de uma trajetória guardada em **MotionStorage**. Em vez de duplicar os pontos do movimento, esta estrutura armazena a definição original sob a forma de um ponteiro, uma variável que não guarda os dados em si, mas apenas o endereço de memória onde a estrutura **MotionStorage** correspondente está alojada. Isto permite aceder à mesma trajetória a partir de uma única referência, sem necessidade de chamar as duas estruturas em separado, mantendo ao mesmo tempo a vantagem de as ter fisicamente separadas na memória. A **MotionInstance** guarda apenas o estado de execução: índices atuais, direção de avanço, temporizadores, multiplicador de amplitude em *Z*, flags de ativação e estado atual da máquina de estados. Esta distinção entre definição e execução permite aplicar a mesma rotina genérica à respiração, ao batimento cardíaco e à vibração da tosse, mantendo cada movimento com o seu próprio estado interno. A figura 4.11 resume os campos destas duas estruturas e a ligação por ponteiro entre elas.

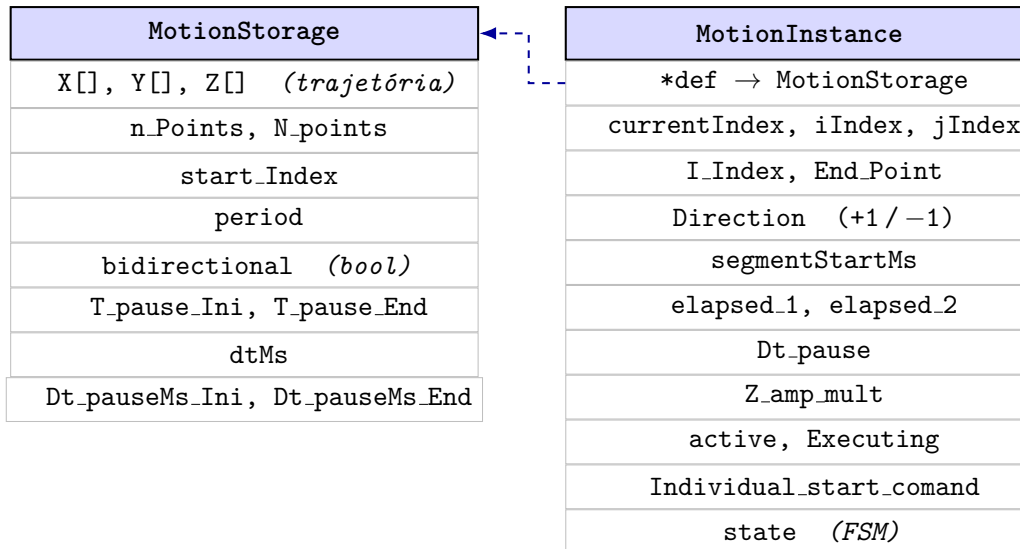


Figura 4.11: Estruturas MotionStorage e MotionInstance.

Na prática, esta separação facilita também a sobreposição de movimentos: a respiração, o batimento cardíaco e a tosse são percorridos por três instâncias distintas, cada uma com a sua própria cadência, o que permite ativar temporariamente a perturbação da tosse sem alterar a definição base das restantes trajetórias. O resultado final de cada instante é obtido mais à frente, na etapa de fusão e cinemática inversa, mas a consistência desse cálculo depende desta estruturação prévia dos dados.

4.6 Máquinas de estados e execução dos movimentos

No ESP32, o firmware não tem como função principal desenhar ou recalculas as curvas geométricas dos movimentos, mas sim gerir a execução, em tempo real, dos pontos de trajetória já definidos e armazenados nas estruturas MotionStorage/MotionInstance apresentadas na secção anterior. É precisamente esta separação entre a definição estática da trajetória e o seu estado dinâmico de execução que permite aplicar a mesma máquina de estados à respiração, ao batimento cardíaco e à vibração da tosse, cada uma com a sua própria instância e cadência de funcionamento.

4.6.1 Desenvolvimento e iterações

Um dos primeiros problemas a resolver foi o de controlar o tempo ao longo de todo o ciclo do movimento, de forma a garantir que este se mantinha consistente, sem atrasos nem adiantamentos. Uma primeira ideia consistiu em associar a cada array de posições um segundo array com a duração até ao ponto seguinte, atualizado por um contador que acompanhava continuamente o tempo decorrido no processador. Esta solução permitia avançar corretamente ao longo da trajetória e terminar o ciclo no fim do array, mas não

resolvia o problema por completo: o movimento resultante ficava marcado por saltos bruscos entre pontos, exatamente o que se pretendia evitar.

Um aspeto ainda não referido, mas decidido desde cedo, era que a geometria destes movimentos exigia zonas de andamento mais rápido e outras mais lento, de modo a transmitir a sensação de abrandamento junto de curvas e de aceleração nos troços mais retos, tornando o movimento tão natural quanto possível. Durante bastante tempo, admitiu-se que isso exigiria calcular um tempo próprio para cada ponto, de modo a preservar o período total pretendido no final do ciclo. Verificou-se, no entanto, que essa complexidade era desnecessária: atribuindo o mesmo intervalo de tempo a todos os pontos e variando, em vez disso, o espaçamento entre eles (mais próximos onde o movimento deveria ser mais lento, mais afastados onde deveria ser mais rápido), obtinha-se exatamente o mesmo efeito percecionado (figura 4.12). Bastava, assim, dividir o período total pelo número de pontos do movimento, partindo do princípio de que os pontos entregues já respeitavam essa distribuição não uniforme desde a sua geração na HMI (secção 4.4). O sistema de atualização de pontos podia então ser tão simples como inicialmente imaginado: um contador e um comparador de tempo, avançando de instante a instante para a coordenada seguinte.

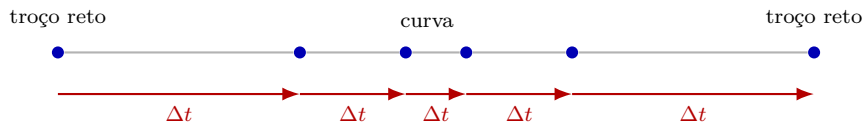


Figura 4.12: Andamento variável obtido apenas por densidade de pontos.

Seguiu-se uma fase de pesquisa de soluções para suavizar esse tipo de transição abrupta entre pontos, tendo sido identificadas duas famílias de abordagens: bibliotecas de suavização de sinal, como a *Ramp* [63] ou a *Easing* [64], que, a partir de um valor atual, de um valor futuro, de uma duração e de um perfil de onda escolhido, geravam uma saída suavizada capaz de atenuar paragens bruscas, e fórmulas matemáticas que subdividiam o movimento em vários micropassos, como um polinómio cúbico capaz de gerar uma trajetória em S entre um ângulo inicial θ_s e um ângulo final θ_e , ao longo de um tempo total T_F , com velocidade nula em ambos os extremos,

$$\theta(t) = \theta_s + a_2 t^2 + a_3 t^3, \quad a_2 = \frac{3(\theta_e - \theta_s)}{T_F^2}, \quad a_3 = \frac{-2(\theta_e - \theta_s)}{T_F^3}.$$

No entanto, ambas revelaram limitações importantes. As bibliotecas de suavização não garantiam que o restante funcionamento do sistema, executado em paralelo, não fosse perturbado, uma vez que várias delas dependiam internamente da função `delay()`, incompatível com o resto do firmware, além de acrescentarem a necessidade de manter entre quatro a cinco arrays por movimento (três para as coordenadas X , Y e Z , um para o tempo entre pontos e um para o tipo de suavização, já que se pretendia variar esse perfil

ao longo do próprio ciclo), numa altura em que se acreditava ainda ser necessário duplicar cada movimento para os dois manipuladores, em vez de o tratar de forma genérica. Já a subdivisão do movimento com gestão da onda de aceleração funcionava bem para um único motor, mas mostrou-se pouco versátil para escalar ao número de motores envolvidos neste projeto (entre seis a nove, em simultâneo). Por estas razões, ambas as abordagens acabaram por ser descartadas.

Com estas limitações identificadas, o sistema a desenvolver teria de cumprir dois requisitos: nunca recorrer à função `delay()` e reduzir ao mínimo o número de arrays necessários por movimento.

Colocou-se então uma nova questão de arquitetura: aplicar essa suavização antes ou depois da fusão dos vários movimentos numa única trajetória por manipulador. Suavizar depois da fusão permitiria fazê-lo apenas uma vez por manipulador, mas suavizar antes trazia uma vantagem decisiva: a fusão passava a ser calculada sempre a partir da posição real e atual de cada movimento, em vez de uma posição já alterada pela suavização, que poderia a qualquer momento distorcer o resultado da fusão e originar trajetórias irregulares. Foi esta segunda opção a escolhida.

Para a suavização em si, adotou-se uma abordagem próxima da subdivisão em micropassos considerada inicialmente, mas sem os calcular todos antecipadamente: aproveitando o próprio contador de tempo já usado na atualização do movimento, calculava-se, a cada ciclo do processador, um ponto intermédio correspondente ao instante decorrido dentro do intervalo Δt esperado entre dois pontos consecutivos. A este processo dá-se o nome de interpolação linear (*lerp*), detalhada mais adiante junto da função que a implementa.

Nos testes realizados a este mecanismo, um movimento de cada vez para simplificar, a estratégia revelou-se eficaz em frequências médias e altas. Em frequências mais baixas, porém, os motores continuavam a apresentar pequenas paragens perceptíveis. Colocou-se a hipótese de a causa estar na transição entre pontos. Em alternativa à interpolação linear entre dois pontos (o atual P_i e o seguinte P_j), testou-se uma interpolação polinomial com três pontos (o atual, o seguinte e um ponto futuro adicional P_k), na expectativa de obter uma aproximação mais suave ao ponto intermédio, sem transições acentuadas entre segmentos (figura 4.13):

$$L(t) = (1 - t) P_i + t P_j, \quad Q(t) = (1 - t)^2 P_i + 2(1 - t)t P_j + t^2 P_k, \quad t \in [0, 1].$$

Por mais que este sistema e a sua variável de aproximação ao ponto intermédio fossem afinados, o resultado mantinha-se semelhante ao da interpolação linear, o que apontava para uma causa distinta. Confirmou-se, com o auxílio da leitura interna do sinal enviado ao servo, que o problema não era de software, mas mecânico: os servos utilizados tinham uma resolução por passo superior a 1°. Em frequências mais baixas,

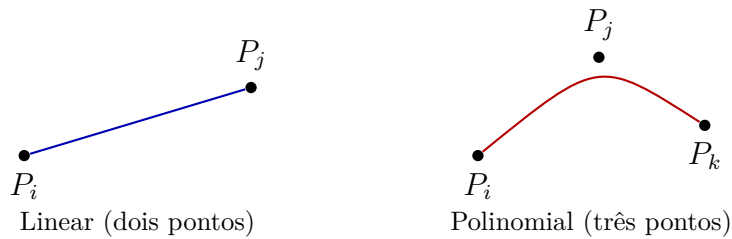


Figura 4.13: Comparação entre a interpolação linear e a interpolação polinomial testadas.

isso exigia incrementos demasiado pequenos para que o servo os conseguisse traduzir num movimento real, resultando em saltos mais perceptíveis do que em frequências mais altas. Não sendo viável substituir os motores nesta fase, optou-se por manter a solução de software e procurar mais tarde soluções mecânicas complementares, como amortecedores ou uma caixa redutora acoplada ao servo.

Identificada a causa dos saltos, voltou-se à interpolação linear, que apresentava resultados semelhantes aos da interpolação polinomial com menor complexidade e menos linhas de código. Esta foi, por isso, a interpolação efetivamente usada no código final, com o seu coeficiente saturado entre 0 e 1, tal como detalhado no módulo final desta secção. Esta saturação trouxe ainda uma vantagem adicional: caso a frequência de um movimento fosse extremamente alta e o intervalo Δt demasiado curto, o valor de saída nunca é comprometido por um passo desatualizado, atravessando o processo praticamente sem efeito.

Chegou-se assim à sequência final de processamento adotada (figura 4.14):

Atualização do movimento → Suavização por interpolação linear → Fusão dos movimentos

Figura 4.14: Sequência final de processamento, repetida a cada ciclo e para cada manipulador.

Com este mecanismo testado e validado, a execução ainda apresentava uma limitação relevante: só aceitava trajetórias organizadas como ciclos fechados, sob pena de introduzir um salto acentuado entre secções mal ligadas. Era este o caso da respiração, testada até então com uma duplicação das suas coordenadas em sentido inverso, de modo a simular artificialmente um ciclo fechado.

A geometria da parábola escolhida para a respiração tornava difícil fechar o ciclo com limites suaves. Manter coordenadas duplicadas em sentidos opostos era, por si, pouco conveniente, sobretudo tendo em vista a futura integração da tosse, um evento de ocorrência imprevisível. Optou-se por isso por deixar este movimento aberto, o que exigiu acrescentar à execução a capacidade de percorrer certas trajetórias nos dois sentidos, para a frente e para trás. Para as trajetórias marcadas como bidirecionais, o ciclo passou a percorrer-se num sentido até ao último ponto e, de seguida, no sentido inverso até ao ponto inicial, com paragens configuráveis no fim de cada percurso quando aplicável. A

gestão dos índices i e j junto destes extremos e das pausas é detalhada mais adiante nesta secção, a propósito da figura 4.15.

Uma última alteração a este sistema surgiu já na implementação da tosse, motivada pela necessidade de aumentar temporariamente a amplitude do movimento no momento da contração brônquica. Como todas as coordenadas geradas na HMI partem do ponto $(0,0)$ do respetivo plano, associado à posição de repouso do manipulador, multiplicar essa amplitude fazia crescer o ciclo apenas no sentido positivo do eixo Z e não de forma simétrica em torno do centro do movimento, como seria fisiologicamente esperado.

A solução passou por iniciar o ciclo num ponto intermédio da curva, em vez de num dos seus extremos, dividindo assim as coordenadas em duas partes, uma positiva acima desse ponto e outra negativa abaixo dele, com o próprio ponto de partida a assumir o papel do novo $(0,0)$ da curva. Para preservar a simplicidade do cálculo de pontos na HMI, esta alteração não foi introduzida na origem dos dados, mas sim diretamente no firmware: sempre que o índice inicial `start_Index` é diferente de zero, a coordenada desse ponto é subtraída, em tempo real, à coordenada do ponto atual, permitindo escolher livremente qualquer ponto como início do ciclo sem alterar os dados armazenados. A única alteração estrutural necessária foi passar de duas para três sequências na atualização do movimento: do ponto inicial até ao último ponto, deste até ao ponto 0 e, por fim, do ponto 0 de volta ao ponto inicial. Esta forma de indexação trouxe ainda uma vantagem adicional: caso a tosse fosse acionada durante o avanço do ciclo, o sentido de atualização podia ser invertido de imediato, recuperando o sistema de forma natural a partir dessa nova direção.

4.6.2 Módulo final

Nesta etapa final, todo o processo de gestão e execução do movimento consiste em percorrer os arrays de pontos já calculados e armazenados, avançar os índices ao ritmo certo e pela sequência correta, gerir eventuais pausas, traduzir esses índices em coordenadas reais e, por fim, unir os três movimentos numa só trajetória contínua e realista para cada manipulador. Esta sequência é executada por três mecanismos distintos: a `FSM_Motion_Update`, que gere os índices e a sequência dos ciclos, a função `Intermed_position()`, que converte esses índices em coordenadas e calcula o ponto intermédio correspondente ao tempo decorrido e, por fim, a função `Fusao_plus_IK.D1/Fusao_plus_IK.D2`, que une os três movimentos numa única saída para a cinemática inversa.

Primeira etapa: intervalo de tempo por segmento. O ritmo a que os índices avançam depende do intervalo Δt atribuído a cada segmento, calculado a partir do período total do movimento T e do número de pontos N , considerando também a ida e o regresso

nos movimentos bidirecionais:

$$N_{\text{seg}} = \begin{cases} 2(N - 1), & \text{movimento bidirecional,} \\ N - 1, & \text{movimento não bidirecional,} \end{cases} \quad \Delta t = \frac{T}{N_{\text{seg}}}. \quad (4.11)$$

Segunda etapa: avanço dos índices. Os índices i e j (que representam, respectivamente, o ponto de partida e o ponto de chegada do segmento atual da trajetória) são avançados ao ritmo definido por Δt pela função genérica `FSM_Motion_Update`, capaz de percorrer qualquer trajetória associada a uma `MotionInstance`, independentemente do movimento em causa. A figura 4.15 representa a sua sequência lógica, incluindo os estados principais, as pausas inicial e final, a inversão de direção e o tratamento do movimento bidirecional.

A `FSM_Motion_Update()` recebe, a cada ciclo de execução, o sinal global de arranque (`start`), a referência da instância de movimento (`inst` $\rightarrow$ `MotionInstance`) e o instante de tempo atual (`nowMs`). Com estes dados, mantém atualizados os índices de trajetória e o cronómetro interno, sem calcular a posição em si, responsabilidade que pertence à terceira etapa.

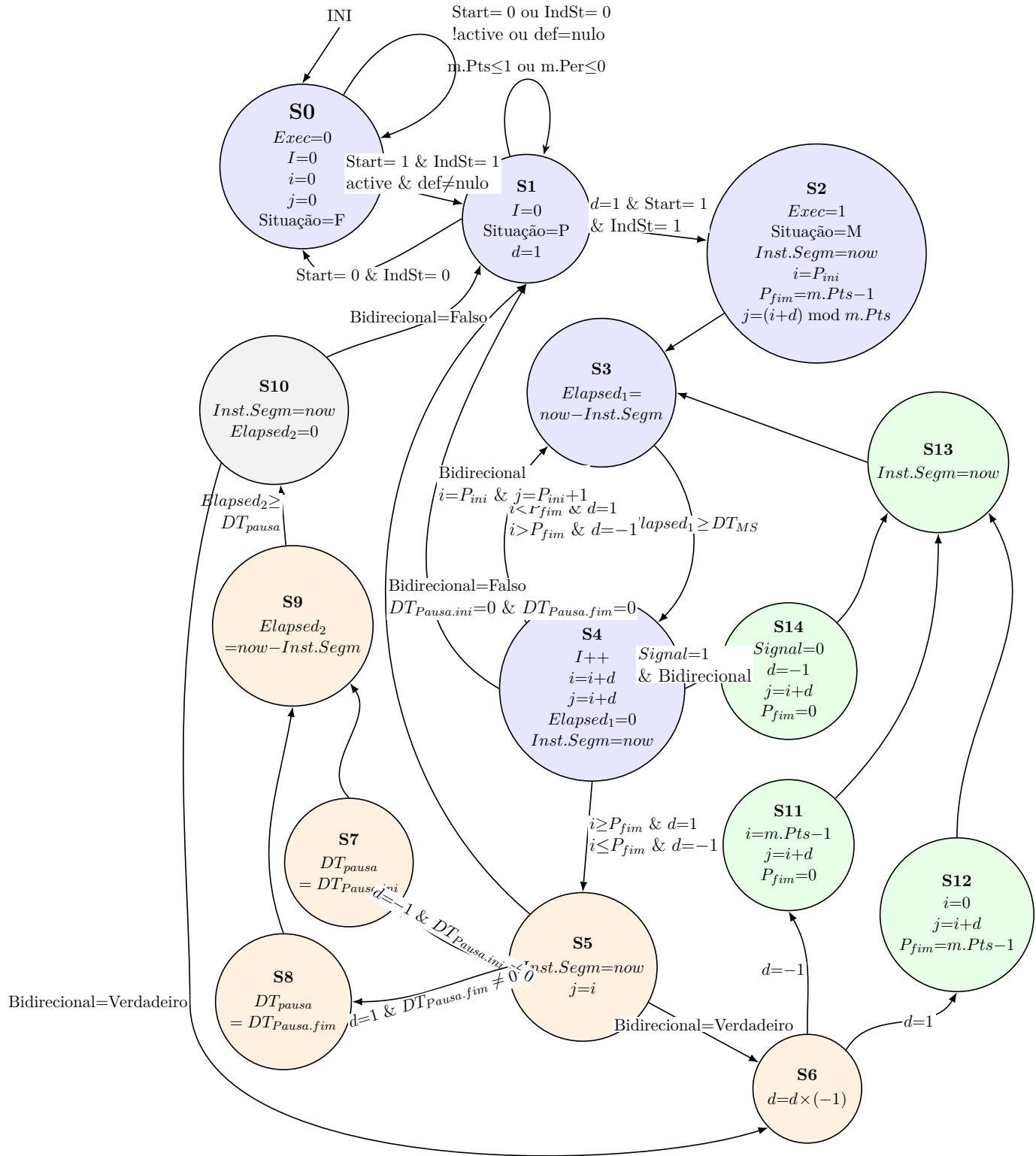


Figura 4.15: Esquema da *FSM_Motion_Update*.

Fases de funcionamento da FSM_motion_update.

- **Arranque (S0→S1→S2):** O processo de arranque serve para preparar a FSM para a realização dos ciclos. Por padrão inicia sempre no estado S0, onde é verificado se existe algum comando de iniciação, caso exista faz várias verificações, se os arrays não são nulos e se o período é diferente de 0. Se todas as verificações passarem sem qualquer erro encontrado é então atribuído os parâmetros iniciais para cada movimento, como ponto de arranque, sentido e tempo do cronómetro.
- **Ciclo de segmentos (S3↔S4):** Estes dois estados têm como função percorrer todo o ciclo do movimento (através dos índices i e j , explicados anteriormente), em ambas as direções (caso seja necessário), comparando constantemente o tempo decorrido até ao momento com o intervalo de tempo pretendido para cada segmento.
- **Gestão de pausas e direções (S5→S7/S8→S9→S10):** Ao atingir o fim de cada ciclo, o estado S5 determina o percurso seguinte a ser tomado, sendo determinado pelas configurações internas do movimento (se tem pausas implementadas, se é um movimento bidirecional, ou não, etc.). S7 e S8 são só caminhos encarregues pela gestão de pausas, e o S6 pela gestão de mudança de direção, caso o movimento permita, sendo feita reinicialização dos índices para acomodar as novas direções.
- **Interrupção de tosse (S4→S14→S13):** Caso o sinal de tosse seja ativado durante o decorrer do ciclo, a FSM transita para o estado S14, o que força a mudança do sentido e encadeia de imediato com o estado S13 para iniciar a descida de emergência.

Terceira etapa: cálculo da posição instantânea. Ao terminar cada chamada da FSM, envia os índices (atual e o índice do ponto de chegada), bem como o tempo decorrido desde o início do segmento. É com este conjunto de valores que a função `Intermed_position()` associa ao movimento as coordenadas reais correspondentes e calcula a posição instantânea, em função do tempo decorrido.

A posição instantânea é obtida por interpolação linear entre dois pontos consecutivos da trajetória:

$$P(\alpha) = P_i + (P_j - P_i) \alpha$$

P_i : ponto de partida do segmento

P_j : ponto de chegada do segmento

t_{dec} : tempo decorrido no segmento (`elapsed_1`)

DT_{MS} : duração total do segmento (`dtMs`)

o coeficiente de interpolação saturado é:

$$\alpha = \begin{cases} 0 & \text{se } \alpha_0 < 0 \\ \alpha_0 & \text{se } 0 \leq \alpha_0 \leq 1 \\ 1 & \text{se } \alpha_0 > 1 \end{cases} \quad \alpha_0 = \frac{t_{dec}}{DT_{MS}}$$

Quarta etapa: fusão das coordenadas. A função `Fusao_plus_IK.D1/Fusao_plus_IK.D2` combina as três coordenadas instantâneas (respiração, batimento e tosse) numa única coordenada por manipulador, somando a cada eixo um peso próprio por movimento e um deslocamento fixo específico do manipulador, de forma a posicionar corretamente a plataforma móvel dentro do espaço de trabalho de cada um. O resultado desta soma é entregue diretamente à função de cinemática inversa correspondente (secção 4.8), fechando a cadeia de execução tratada nesta secção.

4.7 Preparação e sincronização da tosse

A tosse é o único dos três movimentos representados que não corresponde a uma trajetória fisiológica contínua e previsível, mas sim a um evento pontual e imprevisível, sobreposto ao ciclo respiratório em curso. Esta secção descreve o conjunto de funções responsável por decidir quando a tosse deve ocorrer, validar se o momento é seguro e sincronizar esse evento com o movimento de respiração já em execução.

Esta funcionalidade foi implementada numa fase avançada do desenvolvimento, depois de os movimentos de respiração e batimento cardíaco já estarem validados de forma independente. A primeira versão, entretanto substituída e mantida apenas como referência no código, resolvia toda a preparação da tosse numa única função executada de forma sequencial a cada ciclo. Essa abordagem foi reescrita como uma máquina de estados própria, a `FSM_preparacao_tosse`, tornando explícitas as diferentes fases do processo e facilitando a deteção de condições de paragem do sistema a meio de uma tosse.

O comportamento da tosse resulta da interação de três funções distintas: a `AtualizarTosseAleatoria()`, que decide de forma aleatória quando deve ocorrer o próximo evento de tosse, a `PosicaoValidaParaTosse()`, que valida se o ponto da trajetória respiratória em que o sistema se encontra permite iniciar uma tosse sem comprometer o movimento e ainda a `FSM_preparacao_tosse()`, que aplica e repõe as alterações temporárias à respiração necessárias para sobrepor o evento.

Geração aleatória do evento. A função `AtualizarTosseAleatoria()` corre a cada ciclo de execução e é responsável por decidir quando deve ocorrer a próxima tosse, representada na figura 4.16. Enquanto o sistema, a respiração e o modo de tosse estiverem ativos e não existir já uma tosse em curso ou em preparação, a função gera

um tempo de espera até ao próximo evento (`proximaTosseMs`), inicialmente definido de forma aleatória entre 4 e 10 segundos. Ao atingir esse instante, a função invoca a `PosicaoValidaParaTosse()` para confirmar que o ponto atual da trajetória respiratória, definido pelo índice `iIndex` e pelo sentido `Direction`, não está demasiado próximo do início nem demasiado próximo do topo do movimento, evitando que a tosse se sobreponha a uma transição brusca da respiração. Caso a posição não seja válida, o evento é adiado 500 milissegundos e reavaliado no ciclo seguinte.

Quando as condições são cumpridas, a função ativa a flag `senal_tossir` e gera ainda repetições do evento de tosse, agrupando entre uma a quatro tosses consecutivas por cada episódio (`tossesRestantesNoCluster`), com intervalos curtos, de 0,5 a 1,5 segundos, entre tosses do mesmo grupo e com um intervalo maior, de 3 a 10 segundos, entre grupos distintos. Este padrão aproxima o comportamento simulado de séries de tosse observadas clinicamente, em vez de eventos isolados e uniformemente espaçados.

Máquina de estados de preparação. Reconhecido o sinal de tosse, a `FSM_preparacao_tosse()` assume a responsabilidade de sincronizar o evento com a respiração, representada na figura 4.17. No estado de repouso, a máquina aguarda que a respiração esteja ativa e que a flag `sinal_tossir` seja ativada. Ao ser acionada, guarda os parâmetros correntes de período e pausa inicial da respiração, reduz o período para 0,3 segundos, elimina a pausa inicial e aumenta em 50% a amplitude do eixo *Z* da respiração através do multiplicador `Z_amp_mult`, introduzido na fusão das coordenadas (secção 4.6.2). Ativa também a flag `Tosse_signal`, interpretada pela `FSM_Motion_Update` da respiração para forçar a inversão antecipada do sentido do movimento (figura 4.15) e a instância de vibração da tosse, através de `Individual_start_comand`.

A máquina permanece no estado ativo até a respiração atingir o estado que assinala o fim da secção afetada ou até o sistema ser desligado, altura em que são repostas as configurações iniciais dos parâmetros de período, pausas e multiplicador de amplitude, são limpas todas as flags e ocorre o regresso ao estado de espera ou ao estado inicial, consoante o sistema continue ou não em funcionamento.

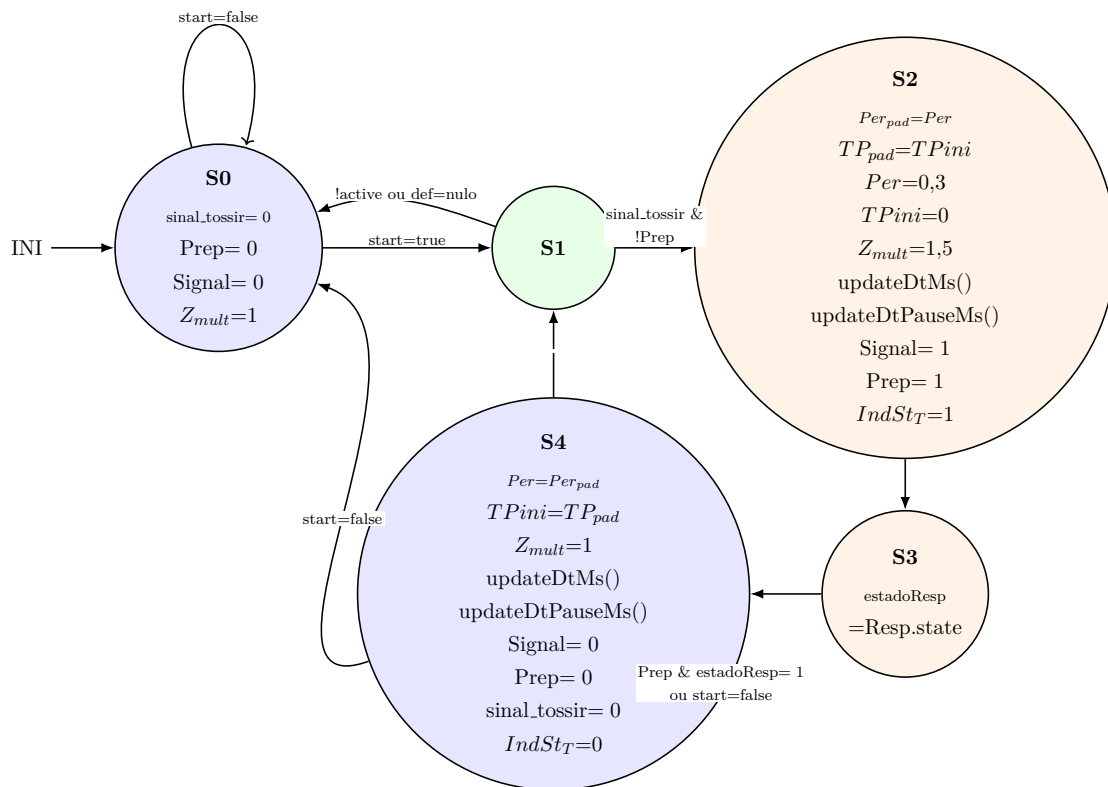


Figura 4.17: Esquema da `FSM_preparacao_tosse`.

4.8 Cinemática inversa do manipulador delta

A cinemática inversa é o algoritmo que converte a posição desejada da plataforma móvel (*X, Y, Z*) num conjunto de ângulos, correspondendo cada um deles ao movimento que o

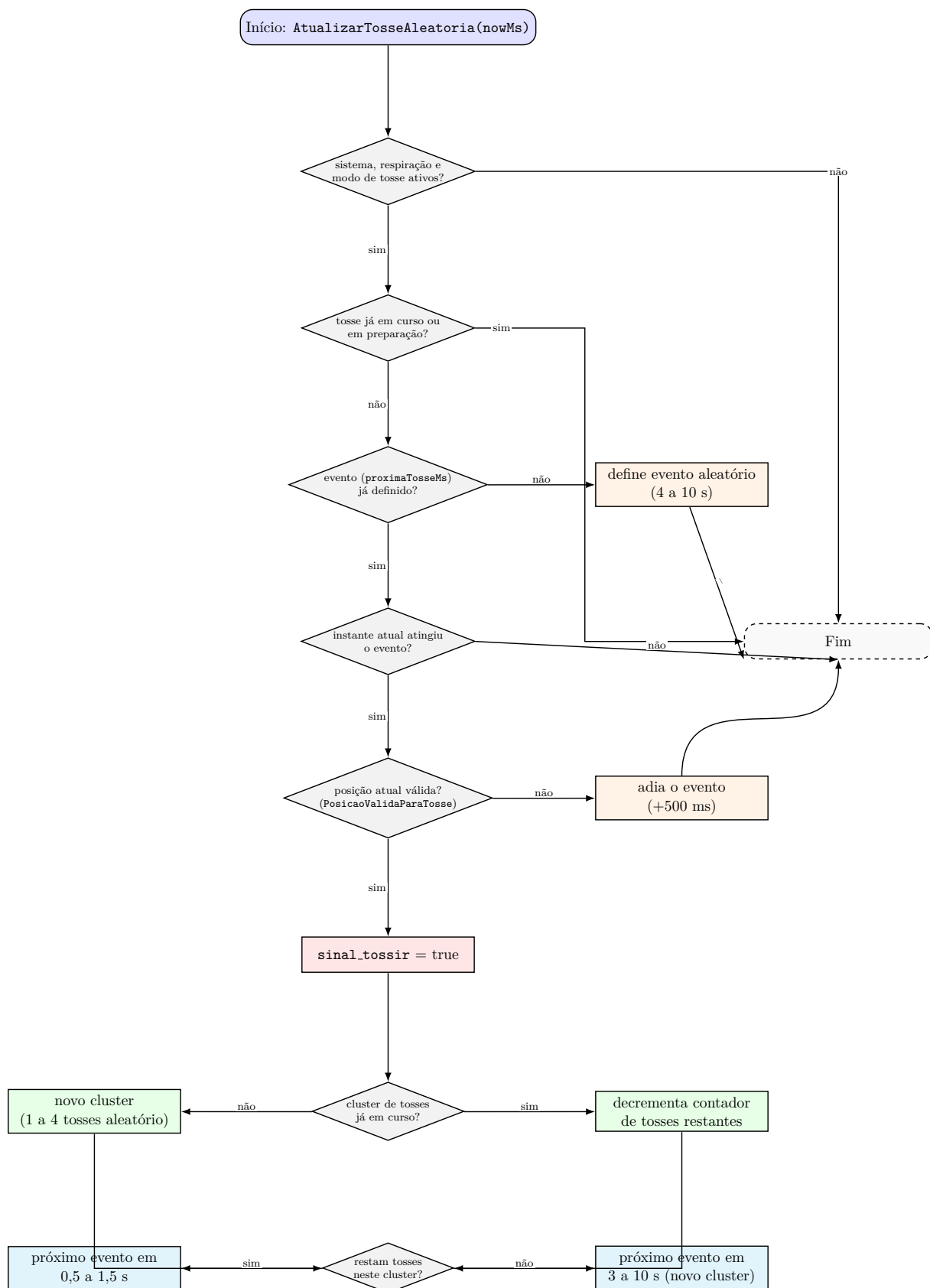


Figura 4.16: Fluxograma da função AtualizarTosseAleatoria().

respetivo atuador do manipulador tem de executar para que essa posição seja atingida. Os cálculos associados a esta conversão, num manipulador delta, são realizados inicialmente para um único braço, sendo depois repetidos de forma independente para os restantes, resultando nos três ângulos distintos θ_1 , θ_2 e θ_3 . Esta secção descreve, num primeiro momento, o percurso seguido até à abordagem final adotada e, de seguida, o algoritmo efetivamente implementado no firmware.

4.8.1 Desenvolvimento e iterações

O desenvolvimento deste cálculo começou por uma fase de pesquisa de trabalhos e repositórios já existentes sobre cinemática inversa para manipuladores delta. Embora existam bastantes artigos ou documentos académicos sobre o tema, a maioria dessas soluções recorria a matrizes de transformação relativamente complexas, cuja implementação direta em C++, num microcontrolador como o ESP32, revelava-se pouco prática e bastante complexa. Os repositórios que acabaram por inspirar o modelo final tinham em comum uma abordagem bem mais simples, assente em trigonometria simples e direta, com uma única matriz de rotação utilizada em todo o cálculo da cinemática inversa para atribuir as coordenadas e os ângulos de cada braço, o que tornava a sua implementação consideravelmente mais acessível.

A maioria destes trabalhos apresentava, no entanto, um problema comum: descreviam a cinemática de um manipulador delta na configuração mais habitual, a de um sistema *pick-and-place*, montado com a plataforma voltada para baixo. Este projeto exigia precisamente o oposto, com a plataforma voltada para cima, de modo a poder ser acoplada ao modelo da árvore brônquica. A diferença pode parecer irrelevante, mas tem uma consequência prática: na orientação habitual, o antebraço do manipulador raramente ultrapassa o nível da base, pelo que o ângulo do atuador costuma ser descrito a partir do prolongamento de uma linha imaginária que liga o exterior do manipulador ao centro da base, cobrindo apenas 180° de amplitude no sentido positivo, com o restante percurso possível representado no sentido negativo (figura 4.18). Era o caso, por exemplo, da *Delta-Kinematics-Library* [65] e da publicação de Nikanorov [66]. Para este projeto, que exigia uma resolução angular mais fina do atuador de modo a permitir compactar ao máximo o sistema, esta convenção obrigaria a separar ângulos positivos e negativos em torno desse horizonte de referência, o que por sua vez exigiria recorrer a transformações adicionais e introduziria uma complexidade evitável.

Uma solução mais adequada surgiu com o repositório de código aberto *Delta-Robot*, de isaac879 [60], que reformulava por completo o cálculo do ângulo do atuador: em vez de o referenciar ao horizonte exterior do manipulador, media-o a partir do centro da própria base, descrevendo uma rotação contínua de até 360° em torno desse centro, sem qualquer separação entre valores positivos e negativos. O repositório incluía ainda verificações de

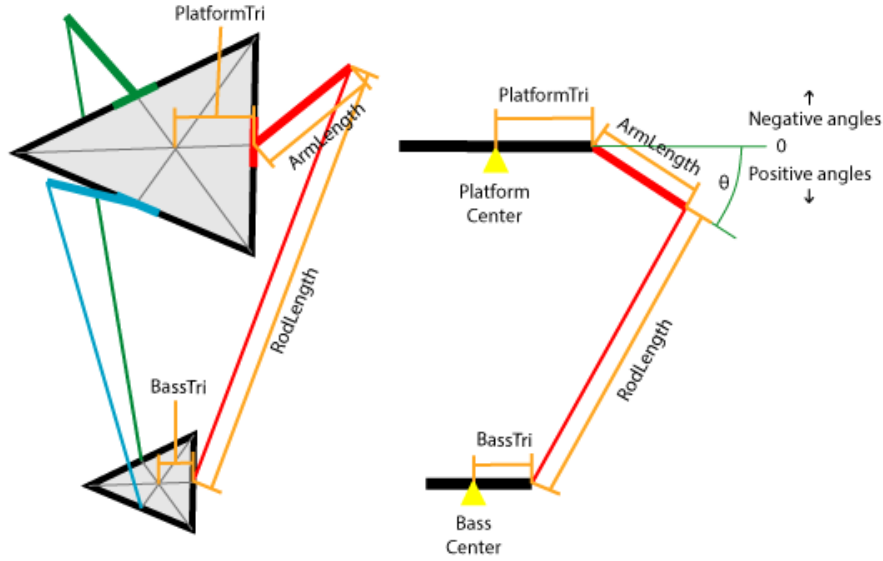


Figura 4.18: Convenção de ângulo referenciada ao horizonte exterior do manipulador, usada na *Delta-Kinematics-Library* [65].

segurança que impediam o manipulador de atingir posições perigosas ou geometricamente impossíveis, assim como parâmetros de ajuste fino para a base, a plataforma, as uniões e os próprios atuadores, permitindo obter um resultado mais preciso do que os métodos anteriores. Continua, por fim, um método de conversão do ângulo calculado para o comando de um motor de posição genérico, bastando calibrar os seus limites em função do motor utilizado, assumindo um intervalo de rotação de 180° , comum em servos, medido entre 45° e 225° a partir do centro da base, uma resolução de movimento mais próxima da habitualmente esperada para este tipo de manipulador.

Este repositório não pôde, no entanto, ser incorporado diretamente no firmware, tendo sido necessário adaptá-lo às necessidades do projeto. Foram removidos dois mecanismos, que não se enquadravam na filosofia do projeto: o sistema de coordenadas que subdividia internamente o movimento em vários micropassos, redundante face ao mecanismo de suavização por interpolação linear entretanto adotado (secção 4.6.1) e o sistema que permitia reorientar livremente o manipulador, desnecessário numa montagem de orientação fixa como a deste projeto. Em contrapartida, foram acrescentados a rotação global de cada manipulador, combinada com a rotação individual de cada braço já existente, bem como um ajuste à conversão final do ângulo, de modo a produzir diretamente o sinal PWM aceite pelos servos MG90S. A função de cinemática inversa foi ainda generalizada para aceitar qualquer configuração de manipulador, evitando duplicar código entre os dois manipuladores do sistema, seguindo a mesma lógica de generalização já aplicada à máquina de estados de execução dos movimentos.

Do lado dos motores, foi ainda necessário calibrar os limites do período de PWM, de modo a fazer corresponder o intervalo de 45° a 225° da cinemática inversa à amplitude

de rotação física de cada servo, um procedimento detalhado no módulo final desta secção.

4.8.2 Módulo final

O algoritmo de cinemática inversa aqui descrito é executado, em cada ciclo do firmware, de forma independente para os três braços de cada manipulador, devolvendo o conjunto de ângulos para os atuadores, referente à posição pretendida da plataforma móvel. A figura 4.19 identifica a terminologia usada ao longo desta secção para os componentes do manipulador delta: a plataforma móvel, o braço acionado diretamente pelo servo e o antebraço passivo que liga o braço à plataforma, todos assentes sobre a base fixa.

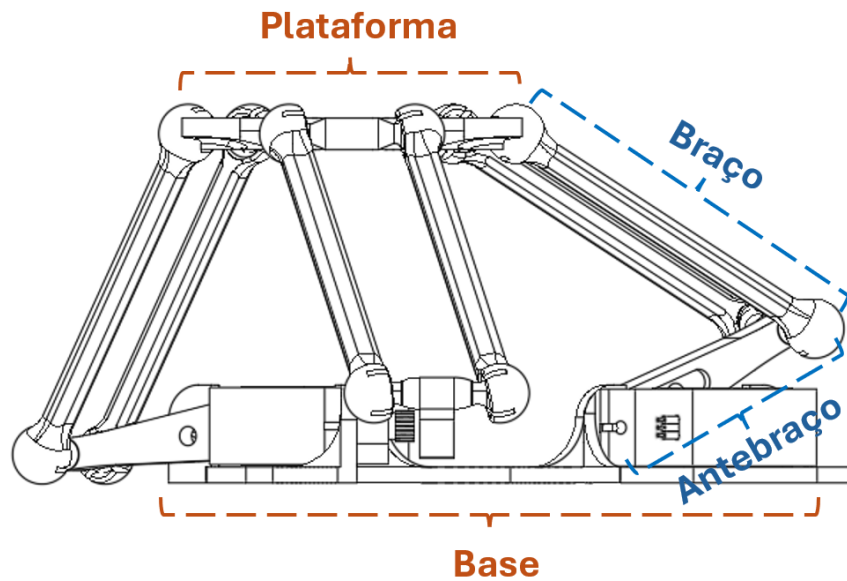


Figura 4.19: Terminologia usada para os componentes do manipulador delta: base, braço, antebraço e plataforma móvel.

A figura 4.20 mostra a representação lateral usada como referência para as variáveis do cálculo: L_1 representa o ante-braço acionado pelo servo, L_2 o braço passivo, L_3 a distância horizontal até à junta esférica da plataforma móvel (`Arm_offset_X` no firmware), `Servo_offset_X` e `Servo_offset_Z` são os deslocamentos fixos entre o eixo do servo e a origem do referencial lateral, ϕ e ω são os ângulos intermédios calculados para cada braço, θ é o ângulo final aplicado ao servo e a *Extensão* corresponde à distância `ext` calculada entre o eixo do servo e a projeção da junta esférica.

O firmware aplica o mesmo cálculo aos três braços, distinguindo-os apenas pela rotação do referencial no plano XY . Para cada manipulador existe um offset global α_m (definido na estrutura `Manipulador_Config`) e para cada servo existe uma rotação

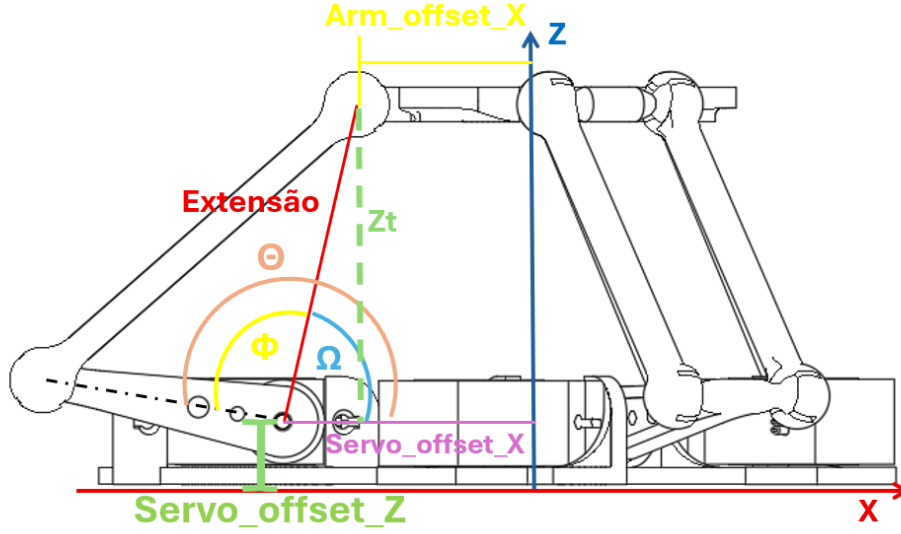


Figura 4.20: Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.

adicional β_k :

$$\alpha_m = \begin{cases} -15^\circ, & \text{Manipulador Delta 1} \\ -45^\circ, & \text{Manipulador Delta 2} \end{cases} \quad \beta_k \in \{0^\circ, 120^\circ, 240^\circ\}$$

A figura 4.21 ilustra esta distribuição, com os três servos de cada manipulador dispostos a 120° uns dos outros em torno do centro da base.

$$\psi_k = \alpha_m + \beta_k \quad R_z(\psi_k) = \begin{bmatrix} \cos \psi_k & -\sin \psi_k & 0 \\ \sin \psi_k & \cos \psi_k & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Assim, o ponto pedido é primeiro expresso no plano de trabalho do servo k :

$$\begin{bmatrix} x'_k \\ y'_k \\ z'_k \end{bmatrix} = R_z(\psi_k) \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ z_{offset} \end{bmatrix} \implies \begin{cases} x'_k = X \cos \psi_k - Y \sin \psi_k \\ y'_k = X \sin \psi_k + Y \cos \psi_k \\ z'_k = Z - z_{offset} \end{cases}$$

(No firmware, o z_{offset} , definido pela constante `SERVO_OFFSET_Z`, representa um ajuste fino à distância entre a base real do manipulador e a base onde os atuadores estão acoplados. Por omissão, este ajuste mantém-se a 0 mm, uma vez que afeta apenas a forma como as coordenadas são referenciadas, podendo estas ser expressas diretamente a partir da base do atuador ou da base real do modelo físico.)

Após esta rotação, o problema passa a ser resolvido inteiramente no plano lateral

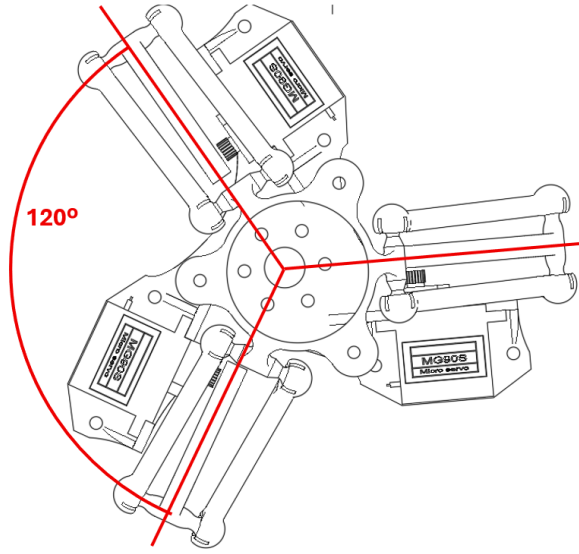


Figura 4.21: Distribuição a 120° dos três servos em torno do centro da base do manipulador delta.

do servo, correspondente à representação bidimensional da figura 4.20. Nesse plano, a extremidade associada à plataforma móvel é deslocada pelo comprimento L_3 :

$$x_{arm,k} = x'_k + L_3$$

O braço passivo L_2 nem sempre se mantém contido no plano lateral do servo, uma vez que a plataforma móvel se pode deslocar lateralmente (eixo Y). O firmware compensa este desvio calculando o comprimento efetivo de L_2 projetado nesse plano, sendo esta projeção apenas válida se a junta esférica da plataforma móvel não ultrapassar o limite angular admitido:

$$L_{2p,k} = \sqrt{L_2^2 - y_k'^2} \quad \psi_{L2p,k} = \arcsin\left(\frac{y_k'}{L_2}\right) \quad |\psi_{L2p,k}| < 0,5934 \text{ rad} \approx 34^\circ$$

Este limite aparece no firmware como proteção para evitar que o ângulo entre a junta esférica e o braço passivo saia da zona mecânica admissível. A figura 4.22 ilustra esta projeção: y_T corresponde ao deslocamento lateral y'_k da plataforma móvel, L_{2p} ao comprimento projetado no plano lateral e ψ_{L2p} ao ângulo $\psi_{L2p,k}$ entre o braço passivo L_2 e a sua projeção.

De seguida calcula-se a distância entre o eixo do servo e a projeção da junta esférica no plano lateral, correspondente à *Extensão* identificada na figura 4.20. Esta distância é designada no código por `ext`:

$$d_k = \sqrt{z_k'^2 + (d_s - x_{arm,k})^2} \quad d_s = 31,797 \text{ mm}$$

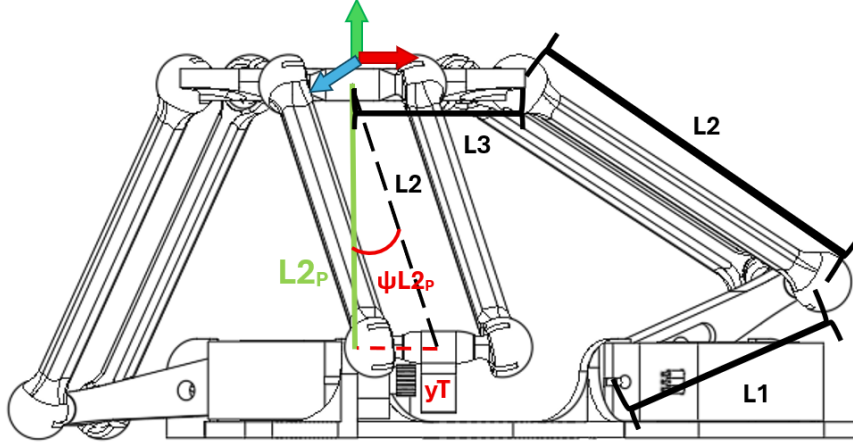


Figura 4.22: Projeção do braço passivo L_2 no plano lateral do servo, resultante do deslocamento lateral da plataforma móvel.

onde d_s , designado no código por `SERVO_OFFSET_X`, é a distância horizontal fixa, definida no desenho mecânico, entre o eixo do servo e a origem do referencial lateral.

Antes de calcular o ângulo, o firmware confirma se os comprimentos L_1 , $L_{2p,k}$ e d_k conseguem formar um triângulo:

$$L_{2p,k} - L_1 < d_k < L_1 + L_{2p,k}$$

Se esta condição falhar, a posição pretendida fica fora do alcance geométrico daquele braço e a função de cinemática inversa devolve erro.

Com a geometria validada, o ângulo interno Φ_k é obtido pela lei dos cossenos, enquanto Ω_k corresponde à inclinação da reta entre o eixo do servo e a projeção da junta esférica:

$$\Phi_k = \arccos\left(\frac{L_1^2 + d_k^2 - L_{2p,k}^2}{2L_1d_k}\right) \quad \Omega_k = \text{atan2}(z'_k, d_s - x_{arm,k})$$

O ângulo enviado ao servo é então:

$$\Theta_k = \Phi_k + \Omega_k \quad 45^\circ \leq \Theta_k \leq 225^\circ$$

O intervalo de 45° a 225° é o limite angular definido no firmware por `SERVO_ANGLE_MIN` e `SERVO_ANGLE_MAX`, pelo que, se algum dos três ângulos calculados ficar fora deste intervalo, a posição é rejeitada e os servos não são atualizados.

Por fim, cada Θ_k é convertido para largura de pulso PWM através do mesmo princípio de interpolação linear já utilizado para calcular a posição instantânea

(secção 4.6.2), aplicado agora entre os limites de pulso mínimo e máximo do servo:

$$PWM_k = PWM_{k,min} + \frac{\Theta_k - \Theta_{min}}{\Theta_{max} - \Theta_{min}} (PWM_{k,max} - PWM_{k,min})$$

Considerando os valores de referência $PWM_{min} = 500 \mu s$ e $PWM_{max} = 2500 \mu s$, a relação é crescente no Manipulador Delta 1 e invertida no Manipulador Delta 2. Esta inversão decorre da montagem em espelho adotada para os dois manipuladores, que permite compactar o conjunto mas deixa o segundo manipulador fisicamente invertido em relação ao primeiro, obrigando a inverter também o sentido da correspondência entre ângulo e sinal PWM para obter o mesmo resultado final de movimento. Esta inversão é feita no arranque do firmware, quando os limites `thetaMinPulse` e `thetaMaxPulse` do segundo manipulador são trocados.

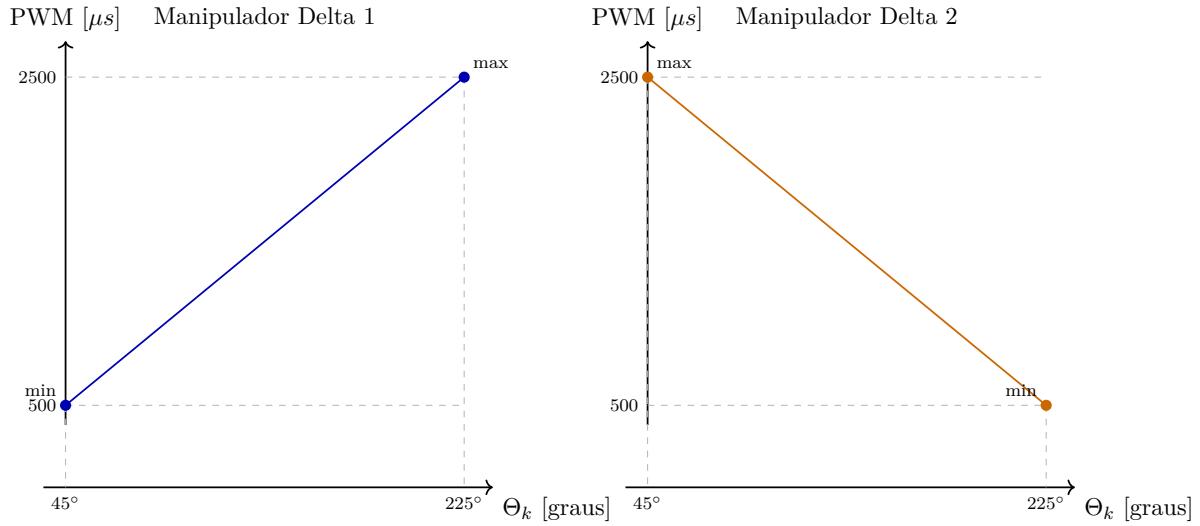


Figura 4.23: Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.

Os limites de PWM usados no código não foram assumidos apenas a partir de valores teóricos do servo. Estes valores foram calibrados, começando pelo alinhamento da patilha de cada servo com a base onde estava acoplado. O objetivo deste alinhamento consistia em garantir que as posições físicas de 0° e 180° permanecessem paralelas em relação à superfície da base. Depois desse alinhamento mecânico, era feita a transição manual da referência física para o intervalo usado pela cinemática, mantendo-se o mesmo intervalo do período de PWM já calibrado. No Manipulador Delta 1, o 0° físico passava a corresponder ao 45° cinemático e o 180° físico ao 225° cinemático. No Manipulador Delta 2, devido à orientação oposta dos servos, o 180° físico passava a corresponder ao 45° cinemático e o 0° físico ao 225° cinemático. Por fim, foi necessário repetir ensaios de calibração para confirmar que os impulsos PWM correspondiam aos limites de 45° e 225°, admitindo uma tolerância aproximada de $\pm 5^\circ$.

$$\text{Manipulador Delta 1 (calibrado):} \left\{ \begin{array}{rcl} & \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 520 \mu s & 2520 \mu s \\ \text{Servo } \theta_2 : & 550 \mu s & 2550 \mu s \\ \text{Servo } \theta_3 : & 560 \mu s & 2560 \mu s \end{array} \right.$$

$$\text{Manipulador Delta 2 (calibrado):} \left\{ \begin{array}{rcl} & \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 460 \mu s & 2460 \mu s \\ \text{Servo } \theta_2 : & 470 \mu s & 2470 \mu s \\ \text{Servo } \theta_3 : & 420 \mu s & 2420 \mu s \end{array} \right.$$

Estes são os intervalos físicos efetivamente usados pelo firmware para associar cada servo ao seu canal PWM, já com a troca entre `thetaMinPulse` e `thetaMaxPulse` do Manipulador Delta 2 refletida nos valores calibrados acima. Note-se que, apesar de os valores absolutos diferirem ligeiramente entre servos, a largura bruta do intervalo mantém-se sempre em $2000 \mu s$, a mesma definida entre os valores de referência $PWM_{min} = 500 \mu s$ e $PWM_{max} = 2500 \mu s$ do datasheet, estando cada intervalo apenas deslocado em relação a essa referência. Isto garante que todos os servos percorrem o mesmo espaço angular, independentemente de os seus limites individuais estarem ligeiramente adiantados ou atrasados face ao valor teórico. Assim, a cinemática inversa gera sempre os mesmos três ângulos lógicos, independentemente do manipulador, cabendo apenas à conversão final para PWM refletir a montagem, o sentido de rotação e a calibração individual de cada servo.

4.9 Comunicação série

A comunicação entre o Raspberry Pi e o ESP32 assenta numa ligação série simples. O tratamento dessa ligação evoluiu, no entanto, de forma distinta em cada lado. No firmware, a comunicação é organizada por duas máquinas de estados dedicadas à receção e interpretação dos comandos, descritas na secção seguinte. No Raspberry Pi, assenta antes numa arquitetura de threads e filas que impede a interface de ficar bloqueada à espera da porta série, descrita mais adiante neste capítulo.

4.9.1 Comunicação série no firmware (ESP32)

A comunicação série, no ESP32, encontra-se organizada em duas máquinas de estados distintas, cada uma responsável por uma função bem definida: a `FSM_Serial_reader()`, que trata apenas da receção dos dados entregues pela porta série e a `FSM_Serial_Command()`,

que interpreta esses dados de forma ordenada e altera o comportamento do sistema, bem como as suas configurações e dados internos. Esta separação evita que o resto do sistema fique sobrecarregado com a execução e o tratamento da informação inerentes à lógica de controlo dos movimentos.

4.9.1.1 Desenvolvimento e iterações

A primeira versão desta separação em duas máquinas de estados ficou operacional numa fase intermédia do projeto, com a `FSM_Serial_reader()` a manter-se, desde então, praticamente inalterada, sujeita apenas a pequenas correções pontuais ao longo do desenvolvimento. Foi sobretudo a `FSM_Serial_Command()` que sofreu sucessivas reformulações, à medida que o conjunto de comandos acompanhava o crescimento da interface gráfica do Raspberry Pi.

A primeira versão interpretava apenas caracteres isolados, associando cada um a uma ação fixa. Seguiu-se a introdução de um sistema de menus assente na mesma lógica de caracteres isolados, em que uma única letra passava a dar acesso a um conjunto de opções adicionais, também elas identificadas por um único carácter. A alteração mais significativa foi, no entanto, a introdução do modo de comandos por cadeia de caracteres, destinado a modificar diretamente valores inteiros ou fracionários das variáveis de movimento. Cada mensagem deste tipo precisava de indicar três coisas: que se tratava de uma escrita (alteração de parâmetros), qual das três estruturas de movimento é que se destinava e a que variável interna dessa estrutura se pretendia modificar. Seguiu-se a conversão do excerto numérico da cadeia para o respetivo valor, podendo esta ser repetida várias vezes, dependendo somente da necessidade das variáveis em questão, como por exemplo nos campos de arrays de pontos.

Enquanto numa primeira implementação deste modo era necessário cerca de 20 estados apenas para alterar uma única variável, a adoção de ponteiros permitiu, nas iterações mais recentes, seleccionar qualquer uma das três estruturas de movimentos apenas com uma variável genérica, compactando assim todo o esquema da máquina de estados.

4.9.1.2 Módulo final

A `FSM_Serial_reader()` funciona como uma camada de aquisição, representada na figura 4.24. Consoante o modo ativo, lê da porta série um único carácter (modo simples) ou uma cadeia de caracteres até ao fim da mensagem (modo de comandos por cadeia), disponibilizando o resultado numa variável de saída própria e ativando, só depois de a leitura estar completa, uma flag de linha pronta. Esta flag existe para impedir que a `FSM_Serial_Command()` leia valores da porta série antes do momento devido, o que poderia sobrepor um novo comando a outro que ainda estivesse a ser executado. Assim, a máquina de comandos nunca precisa de ler diretamente da porta série, consumindo

sempre comandos já preparados.

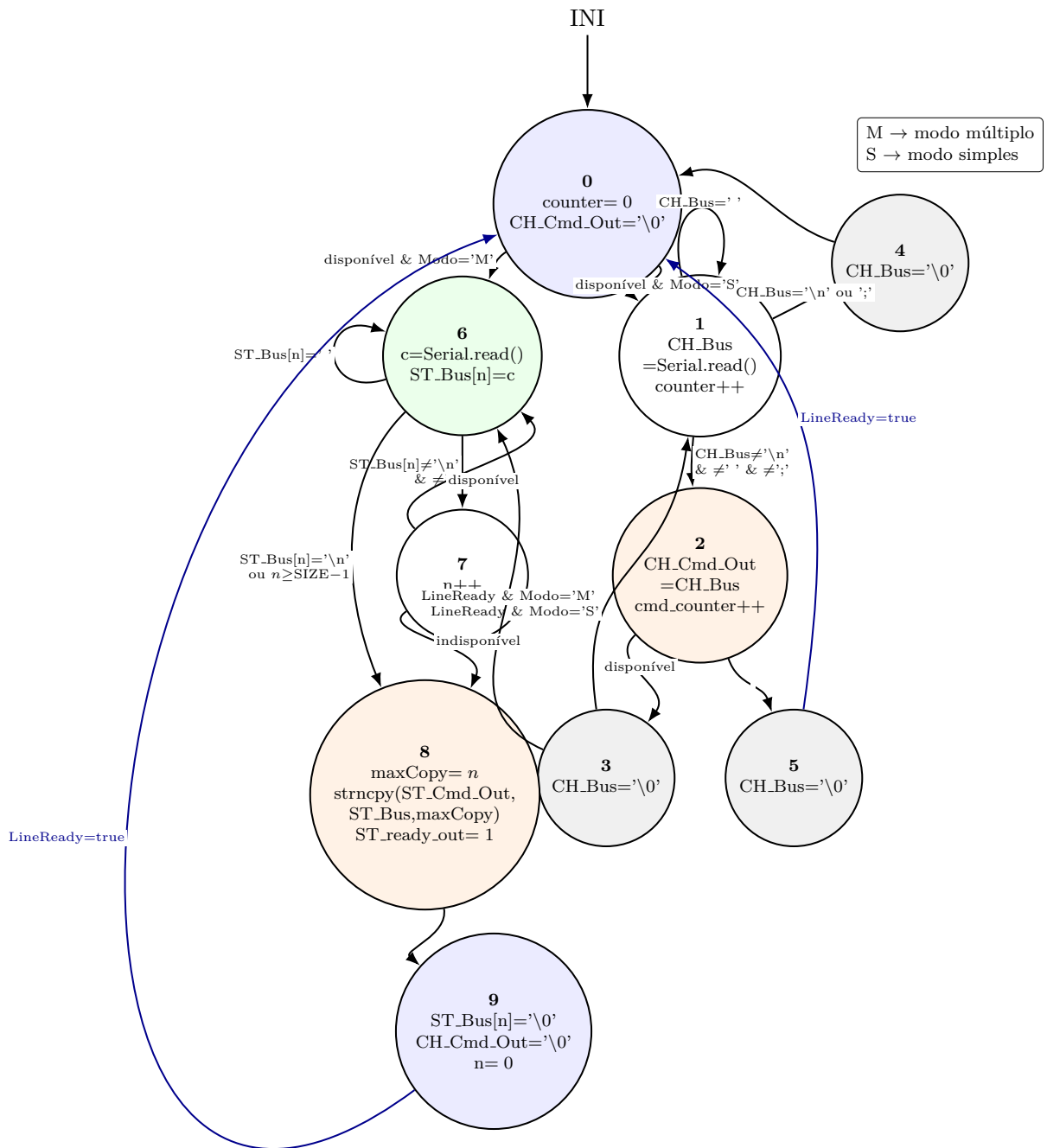


Figura 4.24: Esquema da FSM.Serial_Reader.

A `FSM_Serial_Command()` funciona como a camada de decisão, ilustrada na figura 4.25, na figura 4.26 e na figura 4.27. Esta máquina verifica se existe um comando disponível, interpreta o carácter ou a cadeia recebida, atualiza flags, seleciona que movimentos devem ficar ativos e, nos comandos longos, altera parâmetros das estruturas `MotionStorage`. Os comandos disponíveis dividem-se em três categorias: comandos simples, identificados por um único carácter e que despoletam de imediato uma ação, comandos de menu, que exigem um segundo carácter para selecionar uma opção dentro de um submenu, e comandos longos, introduzidos por uma cadeia de caracteres usada exclusivamente para alterar parâmetros internos de um dos três movimentos.

Os principais comandos simples disponíveis são:

Tabela 4.4: Comandos simples disponíveis na comunicação série.

Comando	Função
S	Inicia a execução do ciclo de movimentos.
P	Para a execução e calcula a duração do ensaio.
L	Entra no menu de leitura, teste e depuração.
M	Entra no menu de seleção do modo fisiológico.
W	Entra no menu de escrita das variáveis.
g	Inicia a gravação na memória interna.
c	Inicia o carregamento da memória interna.

No menu L (leitura), o segundo carácter define a ação a executar:

Tabela 4.5: Subcomandos do menu de leitura, teste e depuração.

Comando	Função
L1	Ativa o modo de debug.
L2	Desativa o modo de debug.
L3	Teste manual da tosse.
Lr	Imprime os dados de respiração.
Lb	Imprime os dados de batimento.
Lt	Reservado para leitura dos dados de tosse.
La	Impressão geral dos dados.

No menu M (Mode), o segundo carácter escolhe que movimentos ficam ativos:

Tabela 4.6: Subcomandos de seleção do modo fisiológico.

Comando	Modo selecionado
Ma	Ativa os modos respiração, batimento e tosse.
Mh	Ativa o modo humano (respiração + batimento).
Mr	Ativa o modo respiração.
Mb	Ativa o modo batimento.
Ms	Desativa todos os movimentos, mantendo o sistema em modo estático.

Os comandos longos são ativados pelo carácter **W** e permitem enviar, de seguida, uma cadeia de caracteres que altera diretamente uma variável interna de um dos três movimentos. Essa cadeia começa sempre por identificar o movimento de destino com **R**, **B** ou **T**, correspondentes a respiração, batimento e tosse, seguido de uma etiqueta de cinco caracteres que identifica a variável a alterar e do respetivo valor entre parênteses retos. Por exemplo, a cadeia **RPerid[4.0]** altera o período da respiração para 4,0 s. A tabela 4.7 lista as oito variáveis atualmente acessíveis por esta via.

Tabela 4.7: Etiquetas dos comandos longos e variáveis internas correspondentes.

Etiqueta	Variável afetada	Tipo
n_pnt	Número de pontos da trajetória (n_Points)	inteiro
Perid	Período do movimento (period)	decimal
TPini	Pausa no início do ciclo (T_pause_Ini)	decimal
TPend	Pausa no fim do ciclo (T_pause_End)	decimal
STInd	Índice inicial do ciclo (start_Index)	inteiro
XYZ_x	Array de coordenadas <i>X</i>	decimal (array)
XYZ_y	Array de coordenadas <i>Y</i>	decimal (array)
XYZ_z	Array de coordenadas <i>Z</i>	decimal (array)

Nos campos que correspondem a arrays de coordenadas (**XYZ_x**, **XYZ_y** e **XYZ_z**), o valor entre parênteses retos pode repetir-se consecutivamente para atualizar vários pontos da trajetória numa única mensagem, como em **RXYZ_x[1.0][2.5][3.0]**.

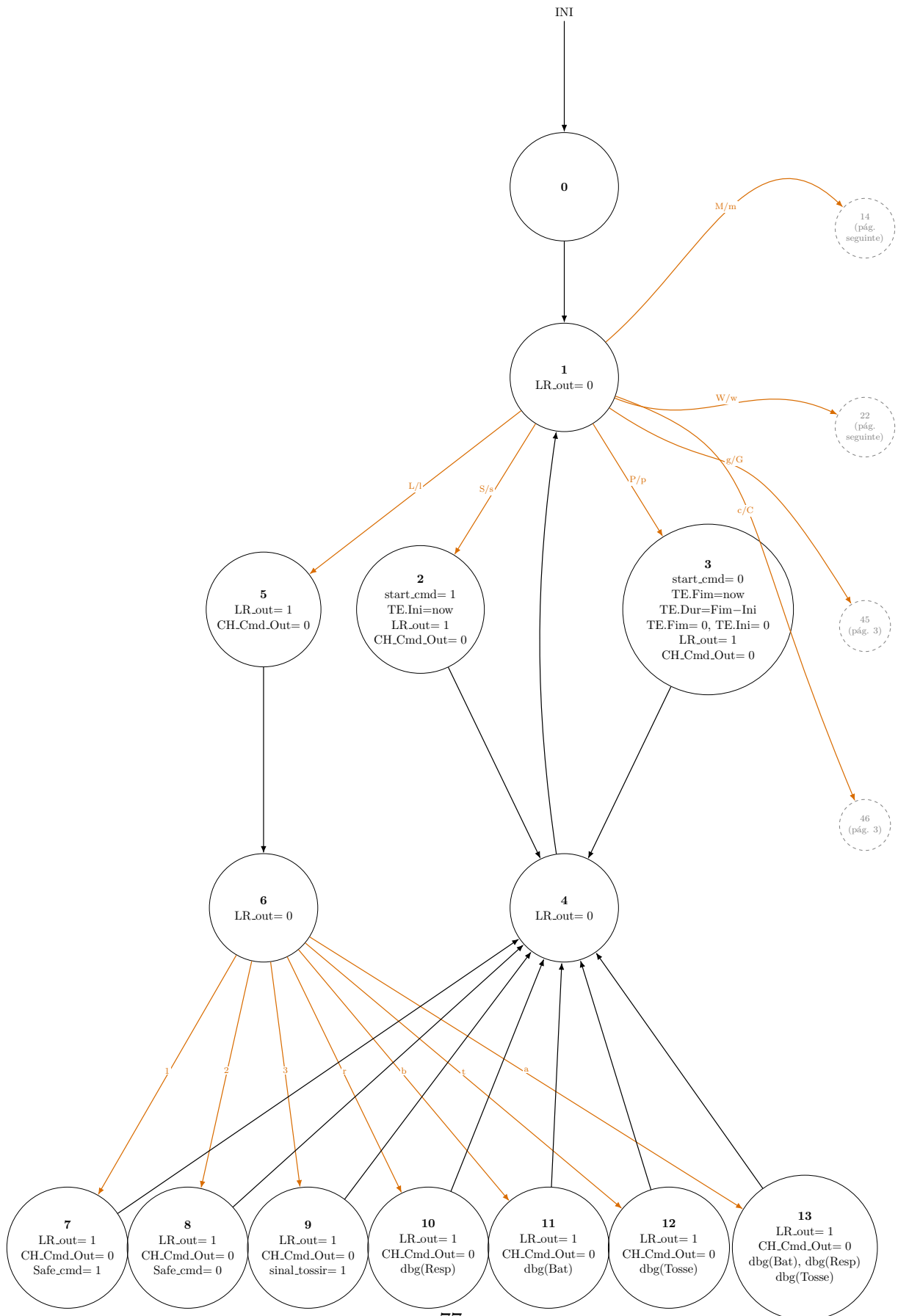


Figura 4.25: FSM_Serial_Command (parte 1 de 3): comandos simples e submenu L.

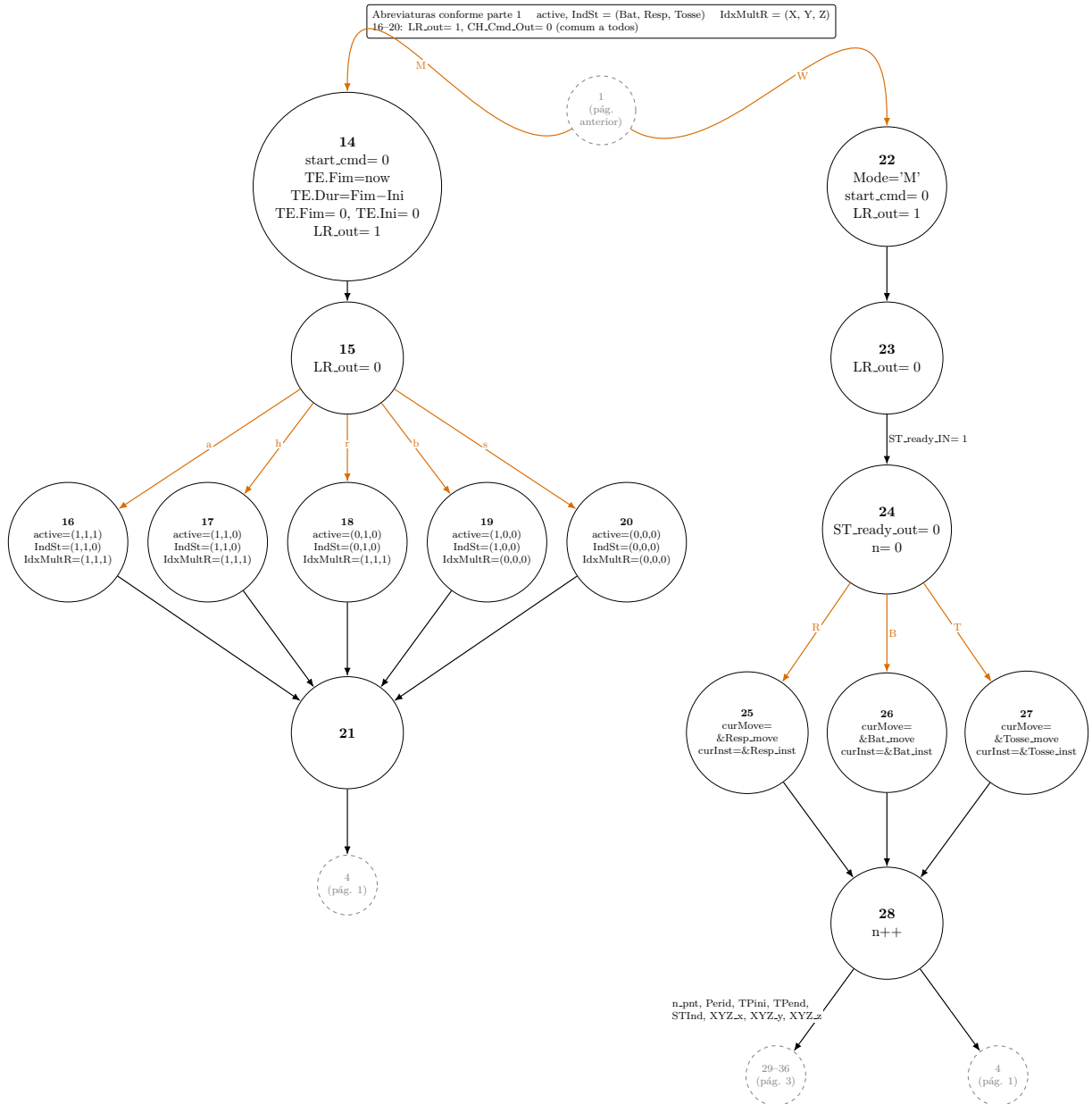


Figura 4.26: FSM_Serial_Command (parte 2 de 3): submenu M e entrada no comando longo W.

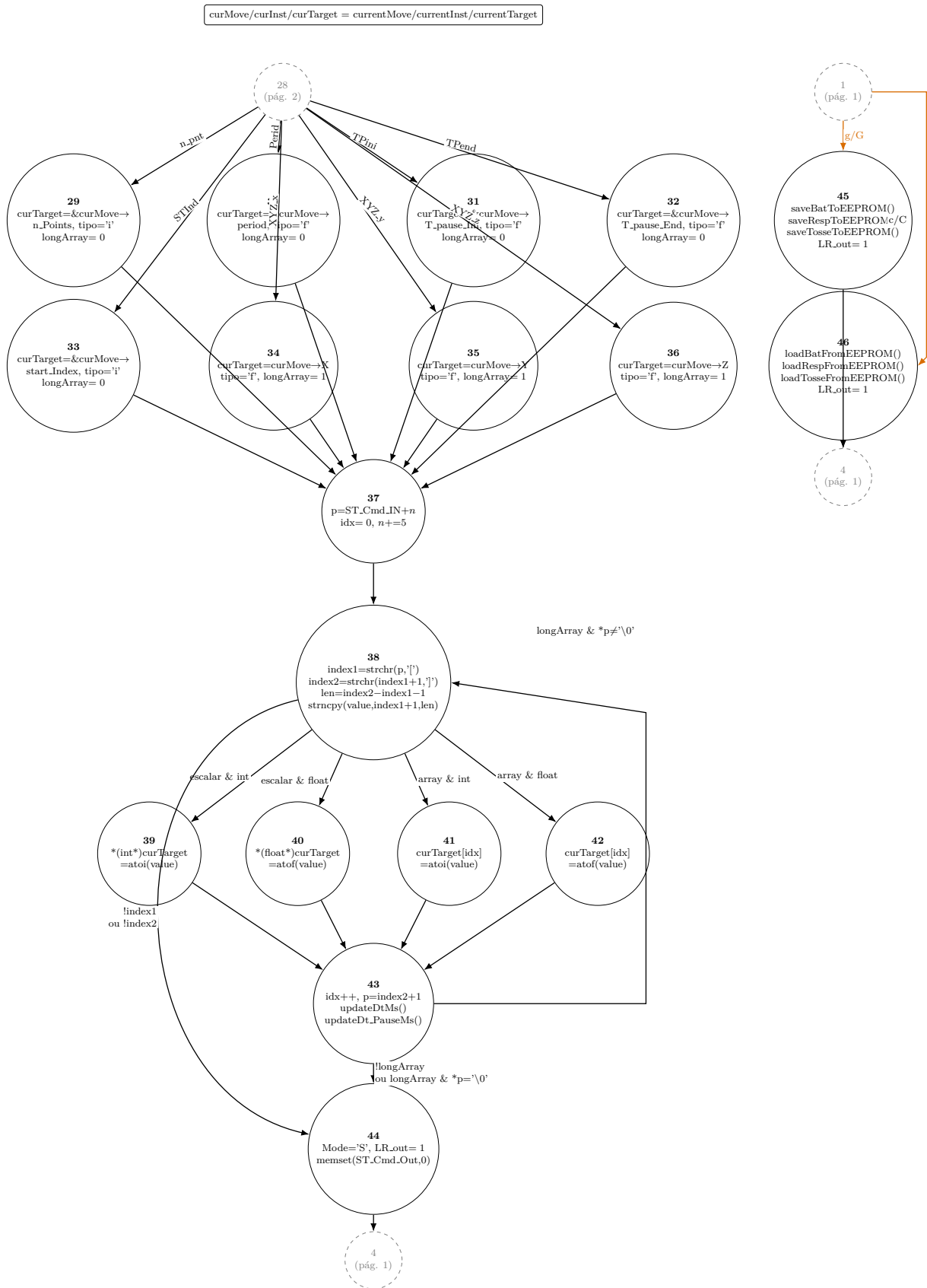


Figura 4.27: FSM_Serial_Command (parte 3 de 3): seleção de campo, escrita do valor e comandos de EEPROM.

4.9.2 Comunicação série no Python (Raspberry Pi)

Tal como na geração de trajetórias, a comunicação série do lado do Raspberry Pi foi também validada de forma isolada, através de uma sucessão de scripts de teste que se foram aproximando gradualmente da arquitetura final. Os primeiros ensaios seguiam ainda um ciclo simples e síncrono (o programa era obrigado a aguardar pela resposta do ESP32, para que o utilizador pudesse enviar um novo comando no monitor série, permitindo assim que o resto do script continuasse a execução).

Já numa fase mais avançada, foi introduzido o controlo manual dos sinais DTR e RTS antes de abrir a porta série, uma técnica também documentada em repositórios [67], com o objetivo de forçar deliberadamente um reset do ESP32 sempre que a ligação série é estabelecida. Foi necessário implementar este controlo, pois nessa altura, era recorrente a abertura da porta série vir acompanhada de um fluxo contínuo de dados corrompidos no monitor série, um comportamento cuja causa exata nunca chegou a ser completamente identificada.

O último destes ensaios aproximou-se já bastante da arquitetura final, ao isolar numa thread dedicada exclusivamente a leitura de cada linha recebida do ESP32, impressa diretamente no terminal, mantendo o ciclo principal dedicado à escrita, que aguarda comandos do utilizador e os envia sem nunca bloquear a receção. Foi também nesta altura que surgiu uma pequena função auxiliar, `print_pi()`, criada para prefixar de forma consistente as mensagens do lado do Raspberry Pi no terminal, distinguindo-as visualmente das recebidas diretamente do ESP32.

As restantes evoluções e modificações foram realizadas, já dentro da aplicação final, para uma arquitetura mais robusta, assente igualmente em duas threads dedicadas: uma de leitura, que consome continuamente as linhas enviadas pelo ESP32 e as coloca numa fila e outra de escrita, que aguarda bloqueada por comandos numa segunda fila e os envia assim que estão disponíveis. Esta separação evita que qualquer chamada à interface fique bloqueada à espera da porta série, permitindo que o resto da aplicação continue a responder mesmo durante falhas ou reconexões.

De modo a completar a arquitetura da comunicação série do lado do Raspberry Pi, foi ainda implementado um sistema de deteção automática da porta série e do respetivo estabelecimento de ligação, com prioridade para os dispositivos identificados de forma persistente em `/dev/serial/by-id`, seguidos de `/dev/ttyUSB*` e `/dev/ttyACM*`, podendo ainda ser forçada uma porta específica através de uma variável de ambiente. Uma vez que a simples abertura da porta série provoca, como referido, o reset do ESP32, a ligação só é considerada efetivamente estabelecida quando uma das mensagens de arranque conhecidas do firmware é reconhecida no fluxo recebido, culminando na deteção da mensagem `"Escolha um comando:"`, que confirma que o microcontrolador terminou o arranque e está pronto a receber comandos. Caso a ligação inicial produza caracteres

corrompidos, uma rotina de reconexão dupla fecha e reabre a porta série duas vezes seguidas, até considerar a ligação estável, garantindo que não é recebido qualquer tipo de dados residuais gerados durante a pausa da ligação.

4.10 Interface HMI

A HMI trata-se do todo que reúne os vários scripts de Python, já mencionados na secção 4.9.2 e na secção 4.4, juntamente com a interface gráfica, que foi por sua vez desenvolvida para transformar os comandos de baixo nível do firmware numa utilização mais simples e natural, já que, sem esta camada, o utilizador teria de memorizar comandos complexos. Com a interface, essas ações passam antes a estar disponíveis através de botões, menus e páginas de configuração, reduzindo a probabilidade de erro e tornando o modelo mais adequado para demonstração e treino.

4.10.1 Escolha da tecnologia e evolução até à versão atual

A escolha da tecnologia para a interface gráfica foi ponderada logo no início desta fase, comparando diretamente quatro alternativas:

- Uma GUI de secretária clássica com Tkinter;
- Uma solução mais robusta mas também mais pesada com PyQt/PySide;
- O PyWebView como simples contentor nativo para uma aplicação web já existente;
- O NiceGUI.

A escolha recaiu sobre este último por permitir escrever a interface inteiramente em Python e obter, a partir do mesmo código, tanto uma janela local no Raspberry Pi como uma página acessível por outros dispositivos na mesma rede, evitando manter duas implementações separadas.

O desenvolvimento em NiceGUI começou com testes de uma janela nativa, e evoluiu depois para uma versão bastante mais completa, já com threads de comunicação série e um estado partilhado guardado em `app.storage.general`, criado para ser acedido tanto pela janela nativa como por um browser ligado à mesma rede. Foi já na transferência deste trabalho para o ambiente do Raspberry Pi, uma vez que até então todo o desenvolvimento e testes tinham decorrido em Windows, que se descobriram problemas de interação entre a biblioteca gráfica e o próprio sistema do Raspberry Pi. Em concreto, tratava-se de um problema conhecido e documentado do WebKitGTK, que corrompia visualmente a janela nativa com riscas e blocos de imagem distorcidos, um problema recorrente na aceleração gráfica do Raspberry Pi e não exclusivo do NiceGUI, reforçando mais tarde a preferência

por soluções de renderização mais simples e previsíveis. Ao longo desta fase, mantiveram-se sempre duas variantes paralelas da interface, uma pensada para testes num computador Windows e outra já direcionada para o Raspberry Pi, distinguidas sobretudo pela forma de deteção da porta série e pela gestão da rede local.



Figura 4.28: Ecrã principal da HMI no Raspberry Pi.

A principal limitação encontrada nesta abordagem não foi a definição da interface em si, mas a sincronização entre clientes: no NiceGUI, os elementos gráficos ficam associados à sessão de cada cliente, pelo que alterar o estado partilhado não atualizava automaticamente o que já tinha sido modificado e executado, nas janelas ou browsers já abertos. A solução passou por um temporizador (`ui.timer`) que, a cada meio segundo, reaplicava o estado partilhado à interface de cada cliente, que se revelou ainda mais problemática quando associada a operações lentas, como a consulta do estado da rede local.

Esta limitação estrutural, associada à necessidade de servir simultaneamente uma página de kiosk e uma página web simplificada para telemóvel, motivou por fim a exploração de uma arquitetura alternativa. A decisão de adotar Flask e Flask-SocketIO resultou de pesquisa adicional sobre arquiteturas de interfaces web em Python, complementada por sugestões do professor orientador, apontando para um modelo de push explícito por eventos SocketIO como forma de resolver de raiz o problema de sincronização entre clientes que a sondagem por temporizador do NiceGUI nunca chegou a resolver de forma satisfatória. Esta mudança trouxe, no entanto, a contrapartida de deixar de ter a definição da interface centralizada em Python, passando a exigir a utilização de templates HTML separados para cada tipo de cliente.

4.10.2 Arquitetura da aplicação final

Na versão para Raspberry Pi, o ficheiro principal da interface é `RB_PI4B_Main.PI.py`. A aplicação usa Flask para servir as páginas web e Flask-SocketIO para enviar atualizações em tempo real aos browsers ligados. Esta arquitetura permite ter uma interface local no monitor do Raspberry Pi e, ao mesmo tempo, uma interface móvel acessível por telemóvel ou tablet na mesma rede. Para facilitar esse acesso, a aplicação gera um endereço local e um código QR que aponta para a rota `/mobile`.

A figura 4.29 representa, de forma global, os ficheiros que são executados ou consultados em conjunto com o ficheiro principal.

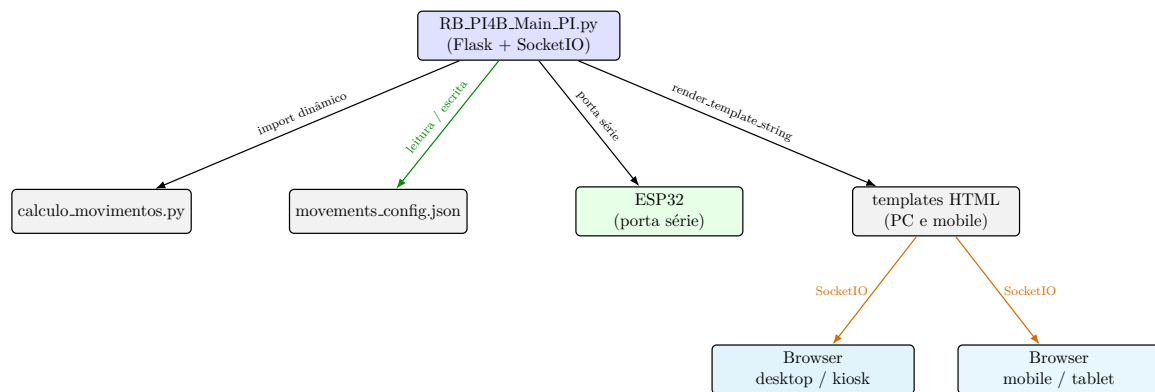


Figura 4.29: Esquema global dos ficheiros e clientes que interagem com a interface HMI.

- O módulo `calculo_movimentos.py`, importado dinamicamente sempre que é pedido um novo cálculo de trajetórias;
- O ficheiro `movements_config.json`, onde ficam guardadas as configurações de movimento entre reinícios;
- A porta série ligada ao ESP32;
- Os dois templates HTML, renderizados consoante o cliente que acede à aplicação e mantidos sincronizados através de eventos SocketIO.

A comunicação com o ESP32 reaproveita o mesmo mecanismo já desenvolvido e testado nas fases iniciais do projeto (secção 4.9.2), passando este a integrar diretamente o próprio ficheiro principal da aplicação, em vez de se manter como um módulo separado, à semelhança do que acontece com o `calculo_movimentos.py`.

O estado da aplicação é guardado numa estrutura partilhada que contém, entre outros campos, o modo de desenvolvimento, o histórico da comunicação série, o estado de arranque/paragem e o modo fisiológico ativo. Sempre que algum destes valores muda, a função de atualização emite um evento SocketIO para todos os browsers ligados. Assim,

se o utilizador abrir simultaneamente a interface no Raspberry Pi e no telemóvel, ambos os ecrãs podem refletir o mesmo estado de funcionamento.

A figura 4.30 detalha a organização interna do `RB_PI4B_Main.PI.py`: as rotas Flask alteram o estado partilhado, consultam o ficheiro de configurações ou invocam o módulo de cálculo de movimentos; os comandos destinados ao ESP32 passam por uma fila de transmissão consumida pela thread de escrita série, enquanto a thread de leitura série atualiza o histórico e, por essa via, o próprio estado partilhado; e qualquer alteração ao estado é distribuída a todos os browsers ligados através de `push_state_update()`.

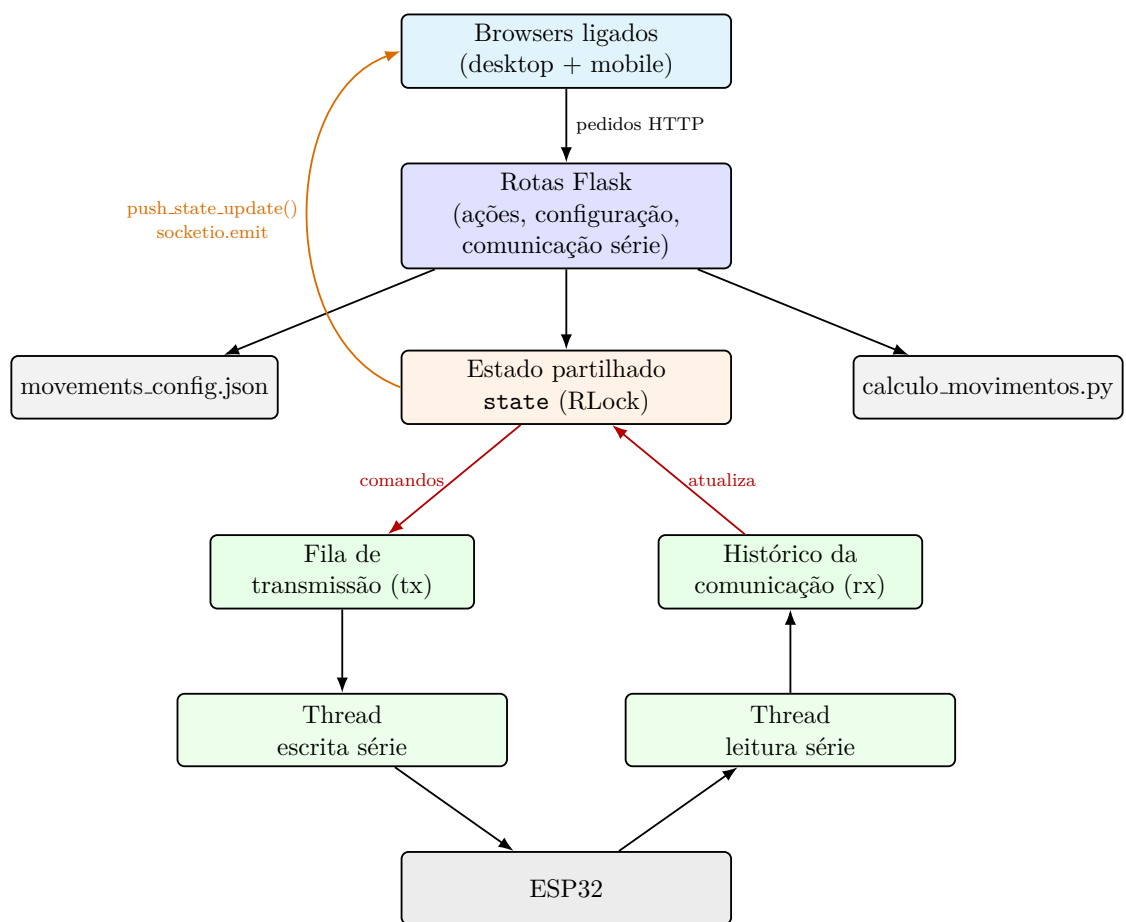


Figura 4.30: Esquema interno do RB_PI4B_Main_PI.py.

4.10.3 Funcionalidades principais

As ações principais da HMI são:

- **Start/Stop:** os botões de arranque e paragem enviam, respetivamente, os comandos `s` e `p` para o ESP32, alterando também o estado visual da interface;
- **Seleção de modo:** o menu de modos converte escolhas como estático, respiração, coração, humano e completo nos comandos série `Ms`, `Mr`, `Mb`, `Mh` e `Ma`;
- **Ligação série:** a interface permite ligar, desligar e tentar reconectar a comunicação com o ESP32, mostrando o estado como desligado, a ligar, ligado ou erro;
- **Monitor série:** em modo de desenvolvimento, o utilizador pode observar as mensagens recebidas do ESP32 e enviar comandos manuais, o que mantém uma via de debugging direta;
- **Modo cliente e modo DEV:** o modo cliente apresenta apenas as ações necessárias à utilização normal, enquanto o modo DEV, protegido por palavra-passe, revela ferramentas de debugging, envio manual de comandos e encerramento completo da aplicação;
- **Configurações de movimentos:** a interface permite editar parâmetros associados à respiração, batimento e tosse, guardar configurações em ficheiro JSON e gerar pontos de movimento a partir do módulo `calculo_movimentos.py`.

4.10.4 Configurações de movimentos

A persistência das configurações assenta num ficheiro JSON (`movements_config.json`), lido e escrito através das funções `load_movements()` e `save_movements()`, o que permite que os parâmetros de cada movimento sobrevivam a reinícios da aplicação. Além do ficheiro permanente, existe ainda um rascunho temporário mantido apenas em memória, que acumula alterações ainda não confirmadas pelo utilizador até serem guardadas ou até a aplicação reiniciar. Sempre que é pedido um novo cálculo de trajetórias, o `RB_PI4B_Main.PI.py` carrega o módulo `calculo_movimentos.py` de forma dinâmica, recorrendo ao módulo `importlib` em vez de um `import` convencional no início do script, o que permite atualizar o ficheiro de cálculo sem necessidade de reiniciar a aplicação principal.

A figura 4.31 detalha a organização interna do `calculo_movimentos.py`, invocado pela rota `/movements/calculador`. A função `gerar_graficos()` combina os parâmetros recebidos com os valores por omissão de cada curva, calcula a curva contínua e os pontos de passagem através de `calcular_curva1/2/3()`, projeta esses pontos para o espaço 3D através de `calcular_3d_curva1/2/3()` e gera, para cada curva, um par de gráficos 2D

e 3D através de `_fig_2d()`/`_fig_3d()`, devolvidos como imagens base64 e como listas de coordenadas X , Y e Z .

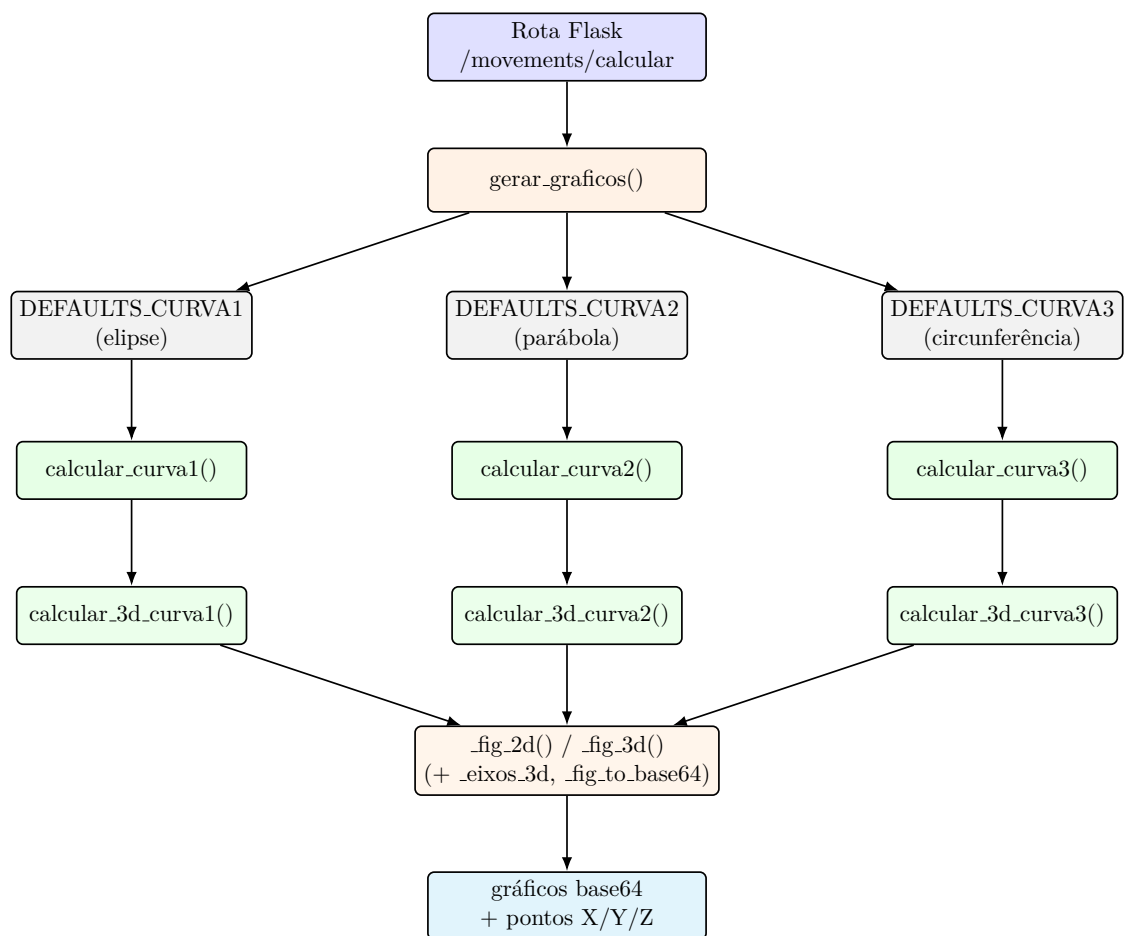


Figura 4.31: Esquema interno do `calculo_movimentos.py`.

Capítulo 5

Testes e Análise de Resultados

De forma a avaliar a capacidade do modelo, foram realizados e documentados dois tipos de teste: o Teste de Laboratório, que avaliou o desempenho mecânico e de software dos movimentos fisiológicos em estudo, de forma individual e em conjunto, e os Testes clínicos, que envolveram a equipa médica e a observação de cenários específicos que simulavam a realidade do exame.

5.1 Teste de Laboratório

Os testes de laboratório serviram para verificar se o sistema integrado, tanto ao nível mecânico como de software, correspondia às expectativas técnicas e motoras definidas para os movimentos do protótipo. Para isso, foram realizados dois ensaios extensos: um teste de deslocamento, dedicado a registar e avaliar de forma independente a precisão dos pedidos de movimento nos três eixos cartesianos, e um teste de frequência, dedicado a registar os movimentos de batimento cardíaco e de respiração às várias frequências em que estes podem ser observados durante um exame médico, confirmando a capacidade do sistema para gerir diferentes velocidades de movimento.

No teste de deslocamento, a plataforma do manipulador delta foi deslocada em linha reta, de forma independente, nos três eixos cartesianos, com o objetivo de avaliar a correspondência entre o deslocamento pedido por software e o deslocamento efetivamente observado na plataforma móvel. Nos eixos X e Y , foram aplicadas posições pretendidas entre -30 mm e 30 mm , com incrementos de 5 mm . Já no eixo Z , o ensaio decorreu entre 45 mm e 105 mm , também com incrementos de 5 mm , por esta ser a gama vertical relevante para a zona de trabalho usada no protótipo.

Os valores registados encontram-se na tabela 5.1. Para cada eixo é apresentado o valor pretendido, o valor obtido por medição e o tamanho do salto entre posições consecutivas. Nos eixos X e Y , o ponto 0 mm foi usado como referência central, pelo que o salto associado a esse ponto é assinalado como nulo. Já no eixo Z , a série de medições começou diretamente à distância de 45 mm , distância esta que equivale ao valor mínimo

permitido para esta configuração do manipulador.

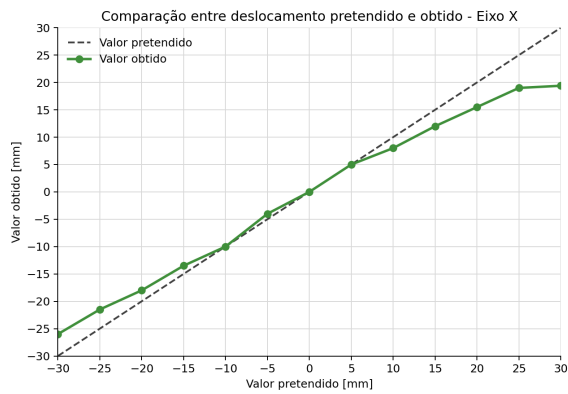
Tabela 5.1: Resultados do ensaio de deslocamento linear do manipulador delta nos eixos X , Y e Z .

Eixo X			Eixo Y			Eixo Z		
Pretendido [mm]	Obtido [mm]	Salto [mm]	Pretendido [mm]	Obtido [mm]	Salto [mm]	Pretendido [mm]	Obtido [mm]	Salto [mm]
-30	-26,0	4,5	-30	-22,5	4,0	45	45,8	–
-25	-21,5	3,5	-25	-18,5	3,5	50	50,0	4,2
-20	-18,0	4,5	-20	-15,0	4,0	55	54,0	4,0
-15	-13,5	3,5	-15	-11,0	3,5	60	58,0	4,0
-10	-10,0	6,0	-10	-7,5	4,0	65	63,5	5,5
-5	-4,0	4,0	-5	-3,5	3,5	70	68,0	4,5
0	0,0	0,0	0	0,0	0,0	75	72,5	4,5
5	5,0	5,0	5	5,0	5,0	80	78,0	5,5
10	8,0	3,0	10	8,5	4,5	85	84,0	6,0
15	12,0	4,0	15	12,0	3,5	90	86,5	2,5
20	15,5	3,5	20	16,5	4,5	95	90,0	3,5
25	19,0	3,5	25	20,5	4,0	100	95,0	5,0
30	19,4	0,4	30	24,5	4,0	105	98,5	3,5

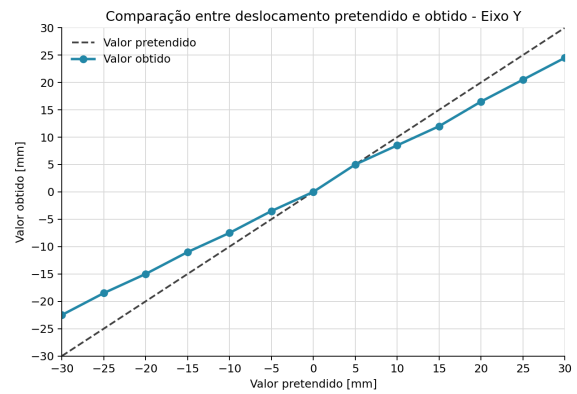
Em cada gráfico, a linha tracejada representa a resposta ideal, isto é, uma relação direta em que o deslocamento medido coincide com o deslocamento pedido. O eixo Y apresenta a resposta mais regular, embora com uma amplitude real inferior à pretendida nas posições negativas. O eixo X segue a tendência esperada na zona central, mas revela uma perda de deslocamento mais evidente nas posições positivas mais distantes (25 mm e 30 mm). No eixo Z , a resposta manteve-se relativamente próxima da trajetória pretendida até à zona intermédia da gama ensaiada, mas passou a apresentar um desvio mais acentuado a partir de cerca de 90 mm , sugerindo perda de precisão nas posições verticais mais elevadas.

Para complementar esta leitura, a figura 5.2 representa apenas o tamanho dos saltos medidos entre posições consecutivas, usando como referência o salto pretendido de 5 mm . Esta análise permite perceber não só o erro absoluto em cada posição, mas também a irregularidade incremental do movimento. Idealmente, todas as barras deveriam aproximar-se da linha tracejada. No entanto, os resultados mostram que os saltos reais variam ao longo da trajetória. No eixo Z , a maior parte dos saltos fica próxima do valor pretendido, mas ainda se observam transições irregulares, nomeadamente entre 85 mm e 95 mm .

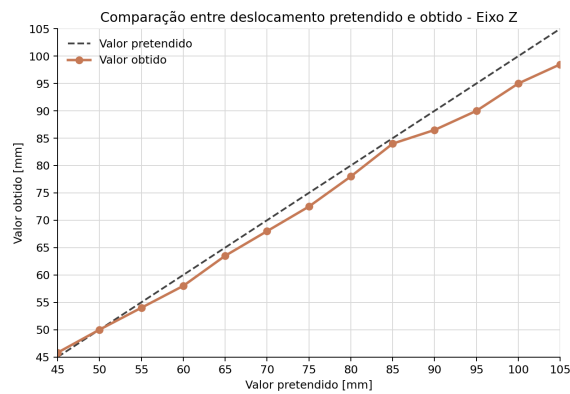
Com estes resultados, foi possível confirmar que o manipulador executa os deslocamentos pedidos dentro de uma margem aceitável, mesmo sendo bastante afetado pela acumulação de erro nas maiores distâncias. Esta limitação poderá dever-se ao facto de ser utilizada uma cinemática inversa simplificada, menos precisa do que as fórmulas matrici-



(a) Eixo X.

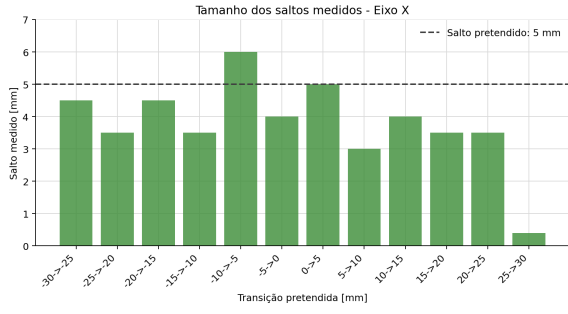


(b) Eixo Y.

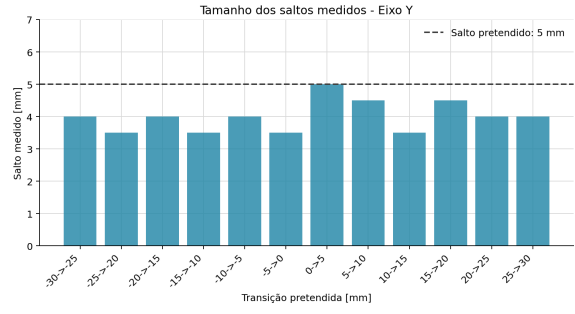


(c) Eixo Z.

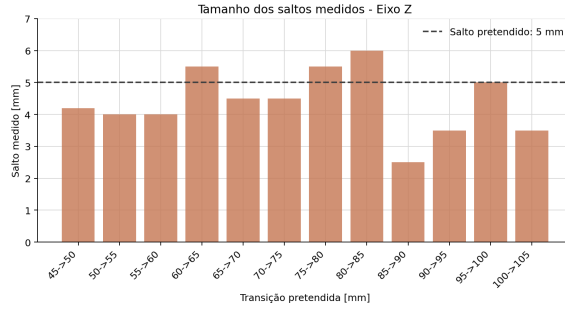
Figura 5.1: Comparação entre deslocamento pretendido e deslocamento obtido no ensaio linear do manipulador delta.



(a) Eixo X.



(b) Eixo Y.



(c) Eixo Z.

Figura 5.2: Tamanho dos saltos medidos entre posições consecutivas no ensaio linear do manipulador delta.

ais mais complexas que descrevem com maior rigor a estrutura de um manipulador delta, à resolução angular limitada do servo, que dificulta a execução de movimentos mais precisos, ou ainda à fricção entre as juntas do manipulador, impressas em vez de metálicas. As discrepâncias observadas são particularmente relevantes para movimentos que dependam de amplitudes maiores ou de posições próximas dos limites mecânicos, devendo por isso ser consideradas em futuras calibrações do sistema. Ainda assim, para os movimentos fisiológicos implementados no protótipo, que recorrem sobretudo a deslocamentos de pequena amplitude e a trajetórias suavizadas, o comportamento observado é suficiente para validar a continuidade do desenvolvimento e avançar para o teste de frequência, descrito de seguida.

O teste de frequência avaliou a componente temporal dos movimentos de batimento cardíaco e de respiração, reproduzindo as diferentes velocidades a que estes podem surgir num exame real. Para ambos os movimentos, foram realizados ensaios em vídeo, filmados a 60 *fps*, nos quais a frequência configurada foi aumentada de forma progressiva, entre 60 *bpm* e 200 *bpm* no caso do batimento cardíaco. Cada vídeo foi analisado e documentado frame a frame, permitindo construir os gráficos e as tabelas de resultados apresentados de seguida.

Os resultados deste ensaio são apresentados na tabela 5.2. A tabela inclui a frequência configurada, a frequência medida a partir do vídeo, o período teórico associado

ao comando, o período medido e o desvio entre os dois valores de frequência. Esta leitura é particularmente importante porque, no modelo implementado, o batimento cardíaco não é apenas uma oscilação contínua: cada ciclo é composto por uma contração rápida e curta, seguida de uma pausa cuja duração varia com a frequência cardíaca configurada.

Tabela 5.2: Resultados do ensaio temporal do movimento de batimento cardíaco.

Frequência configurada [bpm]	Frequência medida [bpm]	Período teórico [s]	Período medido [s]	Desvio [bpm]
60	57,1	1,000	1,051	-2,9
70	66,5	0,857	0,902	-3,5
80	77,2	0,750	0,777	-2,8
90	90,7	0,667	0,662	0,7
100	100,3	0,600	0,598	0,3
110	109,8	0,545	0,546	-0,2
120	119,1	0,500	0,504	-0,9
130	128,3	0,462	0,468	-1,7
140	138,1	0,429	0,434	-1,9
150	146,7	0,400	0,409	-3,3
160	156,1	0,375	0,384	-3,9
170	174,5	0,353	0,344	4,5
180	182,9	0,333	0,328	2,9
190	188,4	0,316	0,318	-1,6
200	192,0	0,300	0,313	-8,0

A figura 5.3 representa graficamente a mesma comparação. A linha tracejada corresponde ao comportamento ideal, no qual a frequência medida coincidiria exatamente com a frequência configurada. De forma geral, o movimento acompanhou a frequência pretendida ao longo de quase toda a gama testada, com desvios relativamente pequenos entre 90 *bpm* e 190 *bpm*. Os maiores afastamentos surgiram nos extremos, em particular aos 60 *bpm*, 70 *bpm* e 200 *bpm*, onde a frequência medida ficou ligeiramente abaixo da frequência configurada.

O movimento de respiração foi avaliado através de um segundo ensaio de frequência, realizado com a câmara reposicionada para tornar mais visível a deslocação global da árvore brônquica durante o ciclo respiratório. Neste ensaio, a frequência configurada foi aumentada progressivamente entre 12 e 20 respirações por minuto, mantendo pausas entre patamares para facilitar a separação dos intervalos de análise.

Os resultados obtidos encontram-se na tabela 5.3. A frequência medida manteve-se muito próxima da frequência configurada em todos os patamares, com desvios reduzidos e sempre inferiores aos observados no ensaio do batimento cardíaco. Este comportamento era expectável, uma vez que o movimento respiratório apresenta uma evolução mais lenta e contínua, facilitando a identificação do período de cada ciclo.

A figura 5.4 apresenta a mesma comparação de forma gráfica. A resposta medida acompanhou de forma consistente a frequência configurada, mantendo-se praticamente sobre a linha de referência. Nas frequências mais elevadas verificou-se apenas uma ligeira

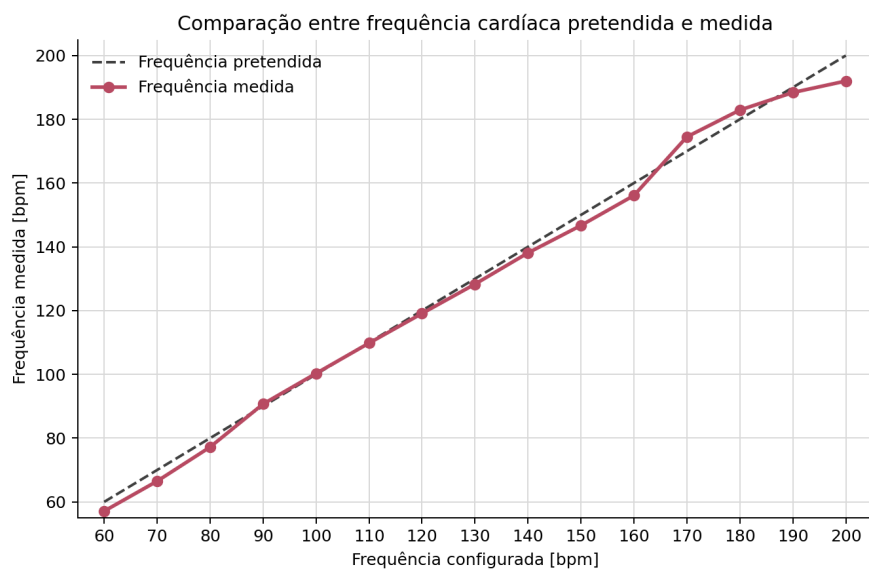


Figura 5.3: Comparação entre a frequência de batimento cardíaco configurada e a frequência medida no vídeo do ensaio.

Tabela 5.3: Resultados do ensaio temporal do movimento de respiração.

Frequência configurada [resp/min]	Frequência medida [resp/min]	Período teórico [s]	Período medido [s]	Desvio [resp/min]
12	12,0	5,000	5,002	0,0
13	12,9	4,615	4,648	-0,1
14	14,0	4,286	4,294	0,0
15	14,9	4,000	4,025	-0,1
16	15,9	3,750	3,783	-0,1
17	16,8	3,529	3,563	-0,2
18	17,8	3,333	3,364	-0,2
19	18,8	3,158	3,198	-0,2
20	19,8	3,000	3,030	-0,2

redução face ao valor pretendido, sem impacto relevante na representação do movimento respiratório, que se manteve regular, progressivo e temporalmente coerente com o comando aplicado.

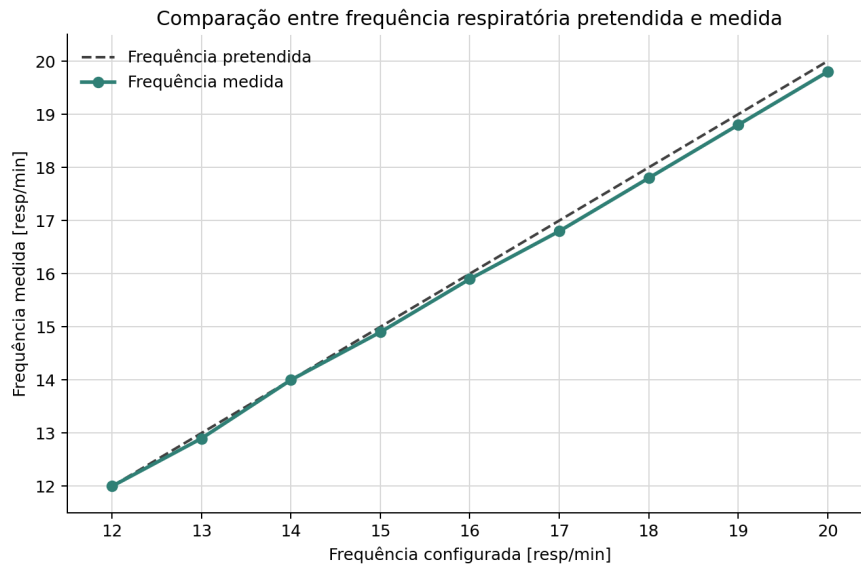


Figura 5.4: Comparação entre a frequência respiratória configurada e a frequência medida no vídeo do ensaio.

O movimento de tosse não foi incluído no teste de frequência. Este perfil foi implementado como um evento transitório e não como um movimento periódico contínuo, pelo que não possui um ciclo completo comparável aos ciclos de respiração ou de batimento cardíaco. Além disso, a sua ocorrência pode ser gerada de forma aleatória pelo sistema, o que inviabiliza a definição de patamares estáveis e repetíveis para análise temporal. Por estes motivos, a tosse foi avaliada apenas como perturbação dinâmica durante os testes de funcionamento do sistema.

5.2 Testes clínicos

Todos os testes realizados com o modelo físico tiveram lugar no Centro Hospitalar Universitário do Algarve (CHUA), em conjunto com a equipa médica do Serviço de Pneumologia responsável pelos exames de broncoscopia, médicos internos e residentes incluídos, que disponibilizou os seus próprios instrumentos de exame para avaliar o desempenho do modelo. Estes testes permitiram avaliar, de forma conjunta, a fidelidade fisiológica do modelo, o comportamento dos movimentos nele implementados e o funcionamento do sistema completo, reunindo assim numa só sessão a validação anatómica, funcional e integrada do protótipo.

Nos primeiros ensaios, a equipa médica confirmou que a geometria da árvore brônquica, no seu todo, se revelava bastante fiel à realidade anatómica, tanto no as-

peto exterior da estrutura impressa como no revestimento interno reconstruído a partir da TAC, chegando mesmo a equipa médica a descrever este último como um aspeto que os “impressionou muito”, por estarem bastante familiarizados com as imagens endoscópicas reais que recolhem no seu dia a dia clínico. Foi, no entanto, necessário reajustar ligeiramente as dimensões do modelo ao longo dos vários testes seguintes, ampliando o diâmetro interno em alguns milímetros, de modo a facilitar a progressão do broncoscópio sem comprometer o realismo da simulação, já que o diâmetro inicial gerava algum conflito de espaço, sobretudo nos brônquios mais distais, face ao instrumento de treino utilizado pela equipa, um broncoscópio Pentax EB 19-J10, com 6,4 mm de diâmetro na extremidade distal.

Foi também notada uma diferença relevante entre o impacto dos movimentos numa perspetiva externa e interna do modelo: na visão proporcionada pelo próprio broncoscópio, as vibrações indesejadas provocadas pelos movimentos respiratórios de baixa frequência não se revelavam tão notórias quanto do lado de fora do modelo, um resultado que a equipa médica considerou positivo para a simulação.

Entre a equipa médica foi ainda discutida a eventual necessidade de alterar o posicionamento do modelo, de modo a simular a posição sentada ou deitada do utente. Apesar de o modelo poder ser adaptado para acomodar essas alterações, concluiu-se que a posição do utente não era suficientemente significativa para comprometer as imagens do exame, pelo que os testes decorreram sempre com o modelo colocado na vertical, como registado na figura 5.5.



(a) Sessão de teste.



(b) Imagem endoscópica.

Figura 5.5: Ensaios clínicos no CHUA.

Ao longo das diferentes iterações do protótipo, foram surgindo problemas adicionais e as respetivas sugestões de melhoria. A passagem para materiais mais flexíveis foi bem recebida pela equipa médica, que classificou esta opção como “mais vantajosa e mais real”. Contudo, um dos aspetos mais referidos pela equipa médica foi precisamente a presença de filamentos e resíduos no interior da árvore brônquica, resultantes tanto de pequenas

imprecisões no processo de impressão como da própria flexibilidade deste tipo de material, que em certas versões dificultaram a progressão do broncoscópio e reduziram a percepção do detalhe anatômico interno. Uma das últimas alterações introduzidas foi, precisamente, um ligeiro aumento adicional do diâmetro interno, de modo a facilitar a deslocação do broncoscópio pelos canais da árvore brônquica e, por consequência, a aprendizagem inicial dos médicos internos.

Das sugestões recolhidas junto da equipa médica ao longo destas sessões, destacam-se ainda as seguintes:

- A introdução de pequenos orifícios nas extremidades dos ramos mais distais, para evitar a acumulação de líquido lubrificante no interior do modelo.
- A criação de orifícios ou fendas em alguns brônquios, onde pudessem ser colocados materiais simulando corpos estranhos ou corpos patológicos a retirar com outros instrumentos médicos.
- O reforço da base de sustentação, tendo a equipa médica sugerido que “maior peso e maior largura dariam maior estabilidade” ao conjunto.
- A possibilidade de, no futuro, acoplar qualquer modelo de árvore brônquica a uma base única equipada com os atuadores, facilitando a substituição entre diferentes versões impressas.

O feedback global da equipa médica foi, ao longo de toda a extensão dos testes, bastante positivo, elogiando tanto o realismo do modelo como a facilidade com que este podia ser manipulado para acomodar diferentes cenários de simulação. Motivada por estes resultados, a equipa médica avançou ainda com a integração de um médico interno do Serviço de Pneumologia, ainda no início da sua formação em broncofibroscopia, como piloto de testes, com o objetivo de medir a evolução da sua curva de aprendizagem com recurso ao modelo, documentando e validando assim a eficácia do projeto também do ponto de vista pedagógico.

Capítulo 6

Conclusão

6.1 Síntese do trabalho

Este projeto teve como objetivo desenvolver um modelo físico e articulado de uma árvore brônquica para apoio ao treino de videobroncofibroscopia. A solução desenvolvida combinou a reconstrução anatômica a partir de dados clínicos, impressão 3D, atuação mecânica por manipuladores delta, controlo eletrónico com ESP32, comunicação com Raspberry Pi 4B, uma interface HMI e perfis de movimento inspirados em fenómenos fisiológicos observáveis durante o exame.

Ao longo do desenvolvimento, foi possível consolidar os requisitos principais do sistema e transformar uma ideia inicial ainda pouco definida num protótipo funcional, modular e capaz de reproduzir, de forma controlada, três movimentos considerados relevantes para a simulação: respiração, batimento cardíaco e tosse. A opção por manipuladores delta permitiu concentrar a atuação fora do modelo anatômico, evitando a instalação de motores diretamente na árvore brônquica e transferindo grande parte da flexibilidade do sistema para o software, através da geração de trajetórias e da cinemática inversa.

Os ensaios laboratoriais permitiram caracterizar o comportamento do manipulador e avaliar a resposta temporal dos movimentos implementados. Embora tenham sido observadas discrepâncias entre os deslocamentos pretendidos e os deslocamentos obtidos, sobretudo em amplitudes maiores e em posições mais afastadas da zona central de trabalho, os resultados mostraram que o sistema era capaz de executar movimentos coerentes com os comandos aplicados. Nos testes de frequência, tanto o batimento cardíaco como a respiração acompanharam de forma consistente os valores configurados, confirmando a viabilidade da arquitetura de controlo adotada para os perfis fisiológicos previstos.

Os testes realizados com a equipa médica permitiram ainda validar qualitativamente a utilidade do modelo enquanto ferramenta de treino. A geometria da árvore brônquica foi considerada próxima da realidade anatômica e o comportamento dinâmico introduzido pelo sistema foi visto como um contributo relevante para aproximar o simu-

lador de um ambiente broncoscópico real. O trabalho desenvolvido cumpriu, assim, o objetivo principal de demonstrar a viabilidade de um simulador físico, acessível e evolutivo, capaz de combinar anatomia impressa em 3D com movimento controlado.

6.2 Limitações

Apesar dos resultados obtidos, o protótipo apresentou limitações importantes. A primeira esteve relacionada com o próprio modelo impresso em 3D, que não foi concebido para lavagem interna. Esta limitação reduz a capacidade de reutilização em cenários onde seja necessário aplicar lubrificantes, introduzir líquidos ou simular procedimentos que exijam limpeza posterior do interior da árvore brônquica. Além disso, a geometria atual não permite ainda a colocação simples e controlada de objetos no interior do modelo, o que limita a simulação de corpos estranhos ou de situações clínicas mais específicas.

Outra limitação relevante esteve associada à precisão dos movimentos. Os servomotores utilizados permitiram construir uma solução compacta e de baixo custo, mas não garantiram uma correspondência perfeita entre a posição calculada e o deslocamento real da plataforma. Esta diferença resulta da resolução e repetibilidade dos atuadores, das folgas mecânicas, da fricção nas juntas impressas e das próprias aproximações usadas na cinemática inversa. Em movimentos de pequena amplitude, estas discrepâncias foram menos críticas, mas tornaram-se mais visíveis em deslocamentos maiores ou próximos dos limites mecânicos.

Foi também observada a presença de vibrações e pequenas irregularidades, sobretudo em movimentos de baixa velocidade. Estas vibrações podem introduzir inconsistências na suavidade do movimento respiratório e alterar ligeiramente a percepção externa do comportamento do modelo. Embora, durante os testes clínicos, estas vibrações se tenham revelado menos evidentes na visão interna do broncoscópio do que na observação exterior, continuam a representar uma limitação importante para futuras versões que procurem maior fidelidade e repetibilidade.

Por fim, o protótipo ainda simplifica vários fenómenos presentes numa árvore traqueobrônquica real. O modelo não reproduz deformações locais do lúmen, variações de calibre ao longo do ciclo respiratório, estenoses dinâmicas, resistência ao contacto com o broncoscópio ou alterações provocadas diretamente pela interação entre o instrumento e as paredes internas. Estes aspetos foram identificados como relevantes, mas ficaram fora do âmbito desta versão devido à complexidade mecânica, material e de controlo que implicariam.

6.3 Trabalho futuro

Como trabalho futuro, poderá ser aprofundada a evolução do modelo físico da árvore brônquica, explorando novas tecnologias e materiais de impressão que permitam obter maior flexibilidade, melhor recuperação elástica e uma resposta mecânica mais próxima dos tecidos reais. Esta evolução poderá passar pela utilização de filamentos flexíveis com diferentes durezas, resinas elásticas, silicones ou processos híbridos de fabrico, procurando combinar uma geometria anatômica detalhada com paredes mais deformáveis e resistentes ao uso repetido.

Dentro dessa evolução construtiva, poderá também ser considerada a lavagem interna do modelo. Para esse efeito, seria útil introduzir soluções que facilitem a drenagem e reduzam a acumulação de resíduos no interior dos ramos, aproximando a utilização do simulador de condições de treino mais completas e aumentando a robustez do modelo em sessões repetidas.

Outra possibilidade de desenvolvimento será a introdução de objetos ou elementos substituíveis no interior da árvore brônquica, permitindo simular corpos estranhos, obstruções ou outros cenários clínicos. Esta capacidade tornaria o modelo mais versátil, sobretudo para médicos em fase inicial de aprendizagem, ao permitir exercícios de navegação, identificação e eventual remoção de elementos colocados em posições específicas.

No sistema de atuação e controlo, poderão ser otimizados tanto o firmware do ESP32 como a interface HMI. Do lado do firmware, futuras versões poderão melhorar a suavização dos movimentos, a calibração dos atuadores, a gestão de limites mecânicos e a sincronização entre respiração, batimento cardíaco e tosse. Do lado da HMI, poderá ser desenvolvida uma interface mais intuitiva e interativa, com melhor organização dos modos de treino, maior controlo sobre os parâmetros dos movimentos, armazenamento de perfis e uma visualização mais clara do estado interno do sistema.

Por último, poderá ser vantajoso melhorar as condições de avaliação e treino disponibilizadas aos médicos. Para além da validação qualitativa já iniciada, poderão ser definidos testes e exames mais estruturados para avaliar o modelo físico, envolvendo diferentes utilizadores, níveis de experiência e cenários clínicos. Esta etapa permitiria perceber com maior rigor que componentes do simulador têm maior valor pedagógico, que limitações afetam mais a experiência de utilização e que melhorias poderiam ser priorizadas para transformar o protótipo numa ferramenta de treino mais completa e clinicamente útil.

Bibliografia

- [1] Mohsen Davoudi e Henri G. Colt. «Bronchoscopy simulation: a brief review». Em: *Advances in Health Sciences Education* 14.2 (2008), pp. 287–296. DOI: [10.1007/s10459-007-9095-x](https://doi.org/10.1007/s10459-007-9095-x).
- [2] Cassie C. Kennedy, Fabien Maldonado e David A. Cook. «Simulation-Based Bronchoscopy Training: Systematic Review and Meta-analysis». Em: *Chest* 144.1 (2013), pp. 183–192. DOI: [10.1378/chest.12-1786](https://doi.org/10.1378/chest.12-1786).
- [3] David I. Fielding, Fabien Maldonado e Septimiu Murgu. «Achieving competency in bronchoscopy: Challenges and opportunities». Em: *Respirology* 19.4 (2014), pp. 472–482. DOI: [10.1111/resp.12279](https://doi.org/10.1111/resp.12279).
- [4] Philip Mørkeberg Nilsson et al. «Simulation in bronchoscopy: current and future perspectives». Em: *Advances in Medical Education and Practice* 8 (2017), pp. 755–760. DOI: [10.2147/AMEP.S139929](https://doi.org/10.2147/AMEP.S139929).
- [5] Mallika Gopal et al. «Bronchoscopy Simulation Training as a Tool in Medical School Education». Em: *The Annals of Thoracic Surgery* 106.1 (2018), pp. 280–286. DOI: [10.1016/j.athoracsur.2018.02.011](https://doi.org/10.1016/j.athoracsur.2018.02.011).
- [6] Pedro Barros, Bruno Santos e Ulisses Brito. «Projeto para desenvolvimento de modelo de simulação de árvore brônquica, com impressão 3D». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2023.
- [7] T. H. Pedersen et al. «A randomised, controlled trial evaluating a low cost, 3D-printed bronchoscopy simulator». Em: *Anaesthesia* 72.8 (2017), pp. 1005–1009. DOI: [10.1111/anae.13951](https://doi.org/10.1111/anae.13951).
- [8] Rao Fu et al. «Dynamic three-dimensional printing: The future of bronchoscopic simulation training?» Em: *Anaesthesia and Intensive Care* 51.4 (2023), pp. 274–280. DOI: [10.1177/0310057X231154015](https://doi.org/10.1177/0310057X231154015).
- [9] Peter Heinbecker. «A method for the demonstration of calibre changes in the bronchi in normal respiration». Em: *Journal of Clinical Investigation* 4.4 (1927), pp. 459–469. DOI: [10.1172/JCI100133](https://doi.org/10.1172/JCI100133).

- [10] Maxwell Ellis. «The mechanism of the rhythmic changes in the calibre of the bronchi during respiration». Em: *The Journal of Physiology* 87.3 (1936), pp. 298–309. DOI: [10.1113/jphysiol.1936.sp003408](https://doi.org/10.1113/jphysiol.1936.sp003408).
- [11] Alexander Chen et al. «The Effect of Respiratory Motion on Pulmonary Nodule Location During Electromagnetic Navigation Bronchoscopy». Em: *Chest* 147.5 (2015), pp. 1275–1281. DOI: [10.1378/chest.14-1425](https://doi.org/10.1378/chest.14-1425).
- [12] Chamindu C. Gunatilaka et al. «The effect of airway motion and breathing phase during imaging on CFD simulations of respiratory airflow». Em: *Computers in Biology and Medicine* 127 (2020), p. 104099. DOI: [10.1016/j.compbiomed.2020.104099](https://doi.org/10.1016/j.compbiomed.2020.104099).
- [13] Sissel Højsted Kronborg et al. «Four different models for simulation-based training of bronchoscopic procedures». Em: *BMC Pulmonary Medicine* 24 (2024), p. 23. DOI: [10.1186/s12890-024-02846-9](https://doi.org/10.1186/s12890-024-02846-9).
- [14] Anke Schertel, Thomas Geiser e Wolf E. Hautz. «Man or machine? Impact of tutor-guided versus simulator-guided short-time bronchoscopy training on students learning outcomes». Em: *BMC Medical Education* 21 (2021), p. 123. DOI: [10.1186/s12909-021-02526-w](https://doi.org/10.1186/s12909-021-02526-w).
- [15] Annamaria Witheridge, Gordon Ferns e Wesley Scott-Smith. «Revisiting Miller’s pyramid in medical education: the gap between traditional assessment and diagnostic reasoning». Em: *International Journal of Medical Education* 10 (2019), pp. 191–192. DOI: [10.5116/ijme.5d9b.0c37](https://doi.org/10.5116/ijme.5d9b.0c37).
- [16] Julien Pico et al. «From Realism to Learner Engagement: Rethinking Fidelity in Simulation-Based Education». Em: *JMIR Medical Education* (2026).
- [17] P. A. Baker et al. «Evaluating the ORSIM simulator for assessment of anaesthetists’ skills in flexible bronchoscopy: aspects of validity and reliability». Em: *British Journal of Anaesthesia* 117.S1 (2016), pp. i87–i91. DOI: [10.1093/bja/aew059](https://doi.org/10.1093/bja/aew059).
- [18] Amir Hossein Hadi Hosseinabadi e Septimiu E. Salcudean. «Force Sensing in Robot-assisted Keyhole Endoscopy: A Systematic Survey». Em: *The International Journal of Robotics Research* (2021).
- [19] Justin L. Garner et al. «Evaluation of a re-useable bronchoscopy biosimulator with ventilated lungs». Em: *ERJ Open Research* 5.2 (2019). DOI: [10.1183/23120541.00035-2019](https://doi.org/10.1183/23120541.00035-2019).
- [20] Martin Schulze et al. «Quality assurance of 3D-printed patient specific anatomical models: a systematic review». Em: *3D Printing in Medicine* 10 (2024), p. 9. DOI: [10.1186/s41205-024-00210-5](https://doi.org/10.1186/s41205-024-00210-5).

- [21] Carlos Aravena e Thomas R. Gildea. «Patient-specific airway stent using three-dimensional printing: a review». Em: *Annals of Translational Medicine* 11.10 (2023), p. 360. DOI: [10.21037/atm-22-2878](https://doi.org/10.21037/atm-22-2878).
- [22] Bengi Yilmaz e Bilge Yilmaz Kara. «Mathematical surface function-based design and 3D printing of airway stents». Em: *3D Printing in Medicine* 8 (2022), p. 24. DOI: [10.1186/s41205-022-00154-8](https://doi.org/10.1186/s41205-022-00154-8).
- [23] Qungang Shan et al. «Customization of stent design for treating malignant airway stenosis with the aid of three-dimensional printing». Em: *Annals of Translational Medicine* (2019).
- [24] Gregory Doucet et al. «Modelling and Manufacturing of a 3D Printed Trachea for Cricothyroidotomy Simulation». Em: *Annals of Simulation Technology* (2017).
- [25] Lily Park et al. «Increasing Access to Medical Training With Three-Dimensional Printing: Creation of an Endotracheal Intubation Model». Em: *JMIR Medical Education* (2023).
- [26] Brian Han Khai Ho et al. «Multi-material three dimensional printed models for simulation of bronchoscopy». Em: *BMC Medical Education* 19 (2019), p. 236. DOI: [10.1186/s12909-019-1677-9](https://doi.org/10.1186/s12909-019-1677-9).
- [27] Christopher T. Leba et al. «Use of a 3D Printed, Color-Coded Airway Model for Bronchoscopy Training and Anatomy Learning». Em: *Journal of 3D Printing in Medicine* 7.2 (2023). DOI: [10.2217/3dp-2023-0005](https://doi.org/10.2217/3dp-2023-0005).
- [28] Bruno Santos. «Simulação dinâmica da videobroncofibroscopia em modelo impresso 3D: introdução de movimento no modelo de árvore brônquica». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2026.
- [29] Byeong-Jun Kim et al. «A novel computer modeling and simulation technique for bronchi motion tracking in human lungs under respiration». Em: *Physical and Engineering Sciences in Medicine* 46 (2023), pp. 1741–1753. DOI: [10.1007/s13246-023-01336-2](https://doi.org/10.1007/s13246-023-01336-2).
- [30] Milovan Savanović et al. «Quantification of Lung Tumor Motion and Optimization of Treatment». Em: *Journal of Biomedical Physics and Engineering* (2023).
- [31] Stephen Dubsky et al. «Cardiogenic Airflow in the Lung Revealed Using Synchrotron-Based Dynamic Lung Imaging». Em: *Scientific Reports* 8 (2018). DOI: [10.1038/s41598-018-23193-w](https://doi.org/10.1038/s41598-018-23193-w).
- [32] Makoto Ando e Takako Nishino. «Understanding the Mechanisms of Main Bronchial Compression in Patients with Intracardiac Anomalies». Em: *Annals of Thoracic Surgery Short Reports* 2 (2024), p. 369. DOI: [10.1016/j.atssr.2024.03.008](https://doi.org/10.1016/j.atssr.2024.03.008).

- [33] Francesco Andrani et al. «Cough, a vital reflex. Mechanisms, determinants and measurements». Em: *Acta Biomedica* 89.4 (2018), pp. 477–480. DOI: [10.23750/abm.v89i4.6182](https://doi.org/10.23750/abm.v89i4.6182).
- [34] Olusegun J. Ilegbusi. «Computational modeling of cough-induced droplets and mucosal film dynamics in the upper airway for pulmonary disease classification». Em: *Frontiers in Physiology* (2025). DOI: [10.3389/fphys.2025.1666826](https://doi.org/10.3389/fphys.2025.1666826).
- [35] Yingtian Li et al. «A Soft Robotic Balloon Endoscope for Airway Procedures». Em: *Soft Robotics* 9.5 (2022). DOI: [10.1089/soro.2020.0161](https://doi.org/10.1089/soro.2020.0161).
- [36] Roberto W. Dal Negro et al. «Prevalence of tracheobronchomalacia and excessive dynamic airway collapse in bronchial asthma of different severity». Em: *Multidisciplinary Respiratory Medicine* 8 (2013), p. 32.
- [37] Jonathan E. Hiorns, Oliver E. Jensen e Bindi S. Brook. «Nonlinear Compliance Modulates Dynamic Bronchoconstriction in a Multiscale Airway Model». Em: *Biophysical Journal* 107 (2014), pp. 3030–3042.
- [38] Plan3D. *Simulador de Broncoscopia*. URL: <https://plan3d.cl/product/simulador-de-broncoscopia/>.
- [39] TruCorp. *AirSim Bronchi*. URL: <https://trucorp.com/product/airsim-bronchi/>.
- [40] USPTO. *Teaching model of the bronchial and lungs useful for teaching the biology of those organs*. URL: <https://patents.google.com/patent/US5597310A/en>.
- [41] M. E. Giannaccini et al. «Respiratory Simulator for Robotic Respiratory Tract Treatments». Em: *2017 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2017. DOI: [10.1109/ROBIO.2017.8324764](https://doi.org/10.1109/ROBIO.2017.8324764).
- [42] Markus A. Horvath et al. «An organosynthetic soft robotic respiratory simulator». Em: *APL Bioengineering* 4 (2020).
- [43] Rachael Pare et al. «Electro-mechanical Lung Simulator Using Polymer and Organic Human Lung Equivalents for Realistic Breathing Simulation (xPULM)». Em: *Scientific Reports* (2019). DOI: [10.1038/s41598-019-56176-6](https://doi.org/10.1038/s41598-019-56176-6).
- [44] Jina Chang, Tae-Suk Suh e Dong-Soo Lee. «Development of a deformable lung phantom for the evaluation of deformable registration». Em: *Journal of Applied Clinical Medical Physics* (2010).
- [45] Rong-Heng Zhao et al. «Precision-Controlled Bionic Lung Simulator for Dynamic Respiration Simulation». Em: *Bioengineering (Basel)* (2025).
- [46] USPTO. *Servo lung simulator and related control method*. URL: <https://patents.google.com/patent/US5975748A/en>.

- [47] CNIPA. *Chest retraction simulating device and medical patient simulator having the device*. URL: <https://patents.google.com/patent/CN101447141B/en>.
- [48] EPO. *Ensemble d'apprentissage et simulateur de torse de nourrisson pour l'apprentissage du geste de la kinésithérapie respiratoire*. URL: <https://patents.google.com/patent/EP2430628B1/fr>.
- [49] CNIPA. *Full-automatic high-simulation body position drainage simulation training model*. URL: <https://patents.google.com/patent/CN107346634B/en>.
- [50] Atsuo Takanishi et al. *WKA (Waseda-Kyoto Kagaku Airway) airway management training robot series*. URL: <https://link.springer.com/article/10.1007/s11548-008-0238-1>.
- [51] Andrea Dunn Beltran et al. *Stop Holding Your Breath: CT-Informed Gaussian Splatting for Dynamic Bronchoscopy*. 2026. arXiv: 2604.28179. URL: <https://arxiv.org/abs/2604.28179>.
- [52] Joshua B. Gafford et al. «A Concentric Tube Robot System for Rigid Bronchoscopy: A Feasibility Study on Central Airway Obstruction Removal». Em: *Annals of Bio-medical Engineering* 48.1 (2019), pp. 181–191. DOI: [10.1007/s10439-019-02325-x](https://doi.org/10.1007/s10439-019-02325-x).
- [53] Daniel Cíntora Morales e Daniel Octavio Delgado Vargas. «Prototipo Simulador de Broncoscopia». Trabalho Terminal, Engenharia em Biónica. Instituto Politécnico Nacional, UPIITA, 2012.
- [54] Formlabs. *Flexible 80A Resin*. URL: <https://formlabs.com/eu/products/flexible-80a-resin/>.
- [55] Inslogic. *Inslogic Flexible 70A Resin*. URL: <https://www.inslogic3d.com/products/inslogic-flexible-70a-resin>.
- [56] Liqcreate. *Is there a flexible SLA 3D-printing resin?* URL: <https://www.liqcreate.com/supportarticles/elastic-flexible-sla-resin/>.
- [57] Formlabs. *Introducing Elastic Resin: A Soft, Resilient 3D Printing Material*. URL: <https://formlabs.com/uk/blog/elastic-resin-soft-resilient-3d-printing/>.
- [58] Formlabs. *100% Silicone 3D Printing Made Accessible: Introducing Silicone 40A Resin*. URL: <https://formlabs.com/blog/silicone-3d-printing-material-silicone-40a-resin/>.
- [59] deltarobotone. *How to Build Your Robot*. Repositório de código aberto; manual de construção do Delta-Robot One, incluindo ficheiros de impressão 3D e corte a laser da estrutura mecânica. 2019. URL: https://github.com/deltarobotone/how_to_build_your_robot.

- [60] isaac879. *Delta-Robot*. Repositório de código aberto; robô manipulador delta impresso em 3D controlado por Arduino Nano. 2019. URL: <https://github.com/isaac879/Delta-Robot>.
- [61] Luigi Arcieri et al. «The role of posterior aortopexy in the treatment of left mainstem bronchus compression». Em: *Interactive CardioVascular and Thoracic Surgery* 23.5 (2016), pp. 699–704. DOI: [10.1093/icvts/ivw209](https://doi.org/10.1093/icvts/ivw209).
- [62] Charles S. Chung, Mustafa Karamanoglu e Sándor J. Kovács. «Duration of diastole and its phases as a function of heart rate during supine bicycle exercise». Em: *American Journal of Physiology-Heart and Circulatory Physiology* 287.5 (2004), H2003–H2008. DOI: [10.1152/ajpheart.00404.2004](https://doi.org/10.1152/ajpheart.00404.2004).
- [63] Sylvain Garnavault. *RAMP*. Biblioteca Arduino de código aberto para interpolação e suavização de movimento. 2016. URL: <https://github.com/siteswapjuggler/RAMP>.
- [64] ArminJo. *ServoEasing*. Biblioteca Arduino de código aberto para suavização de movimento de servo-motores. 2019. URL: <https://github.com/ArminJo/ServoEasing>.
- [65] William Bailes. *Delta-Kinematics-Library*. Biblioteca Arduino de código aberto para cinemática direta e inversa de manipuladores delta. 2018. URL: <https://github.com/tinkersprojects/Delta-Kinematics-Library>.
- [66] Alex Nikanorov. *Delta robot kinematics*. Tutorial online, HyperTriangle. URL: <https://hypertriangle.com/~alex/delta-robot-tutorial/>.
- [67] ItayMiz e comunidade Stack Overflow. *Resposta sobre controlo manual dos sinais DTR/RTS para evitar resets indesejados de placas Arduino/ESP32 durante a abertura da porta série*. URL: <https://stackoverflow.com/a/68914329>.
- [68] P. J. Zsombor-Murray. *Descriptive Geometric Kinematic Analysis of Clavel's "Delta" Robot*. McGill University, Centre for Intelligent Machines, 2004. URL: <https://cim.mcgill.ca/~paul/clavdelt.pdf>.
- [69] M. Hasanlu e M. Siavashi. «Optimum kinematic–dynamic performance of the reconfigurable delta robot through genetic algorithm optimization». Em: *Robotic Systems and Applications* 5.1 (2025). DOI: [10.21595/rsa.2025.24731](https://doi.org/10.21595/rsa.2025.24731).

Apêndice

Lista de materiais

A tabela A.1 resume os componentes principais previstos para o sistema.

Tabela A.1: Componentes principais previstos para o sistema.

Componente	Função	Observações
Raspberry Pi 4B	Interface e coordenação	Envia comandos ou posições para o ESP32
ESP32-S3	Controlo dos atuadores	Executa FSMs e gera comandos para servos
Servo-motores	Atuação mecânica	Seis unidades distribuídas pelos dois manipuladores delta
Driver Servo-motores	Interface de potência/comando	Cada driver comanda três servo-motores de um manipulador delta
UPS/HAT	Gestão de energia	Alimentação do Raspberry Pi e transição para bateria
Fonte de Alimentação 5 V–10 A	Alimentação principal	Recebe 230 V e fornece 5 V ao sistema
LiPo Battery 2100 mA 3,7 V	Alimentação de reserva	Bateria associada ao sistema UPS/HAT

Anexos

A.1 Código e configurações

Incluir excertos relevantes de firmware, ficheiros de configuração, mapas de pinos e instruções de carregamento.